



Advanced Topics in Deep Learning

MINI-PROJECT 03

MULTI-STEP TEMPERATURE FORECASTING WITH GRU AND TRANSFORMERS

António Cruz (140129), Cátia Brás (120093), Ricardo Kayseller (95813)

Table of Contents

1. Introduction	6
2. Methodology	7
2.1 Computational Environment	7
2.2 Dataset Description	7
2.3 Data Acquisition and Initial Cleaning	8
2.5 Exploratory Data Analysis (EDA)	8
2.5.1 Dataset Structure	9
2.5.2 Missing Values	9
2.5.3 Duplicate Records	9
2.5.3.1 Drop Duplicated Values	9
2.5.4 Descriptive Statistics	9
2.5.5 Time-Series Visualization	10
2.5.6 Identification of Numerical Variables	11
2.5.7 Distribution Analysis (Histograms)	12
2.5.8 Boxplots	17
2.5.9 Outlier Detection	22
2.5.10 Correlation Between Variables	22
2.5.11 Correlation with Target Variable (Temperature)	24
2.5.12 Temporal Evolution of Temperature	25
2.5.13 Mean Temperature by Hour of Day	26
2.5.14 Annual Seasonality	27
2.5.15 Relationship Between the Target Variable and the Remaining Features	27
2.5.16 Target Mean Across Binned Variables	32
2.5.17 Variables Mean by Hour of Day	38
3. Data Understanding – Detailed Analysis	40
3.1 Temporal Regularity and Gaps	40
3.2 Seasonal Patterns	41
3.2.1 Stationarity (Rolling Mean and Standard Deviation)	41
3.2.2 Preview of Hourly Resampling	41
3.3 Identification of Missing Hourly Observations	43
3.3.1 Removal of Missing Hourly Rows	43
4. Data Preparation	44
4.1 Dataset Scope and Columns	44
4.1.1 Define Target, Inputs and Frequency	44
4.1.2 Sanity Checks	44

4.1.3 Confirm Frequency is Hourly	44
4.1.4 Set Date Time as Index	45
4.2 Temporal Split (Train / Validation / Test)	45
4.2.1 Split Proportions	45
4.2.2 Contiguous Temporal Splits.....	46
4.2.3 Visual target over splits	46
4.3 Feature Engineering	47
4.4 Feature Scaling	47
4.5 Windowing (L = 120, H = 24)	48
4.5.1 Baseline: Persistence.....	48
4.5.2 Evaluation Metrics	48
4.5.3 MAE by Horizon.....	49
5. GRU	50
5.1 Reproducibility Setup	50
5.2 Model Architecture and Hyperparameters	50
5.3 Training Wrapper and Callbacks.....	51
5.4 Hyperparameter Optimization with Optuna.....	52
5.5 Hyperparameter Optimization Run	53
5.6 Best Model Configuration	54
5.6.1 Analysis of Hyperparameter Trials	55
5.7 Training Curves (Best GRU)	57
5.8 Evaluation of Test Set (Scaled)	58
5.9 Inverse Scaling and Metrics in °C.....	59
5.9.1 Metrics in °C	59
5.10 MAE by Horizon	60
6. Overview of Hyperparameter Configurations	61
6.1 Distribution of Validation Performance	62
6.2 Hyperparameter Search Progress and Learning Rate Analysis	63
6.3 Performance vs Training Time	64
6.4 Important Hyperparameters	65
6.4.1 Effect of the Number of GRU Layers	65
6.4.2 Effect of Dropout Regularization	66
6.4.3 Effect of the Optimizer	67
7. Model Evaluation and Forecast Analysis	69
7.1 Horizon-wise Comparison: Baseline vs Best GRU	69
7.2 Error Distribution (Residual Analysis).....	70
7.3 One-Step-Ahead Forecast Visualization.....	71

7.4 Multi-Step Forecast Trajectories	72
7.5 Learning Curves of the Best Configurations.....	74
8. Forecasting.....	76
8.1 Generating a 24-Hour Temperature Forecast	76
8.2 Five-Day Forecasting.....	77
8.2.1 Five-Day Temperature Forecast	78
9. Transformer	79
9.1 Reproducibility Setup	79
9.2 Model Architecture and Hyperparameters	79
Patch-based Input Representation	79
Positional Encoding.....	80
Transformer Encoder Blocks	80
Output Representation	80
Training Configuration	80
9.3 Training Wrapper and Callbacks.....	81
9.4 Hyperparameter Optimization with Optuna.....	81
Focused Search Strategy	82
Architectural Constraints	82
Training and Pruning.....	82
9.5 Hyperparameter Optimization Run	83
9.5.1 Best Transformer Configuration	83
9.5.2 Analysis of Hyperparameter Trials	84
Model Depth	85
Embedding Size and Attention Heads	85
Patch Size	85
Dropout and Regularization	85
Learning Rate and Batch Size	85
9.5.3 Interpretation of the Optimization Behavior	86
9.6 Learning Curves of the Best Transformer Configuration	87
Training and Validation Loss	87
Training and Validation MAE.....	87
Convergence Behavior	87
9.7 Test Set Evaluation (Scaled Space).....	88
9.8 Convert Predictions to °C	89
9.8.1 Metrics in °C	89
9.9 MAE by Horizon	89
10. Overview of Transformer Hyperparameter Configurations.....	90

10.1 Histogram: Distribution of Validation Performance	91
10.2 Hyperparameter Search Progress and Learning Rate Analysis	92
Search Progress.....	92
Effect of Learning Rate	92
10.3 Scatter: Performance vs Training Time	93
10.4 Important Hyperparameters	94
10.4.1 Effect of the Number of Encoder Blocks.....	94
10.4.2 Effect of Patch Size	95
10.4.3 Effect of Dropout.....	96
10.4.4 Effect of the Number of Attention Heads.....	97
10.4.5 Effect of the Embedding Dimension (d_{model}).....	98
11. Horizon-wise Comparison.....	99
11.1 Error Distribution (Residuals) on Test ($^{\circ}\text{C}$)	100
11.2 Plot Actual vs Predicted for a Continuous Slice	101
11.3 Plot 24-hour Trajectories for a Few Examples	102
12. Forecasting	104
12.1 Forecasting for 1 Hour	104
12.2 Five-Day Forecasting	106
13. Limitations and Future Work	107
14. Conclusion	108
15. References.....	110

1. Introduction

Time series forecasting plays a central role in meteorology, environmental monitoring, and climate-aware decision-making. The availability of long-term, high-resolution atmospheric measurements enables the development of deep learning models capable of capturing complex temporal dependencies and nonlinear relationships in environmental systems.

This project focuses on the Jena Climate, a dataset containing a multivariate meteorological time series collected between 2009 and 2016 at the [Max Planck Institute for Biogeochemistry](#). The dataset contains 14 atmospheric variables recorded every 10 minutes, including temperature, pressure, humidity, and wind-related measurements.

However, in accordance with the project guidelines, only a subset of variables is used in the experiments to avoid near-deterministic relationships with the target variable. The input variables considered in this study are:

- Past air temperature (T), in $^{\circ}\text{C}$
- Atmospheric pressure (p), in mbar
- Relative humidity (rh), in %
- Wind speed (wv), in m/s
- Maximum wind speed ($max. wv$), in m/s
- Wind direction (wd), in degrees

Each variable is paired with its corresponding timestamp.

Following the project guidelines, the forecasting task is formulated as a multivariate-input, univariate-output, multi-step forecasting problem, where past meteorological observations are used to predict the future air temperature over a fixed horizon of 24 hours. The input variables include past air temperature together with atmospheric pressure, relative humidity, wind speed, maximum wind speed, and wind direction.

To address this problem, we implement and evaluate deep learning models designed for sequential data, namely Gated Recurrent Units (GRU) and Transformer-based architectures. The study follows a complete time series modelling pipeline including exploratory data analysis, data preprocessing, temporal resampling, windowing strategies, model development, hyperparameter exploration, and performance evaluation.

The final objective is not only to obtain accurate forecasts, but also to understand how different architectures and modelling choices influence forecasting performance, particularly across different forecast horizons.

2. Methodology

2.1 Computational Environment

The experiments were conducted using Python and a set of widely adopted scientific computing libraries. Data manipulation and preprocessing were performed using pandas and NumPy, while matplotlib was used for exploration visualizations. The Deep Learning models were implemented with TensorFlow/Keras, which provides efficient tools for constructing and training recurrent neural networks such as GRUs.

Additional utilities included scikit-learn for normalization and evaluation metrics (MAE, RMSE, R^2), Optuna for hyperparameter optimization, and standard Python modules such as random and time for reproducibility and runtime management.

This computational setup ensures a robust and reproducible environment for all stages of the forecasting pipeline.

2.2 Dataset Description

The dataset used in this project is the Jena Climate dataset, recorded by the [Max Planck Institute for Biogeochemistry](#). It contains 14 meteorological variables measured every 10 minutes over the period January 10, 2009, to December 31, 2016. According to the dataset documentation: “The dataset consists of 14 features recorded once per 10 minutes.

The measurements include atmospheric pressure, temperature, humidity, wind characteristics, and several derived thermodynamic variables. The table below summarizes the available features, their formats, and their physical meaning.

The table below shows the column names, their value formats, and their description.

Index	Features	Format	Description
1	Date Time	01.01.2009 00:10:00	Date-time reference
2	p (mbar)	996.52	The pascal SI derived unit of pressure used to quantify internal pressure
3	T (degC)	-8.02	Temperature in Celsius
4	Tpot (K)	265.4	Temperature in Kelvin
5	Tdew (degC)	-8.9	Temperature in Celsius in relation to humidity.
6	rh (%)	93.3	Relative Humidity is a measure of how saturated the air is with water vapor
7	VPmax (mbar)	3.33	Saturation vapor pressure
8	VPact (mbar)	3.11	Vapor pressure
9	VPdef (mbar)	0.22	Vapor pressure deficit
10	sh (g/kg)	1.94	Specific humidity

Index	Features	Format	Description
11	H2OC (mmol/mol)	3.12	Water vapor concentration
12	rho (g/m ** 3)	1307.75	Airtight
13	wv (m/s)	1.03	Wind speed
14	max. wv (m/s)	1.75	Maximum wind speed
15	wd (deg)	152.3	Wind direction in degrees

Table 1 - Jena Climate Dataset: Feature Overview

2.3 Data Acquisition and Initial Cleaning

The Jena Climate dataset was retrieved from the official TensorFlow/Keras repository, which provides a compressed CSV file containing the full time series. The data was downloaded and extracted programmatically using Python's zipfile module and Keras utilities.

After loading the dataset into a Pandas DataFrame, several preprocessing steps were applied:

- **Sensor error correction:** Wind speed columns (wv (m/s) and max. wv (m/s)) contained erroneous values of -9999.0, corresponding to equipment failures. These values were replaced with NaN to ensure they would be ignored during resampling and aggregation.
- **Datetime parsing:** The Date Time column was originally stored as a string in European format (DD.MM.YYYY HH:MM:SS). It was converted to a proper datetime object using `pd.to_datetime()` with an explicit format string. This ensures correct temporal indexing and compatibility with time-based operations.
- **Initial inspection:** The first few rows of the dataset were examined using `df.head(5)` to confirm successful loading and formatting. The output confirmed the presence of all expected variables, with realistic values and timestamps starting from January 1st, 2009.

These steps ensure that the dataset is clean, correctly formatted, and ready for further preprocessing and analysis.

2.5 Exploratory Data Analysis (EDA)

The exploratory data analysis aims to assess the structure, quality, and statistical behavior of the Jena Climate dataset before applying any modelling techniques. This includes inspecting dataset dimensions, identifying missing or duplicated values, analyzing variable distributions, detecting outliers, and examining correlations between meteorological variables.

2.5.1 Dataset Structure

The dataset contains 420,551 observations and 14 meteorological variables, together with a timestamp column (Date Time). As confirmed by the output of `df.shape`, the Date Time column is stored as a `datetime64` object, while all meteorological variables are numerical (`float64`).

The timestamp is used as the temporal reference for the series and allows operations such as indexing, resampling, and windowing. This structure is well suited for time-series modelling and facilitates the application of sequential deep learning models.

2.5.2 Missing Values

An inspection of missing values using `df.isnull().sum()` shows that the dataset is almost entirely complete.

Only two variables contain missing entries:

- `wv (m/s)`: 18 missing values
- `max. wv (m/s)`: 20 missing values

These missing values correspond to sensor failures originally encoded as `-9999.0`, which were intentionally replaced with `NaN` during the cleaning process.

No other variables contain missing data.

2.5.3 Duplicate Records

A total of 327 duplicated rows were identified using `df.duplicated().sum()`. Since each duplicated record appears twice when both the original and repeated entries are considered, this corresponds to 654 duplicated observations in total. Inspection of a sample of these records showed that the duplicated rows were exact copies, including identical timestamps and meteorological values.

These duplicates are likely due to repeated logging or data collection redundancies rather than meaningful repeated measurements. To ensure temporal consistency and avoid bias in the statistical analysis, resampling step, and model training, duplicated rows were removed using `drop_duplicates(keep="first")`.

After this operation, no duplicated rows remained in the dataset, and the final shape was reduced from 420,551 to 420,224 observations.

2.5.3.1 Drop Duplicated Values

```
Remaining duplicates: 0
New shape: (420224, 15)
```

2.5.4 Descriptive Statistics

Descriptive statistics were computed for all numerical variables using `df.describe().T` to obtain a first overview of the scale and variability of the meteorological measurements.

Key observations include:

- Temperature (T) ranges from -23.01°C to 37.28°C , reflecting substantial seasonal variability over the observation period.
- Relative humidity (rh) spans from 12.95% to 100%, indicating a wide range of atmospheric moisture conditions.
- Wind speed (wv) ranges from 0 to 28.49 m/s, suggesting generally low wind conditions with occasional high-speed events.
- Atmospheric pressure (p) varies between 913.60 mbar and 1015.35 mbar, which is consistent with typical pressure range observed in mid-latitude weather systems.

These statistics provide an initial understanding of the scale and variability of the meteorological variables and help guide subsequent preprocessing and modelling decisions.

Additionally, the statistics confirm that all variables fall within physically plausible ranges, indicating that no major data corruption or unrealistic values are present in the dataset.

2.5.5 Time-Series Visualization

To better understand the temporal behavior of the meteorological variables, raw time-series plots were generated for all features in the dataset. These visualizations allow the identification of long-term trends, seasonal patterns, and abrupt changes in atmospheric conditions.

The plots reveal clear annual seasonality in temperature-related variables (T, Tpot, Tdew), with warm summers and cold winters repeating consistently across the years. Humidity-related variables (rh, VPact, sh, H2OC) also exhibit seasonal fluctuations, typically increasing during colder months.

Wind-related variables (wv, max. wv, wd) show high short-term variability, reflecting the turbulent nature of wind dynamics. Atmospheric pressure (p) displays slower oscillations associated with synoptic-scale weather systems.

These temporal patterns confirm that the dataset contains rich and diverse dynamics, which must be captured by the forecasting models (figure 1).

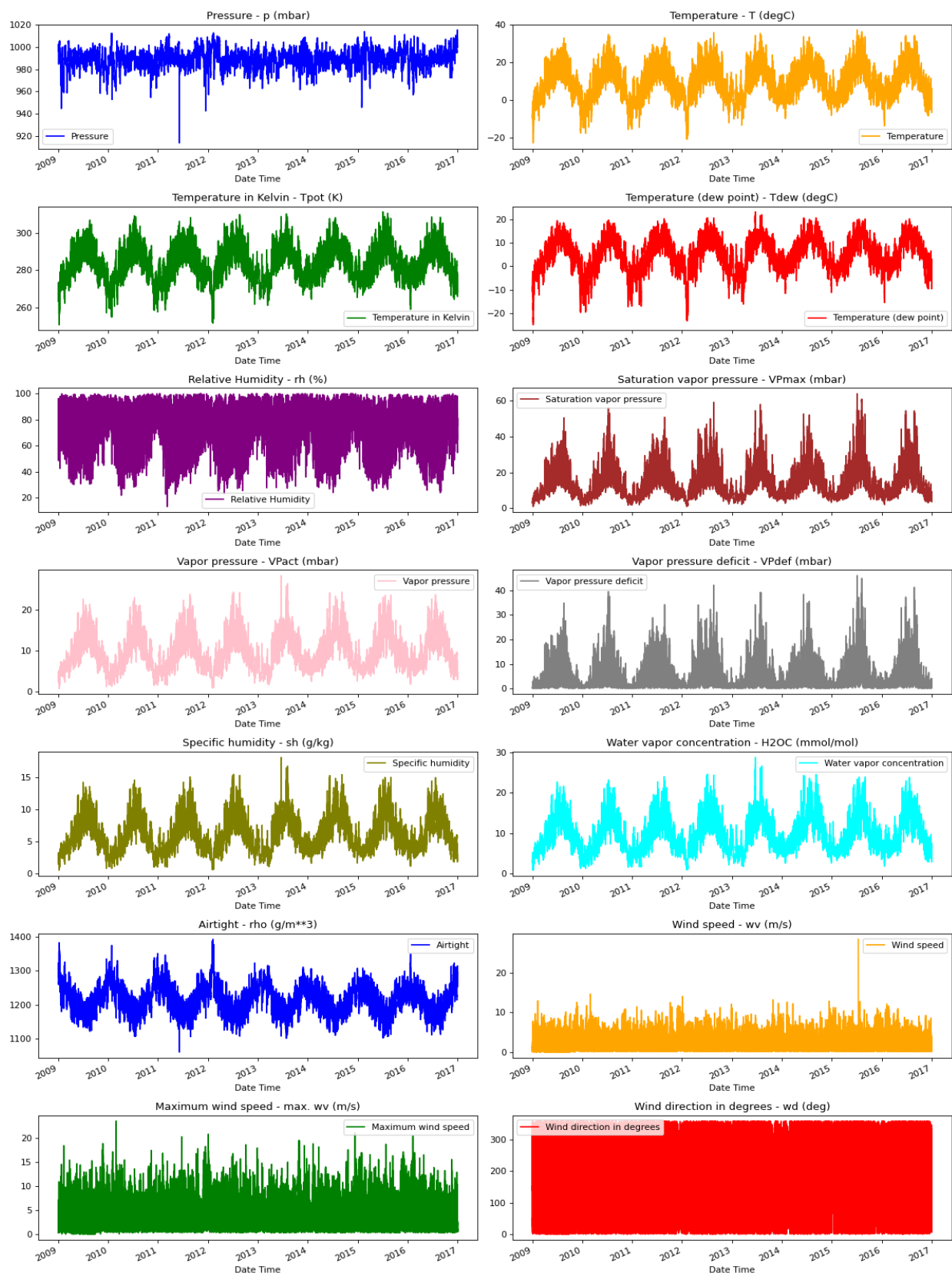


Figure 1 - Time-Series Overview of Jena Climate Variables

2.5.6 Identification of Numerical Variables

Before analyzing variable distributions, all numerical features in the dataset were identified using `df.select_dtypes(include=[np.number])`. A total of 14 numerical variables were

detected, corresponding to the meteorological measurements recorded by the weather station.

These variables include temperature-related features (T, Tpot, Tdew), humidity-related features (rh, VPact, sh, H2OC), pressure (p), wind-related variables (wv, max. wv, wd), and other atmospheric indicators. Identifying the numerical columns ensures that subsequent statistical and distributional analyses are applied consistently across all quantitative features.

2.5.7 Distribution Analysis (Histograms)

To analyze the empirical distribution of the meteorological variables, histograms were generated for all numerical features in the dataset. This step helps identify skewness, concentration of values, and potential extreme observations that may influence forecasting models.

The histograms reveal that many variables exhibit non-Gaussian distributions:

- Wind-related variables (wv, max. wv) show strong right-skewness due to occasional high-intensity wind events.
- Humidity-related variables (rh, VPact, sh, H2OC) tend to cluster near their physical limits, reflecting atmospheric saturation behavior.
- Temperature-related variables (T, Tpot, Tdew) exhibit broad distributions reflecting seasonal variability over the observation period.
- Atmospheric pressure (p) shows a narrow distribution with relatively low variance compared with other variables.

These observations highlight the importance of applying normalization before model training and motivate the use of deep learning models capable of capturing non-linear relationships in meteorological time series (figures 2 - 15).

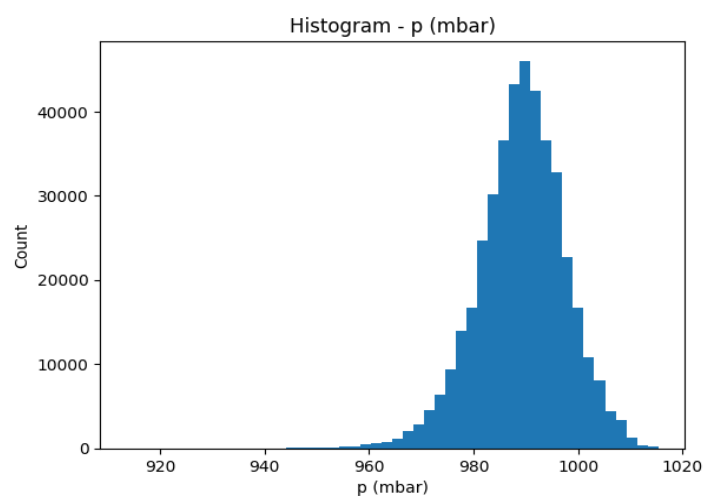


Figure 2 - Distribution of Atmospheric Pressure (p) in the dataset

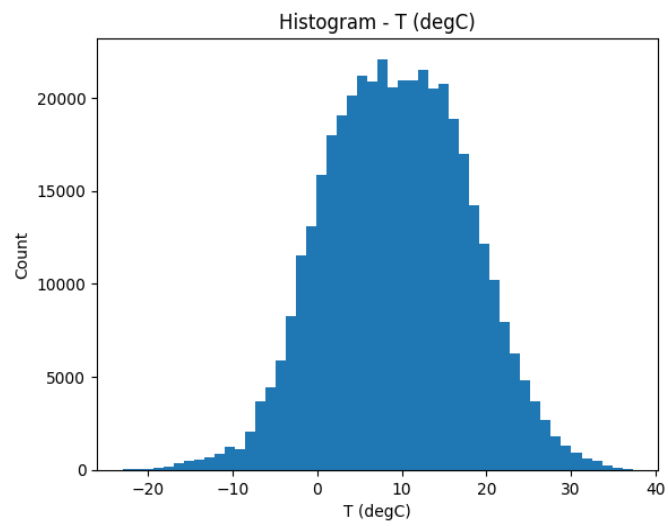


Figure 3 - Distribution of Air Temperature (T) in the dataset

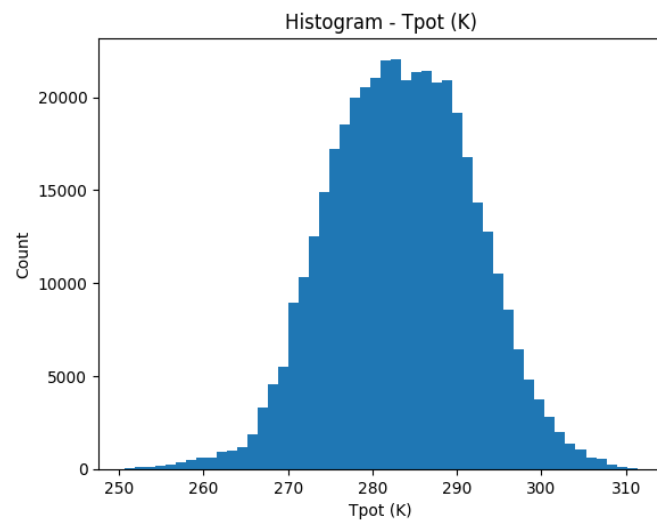


Figure 4 - Distribution of Potential Temperature (Tpot) in the dataset

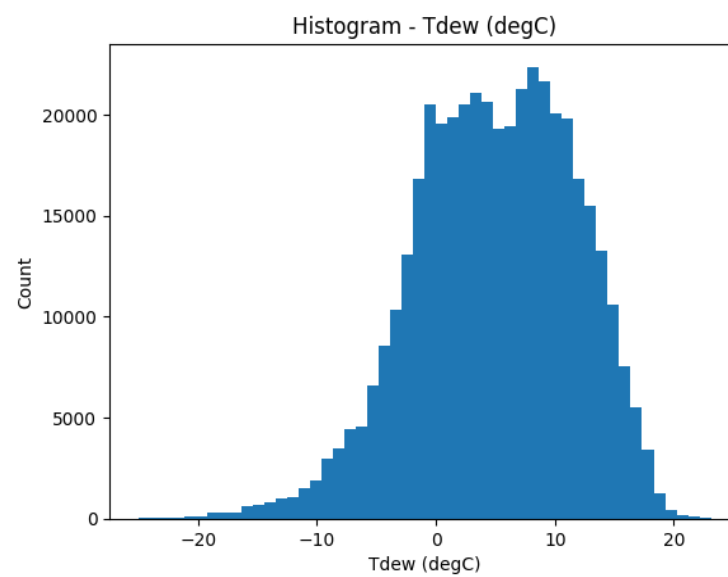


Figure 5 - Distribution of Dew Point Temperature (Tdew) in the dataset

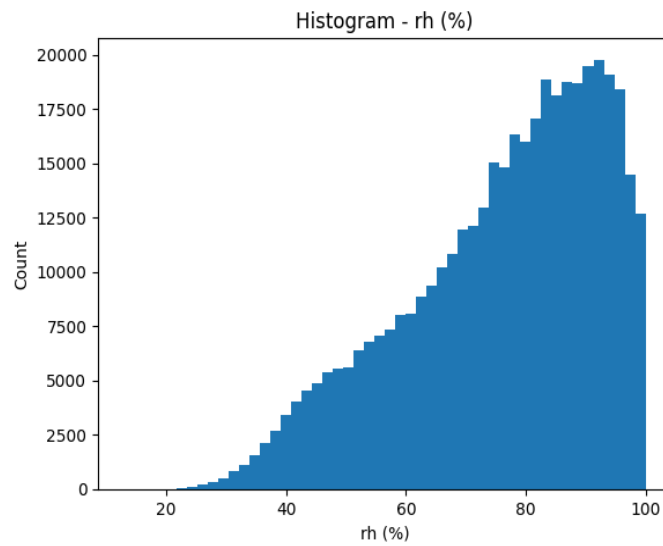


Figure 6 - Distribution of Relative Humidity (rh) in the dataset

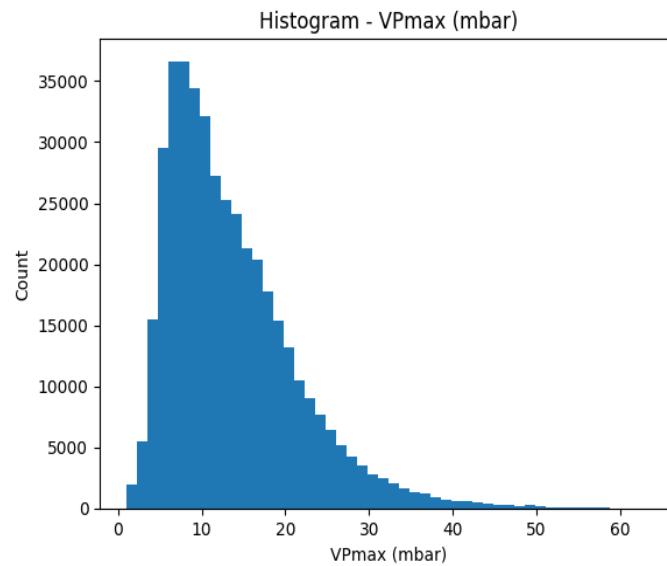


Figure 7 - Distribution of Maximum Vapor Pressure (VPmax) in the dataset

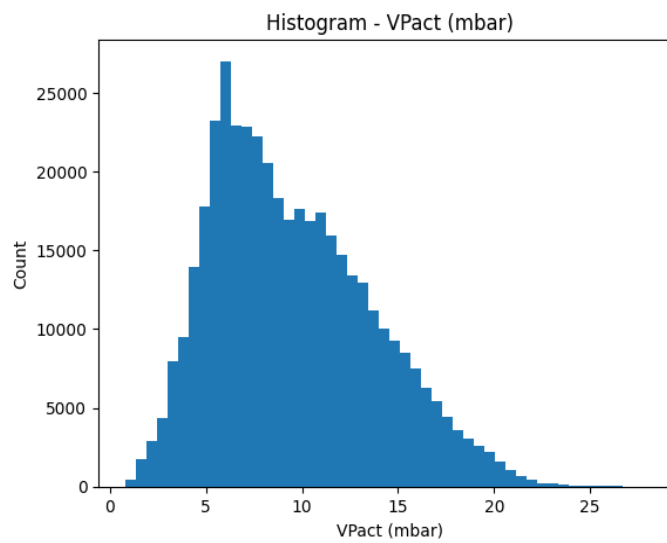


Figure 8 - Distribution of Actual Vapor Pressure (VPact) in the dataset

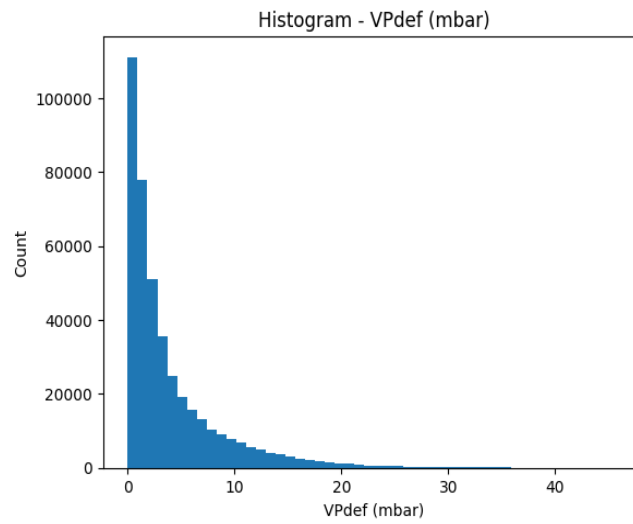


Figure 9 - Distribution of Vapor Pressure Deficit (VPdef) in the dataset

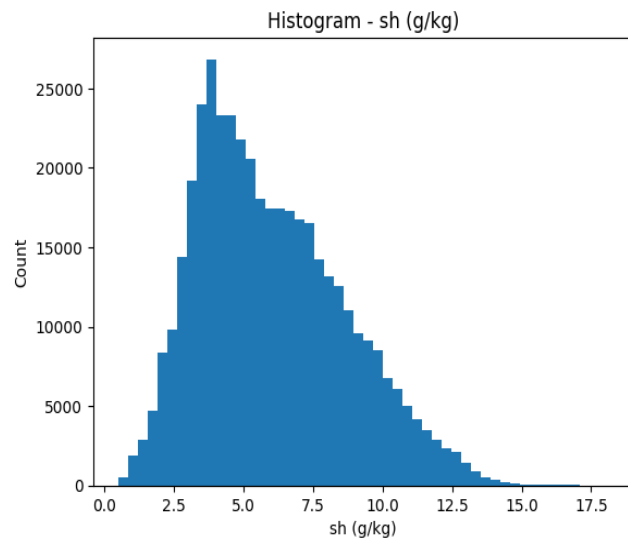


Figure 10 - Distribution of Specific Humidity (sh) in the dataset

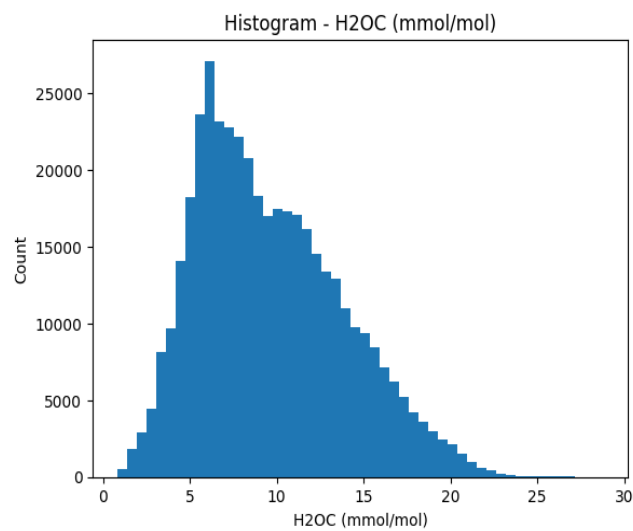


Figure 11 - Distribution of Water Vapor Concentration (H2OC) in the dataset

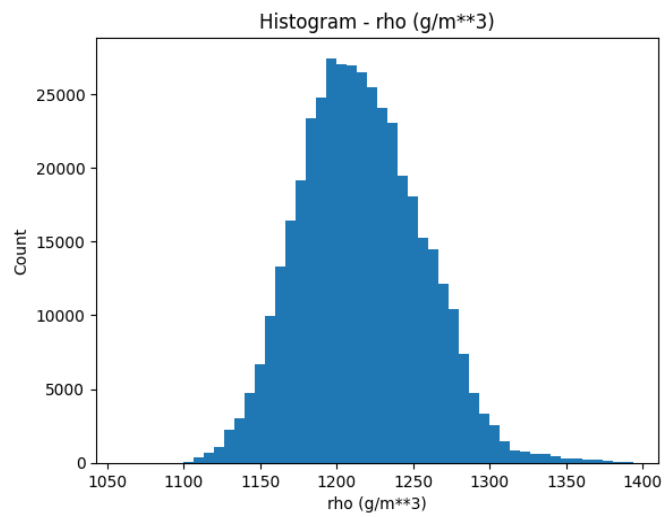


Figure 12 - Distribution of Air Density (rho) in the dataset

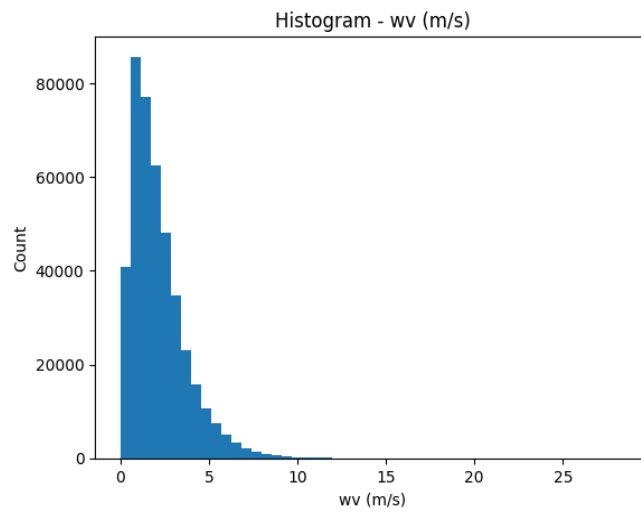


Figure 13 - Distribution of Wind Velocity (wv) in the dataset

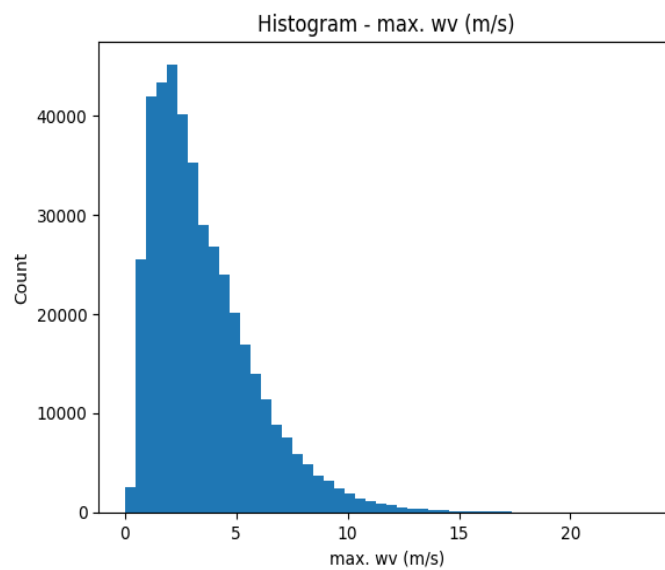


Figure 14 - Distribution of Maximum Wind Velocity (max. wv) in the dataset

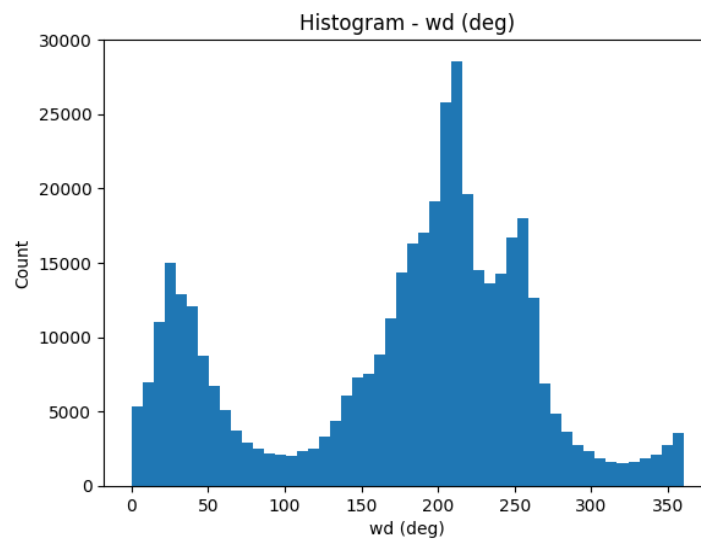


Figure 15 - Distribution of Wind Direction (wd) in the dataset

2.5.8 Boxplots

Boxplots were generated for all numerical variables to visualize their dispersion, central tendency, and potential extreme observations. These plots provide a compact summary of the median, interquartile range (IQR), and values outside the IQR range (figures 16 – 29).

Most variables exhibit a considerable number of statistical outliers according to the IQR criteria. In large meteorological datasets, such observations often correspond to rare but physically plausible weather events rather than measurement errors.

Wind-related variables (wv and max. wv) and vapor pressure deficit (VPdef) display particularly large ranges and numerous extreme observations, reflecting the intermittent nature of wind events and atmospheric moisture dynamics. In contrast, atmospheric pressure (p) shows a relatively narrow distribution with limited variability.

Overall, the boxplots confirm the presence of natural variability and occasional extreme values in the dataset, which is typical for environmental measurements.

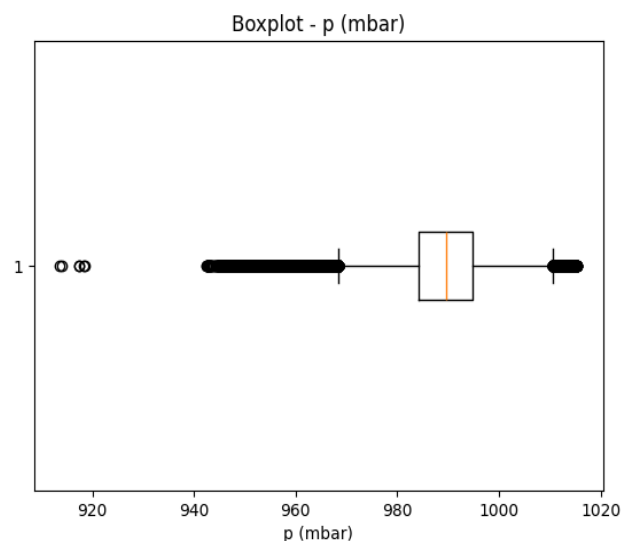


Figure 16 - Boxplot of Atmospheric Pressure (p) in the dataset

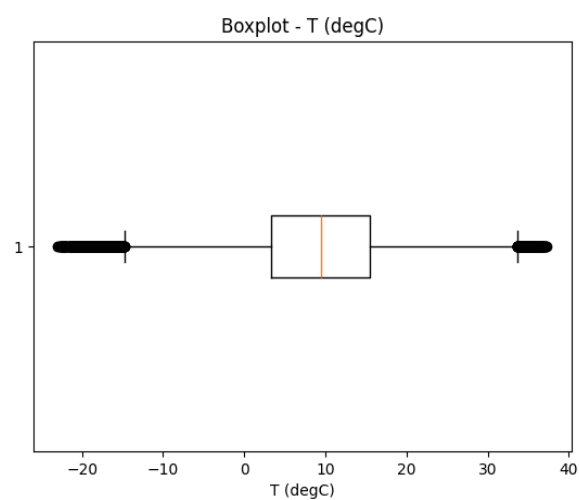


Figure 17 - Boxplot of Air Temperature (T) in the dataset

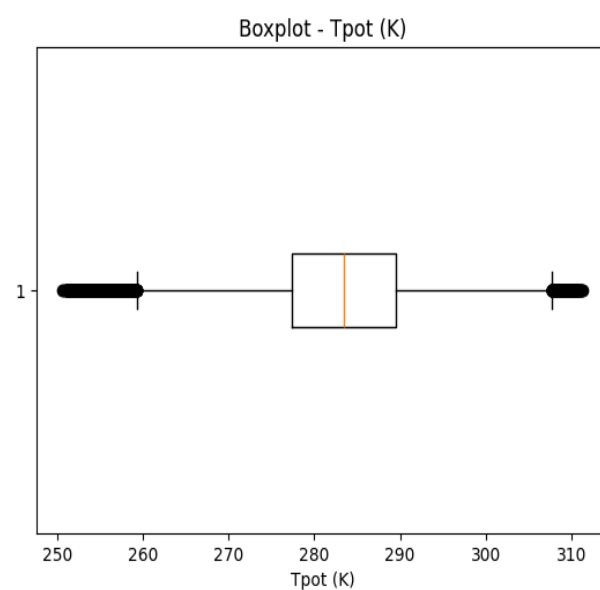


Figure 18 - Boxplot of Potential Temperature (Tpot) in the dataset

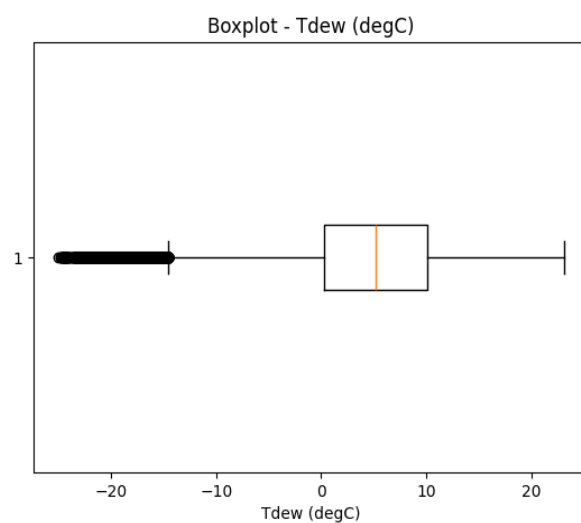


Figure 19 - Boxplot of Dew Point Temperature (Tdew) in the dataset

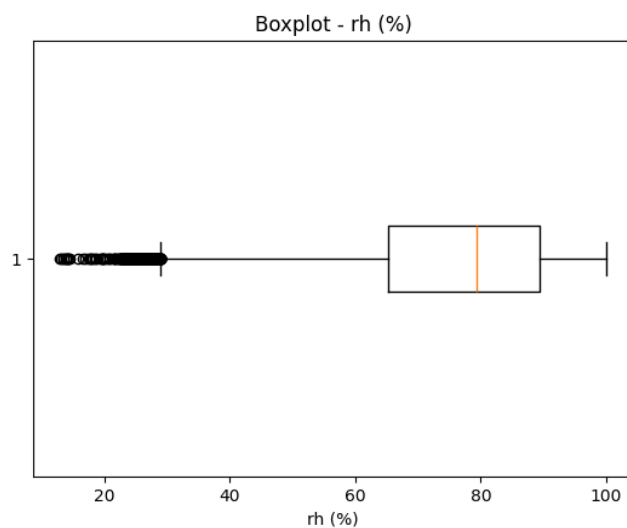


Figure 20 - Boxplot of Relative Humidity (rh) in the dataset

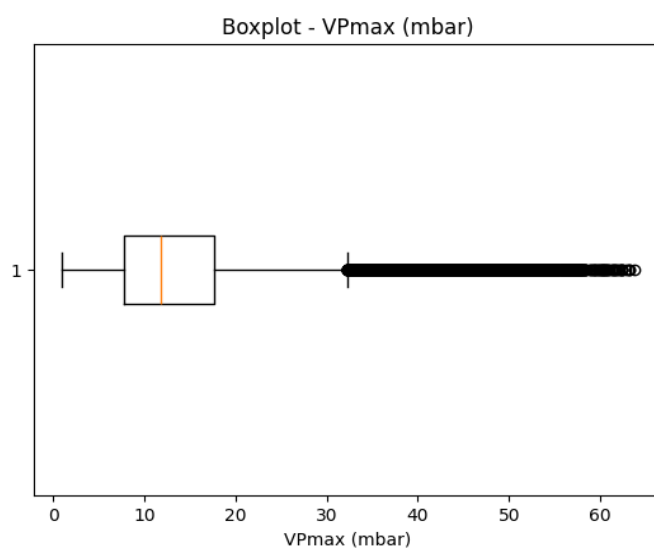


Figure 21 - Boxplot of Maximum Vapor Pressure (VPmax) in the dataset

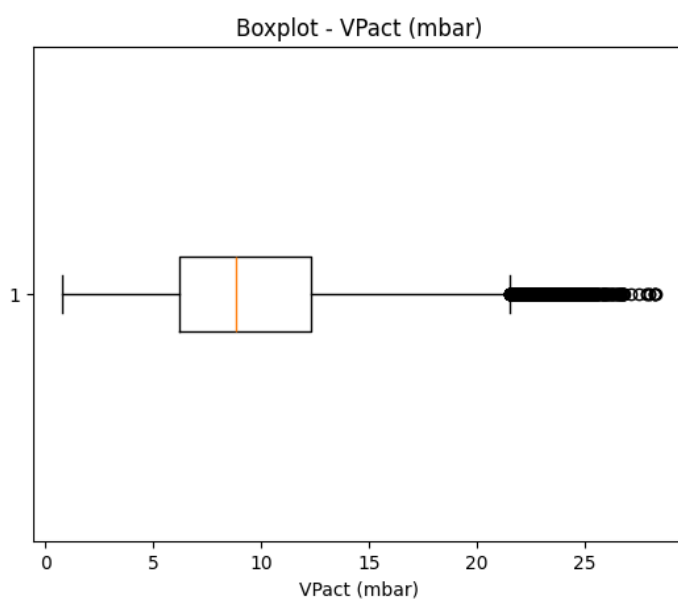


Figure 22 - Boxplot of Actual Vapor Pressure (VPact) in the dataset

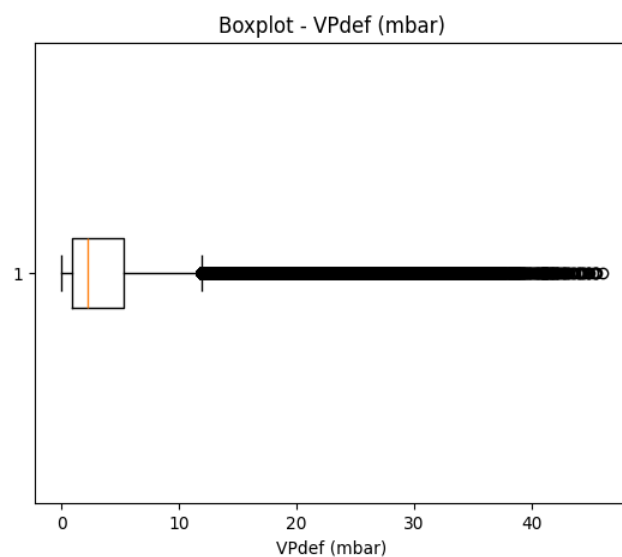


Figure 23 - Boxplot of Vapor Pressure Deficit (VPdef) in the dataset

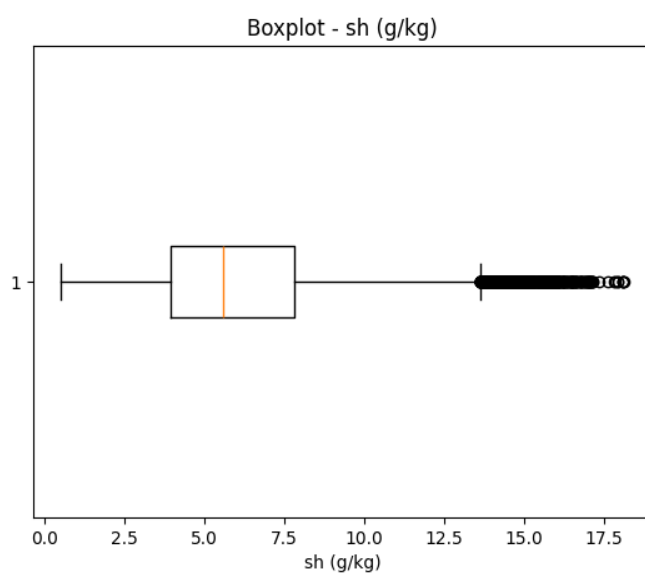


Figure 24 - Boxplot of Specific Humidity (sh) in the dataset

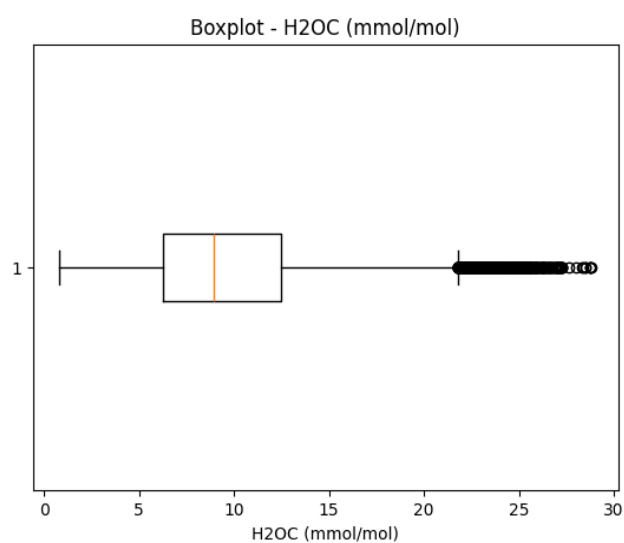


Figure 25 - Boxplot of Water Vapor Concentration (H2OC) in the dataset

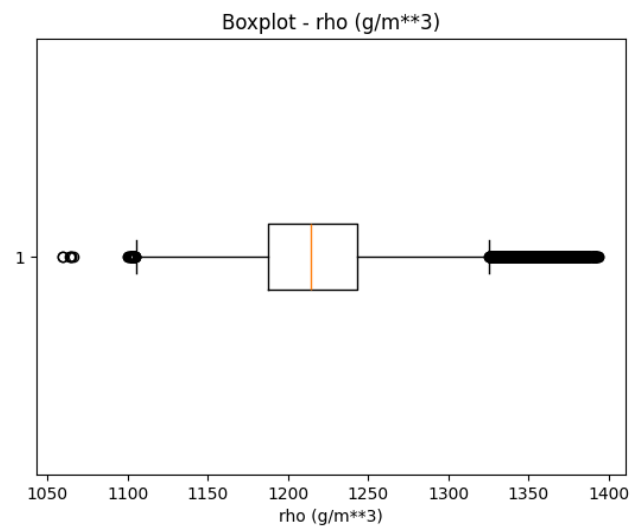


Figure 26 - Boxplot of Air Density (rho) in the dataset

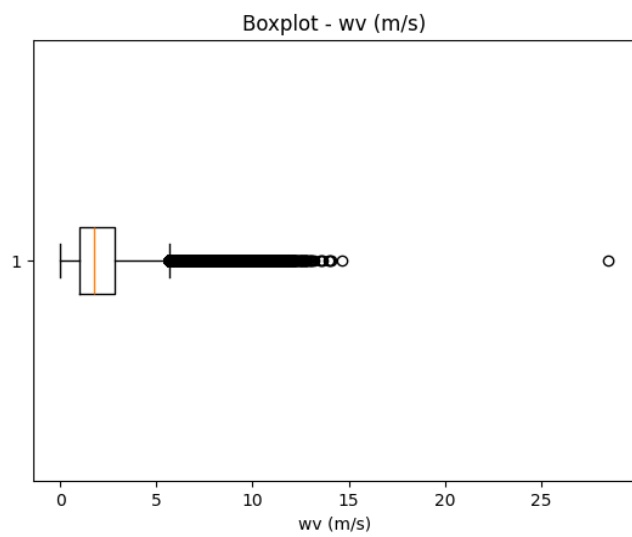


Figure 27 - Boxplot of Wind Velocity (wv) in the dataset

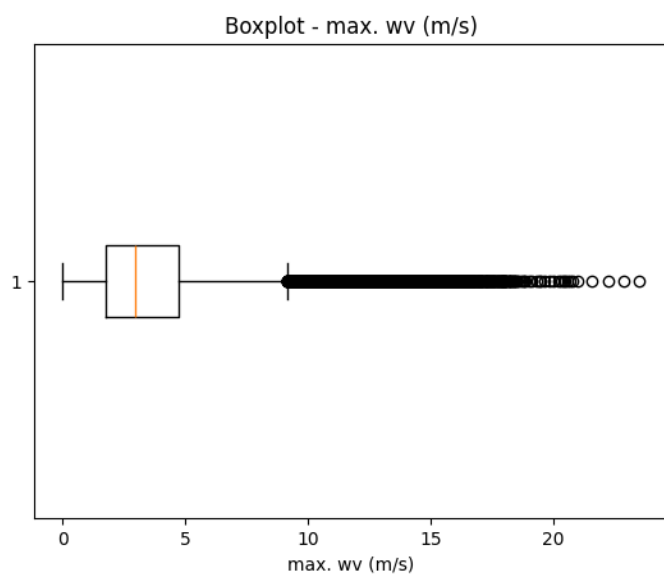


Figure 28 - Boxplot of Maximum Wind Velocity (max. wv) in the dataset

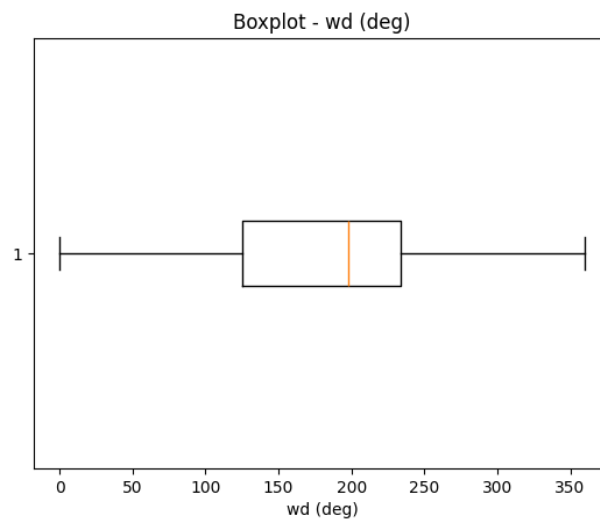


Figure 29 - Boxplot of Wind Direction (wd) in the dataset

2.5.9 Outlier Detection

Outliers were identified using the Interquartile Range (IQR) method. For each numerical variable, observations below $(Q1 - 1.5 \text{ IQR})$ or above $(Q3 + 1.5 \text{ IQR})$ were flagged as statistical outliers.

The variables with the highest number of detected outliers include:

- VPdef (mbar): approximately 32,000 observations
- vv (m/s) and max. vv (m/s): together more than 26,000 observations
- VPmax (mbar): over 11,000 observations
- p (mbar): approximately 6,700 observations

It is important to note that the IQR method identifies *statistical* outliers rather than erroneous measurements. In meteorological datasets, such values often correspond to rare but physically plausible atmospheric conditions, including strong wind events or extreme humidity variations.

Given the large size of the dataset and the fact that these observations fall within physically realistic ranges, no outlier removal was performed. Retaining these values preserves the natural variability of the atmospheric processes represented in the dataset.

2.5.10 Correlation Between Variables

A correlation matrix was computed to analyze linear relationships between the numerical variables. The resulting heatmap reveals several strong dependencies between atmospheric variables (figure 30).

Temperature-related variables (T, Tpot, Tdew) show strong positive correlations with vapor pressure and humidity-related variables such as VPmax, VPact, sh, and H2OC. These relationships are expected, since atmospheric moisture capacity is strongly influenced by temperature.

Air density (ρ) is strongly negatively correlated with temperature (≈ -0.96), which is consistent with the physical relationship described by the ideal gas law.

Wind-related variables (wv , max. wv , wd) exhibit relatively weak correlations with most other variables, indicating that wind dynamics behave more independently from temperature and humidity conditions.

The correlation matrix also reveals extremely strong correlations among several humidity-related variables, particularly between sh , H_2OC , and $VPact$, where correlations approach values close to 1. These near-deterministic relationships indicate a high degree of redundancy between certain variables.

Such redundancy between variables may introduce multicollinearity, which can negatively affect model interpretability and training stability. For this reason, and following the project guidelines, only a subset of meteorological variables is used during modelling to avoid introducing redundant predictors.

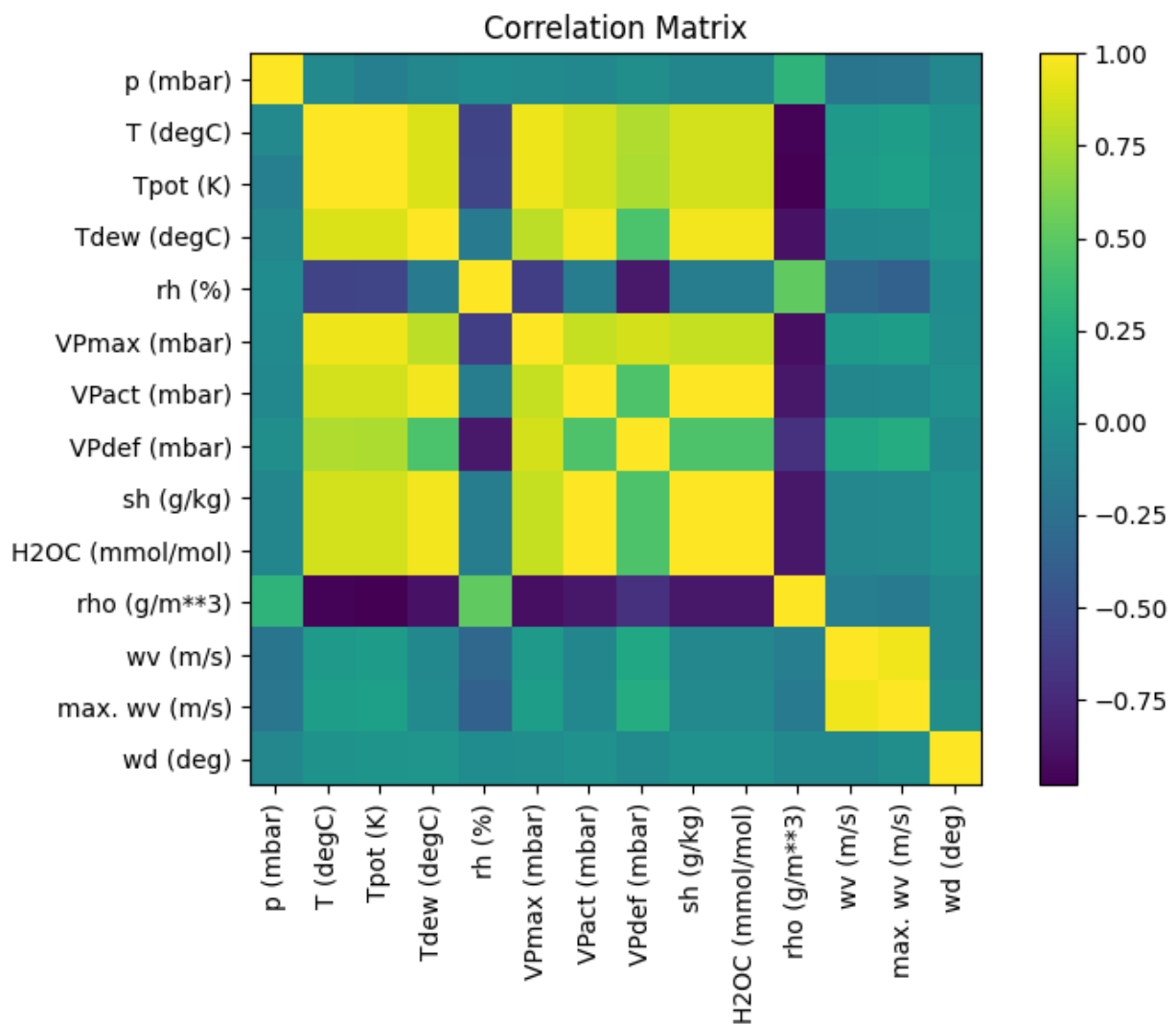


Figure 30 - Correlation Matrix of Meteorological Variables

2.5.11 Correlation with Target Variable (Temperature)

To better understand which variables are most informative for forecasting temperature, the correlation of each feature with the target variable T (degC) was computed (table 2).

The strongest positive correlations include:

- T_{pot} (K)
- VP_{max} (mbar)
- T_{dew} (degC)
- VP_{act} (mbar)
- sh (g/kg)
- H_2O_C (mmol/mol)

These strong positive correlations reflect the physical relationships between temperature, atmospheric moisture capacity, and vapor pressure dynamics.

Strong negative correlations include:

- ρ (g/m³)
- rh (%)

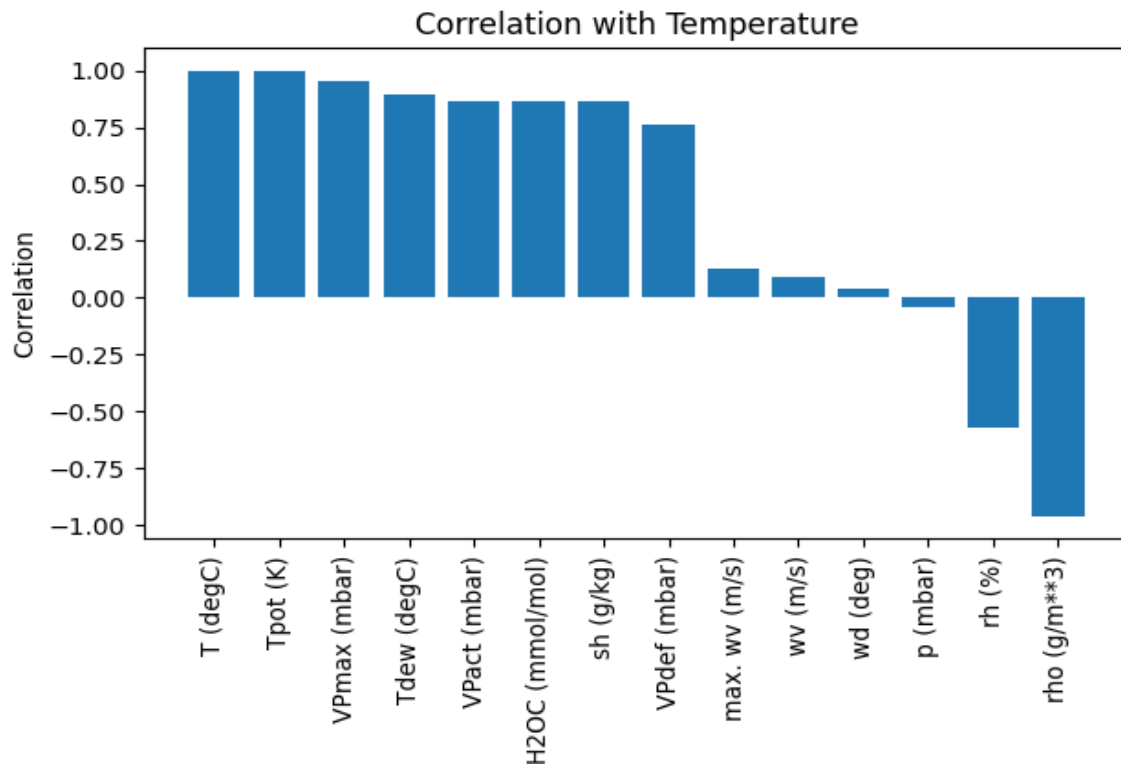
Specifically, air density (ρ) shows a very strong negative relationship with temperature (≈ -0.96), which is consistent with the ideal gas law (figure 31).

Wind-related variables (wv , max. wv , wd) exhibit weak correlations with temperature, suggesting they provide limited direct linear predictive power for this target variable. However, they may still contribute indirectly through nonlinear relationships captured by deep learning models.

It is also important to note that several highly correlated variables will not be used during model training to avoid redundancy, following the project guidelines.

Variable	Correlation with T (degC)
T (degC)	1.000
T_{pot} (K)	0.997
VP_{max} (mbar)	0.951
T_{dew} (degC)	0.896
VP_{act} (mbar)	0.868
H_2O_C (mmol/mol)	0.867
sh (g/kg)	0.867
VP_{def} (mbar)	0.762
max. wv (m/s)	0.125
wv (m/s)	0.088

Variable	Correlation with T (degC)
wd (deg)	0.039
p (mbar)	-0.045
rh (%)	-0.572
rho (g/m ³)	-0.963

Table 2 - Correlation of All Variables with Temperature (T (°C))**Figure 31** - Correlation with Temperature (Bar Chart)

2.5.12 Temporal Evolution of Temperature

A time-series plot of air temperature over the observation period (2009–2016) reveals clear seasonal patterns (figure 32). The data exhibits strong annual cycles, consistently warmer temperatures during summer months and colder temperatures during winter periods.

In addition to these long-term seasonal patterns, the series also shows substantial short-term variability, reflecting daily fluctuations and synoptic-scale weather dynamics.

The absence of a strong long-term trend suggests that the temperature dynamics are primarily driven by seasonal and short-term atmospheric processes rather than structural changes over time.

These observations confirm that the dataset contains pronounced temporal structure and periodic components, which must be captured by the forecasting models. Sequential deep learning architectures such as recurrent neural networks and transformers are particularly well suited for modelling such temporal dependencies.

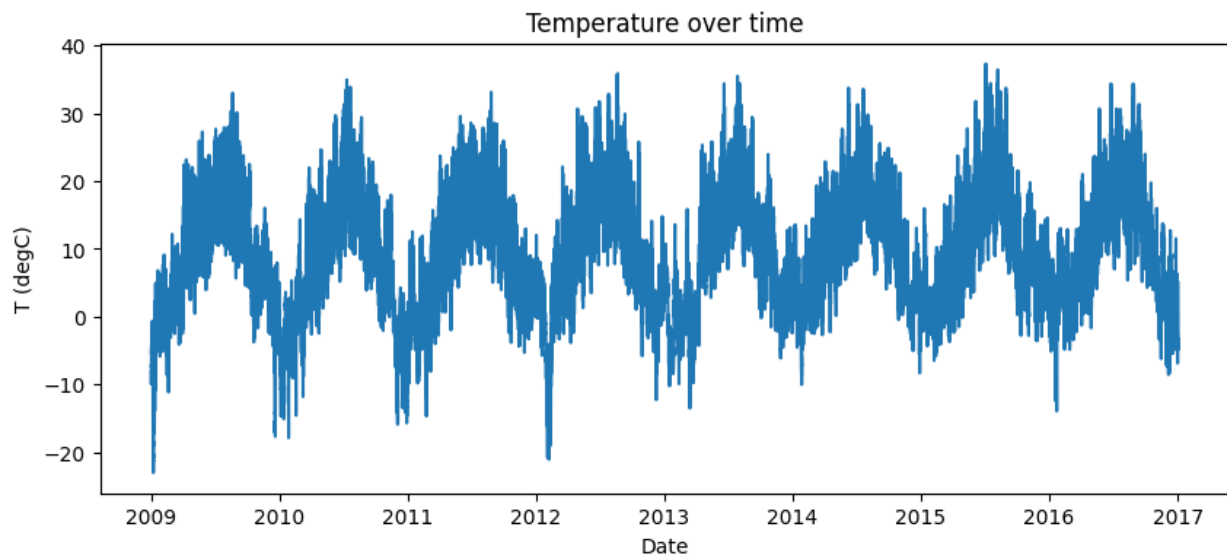


Figure 32 - Temperature Time Series (2009-2017)

2.5.13 Mean Temperature by Hour of Day

To analyze diurnal patterns, the average temperature was computed for each hour of the day. The resulting profile reveals a clear daily cycle in temperature (figure 33).

Minimum temperatures occur during the early morning hours, followed by a steady increase throughout the morning. Peak temperatures are observed in the mid-afternoon, after which temperatures gradually decline during the evening and nighttime hours.

This pattern confirms the presence of strong daily periodicity in the dataset. Such cyclic behavior is particularly relevant for short-term forecasting and motivates the inclusion of time-based features (e.g., hour-of-day encodings) to help the model capture daily temperature dynamics.

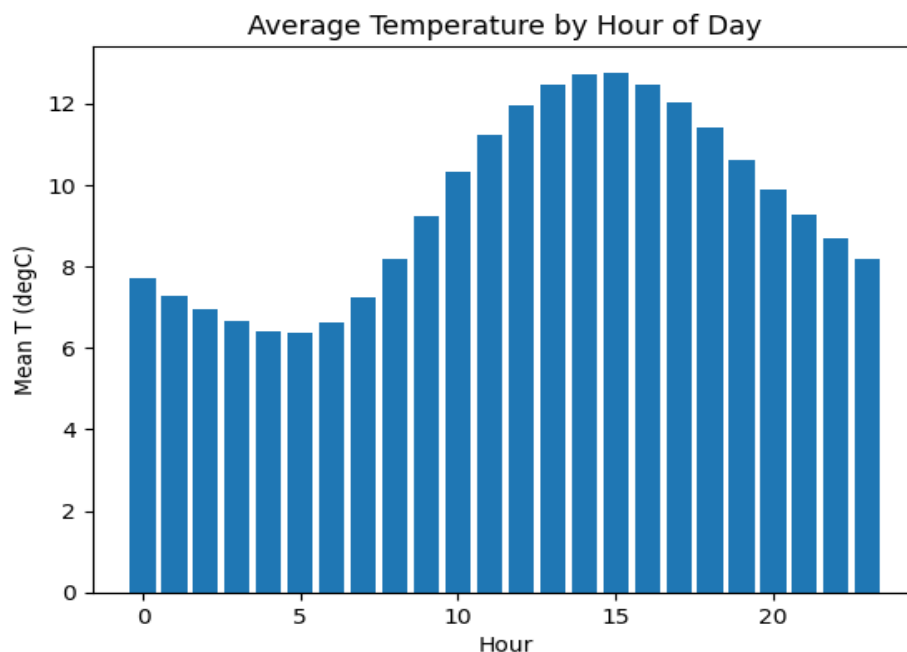


Figure 33 - Average Temperature by Hour of Day

2.5.14 Annual Seasonality

To analyze long-term seasonal behavior, the average temperature was computed for each month of the year. The results show a clear annual cycle, with minimum temperatures occurring in winter (January and December) and maximum temperatures during summer (July and August) (figure 34). This confirms the presence of strong yearly seasonality in the dataset, which is relevant for medium- and long-term forecasting.

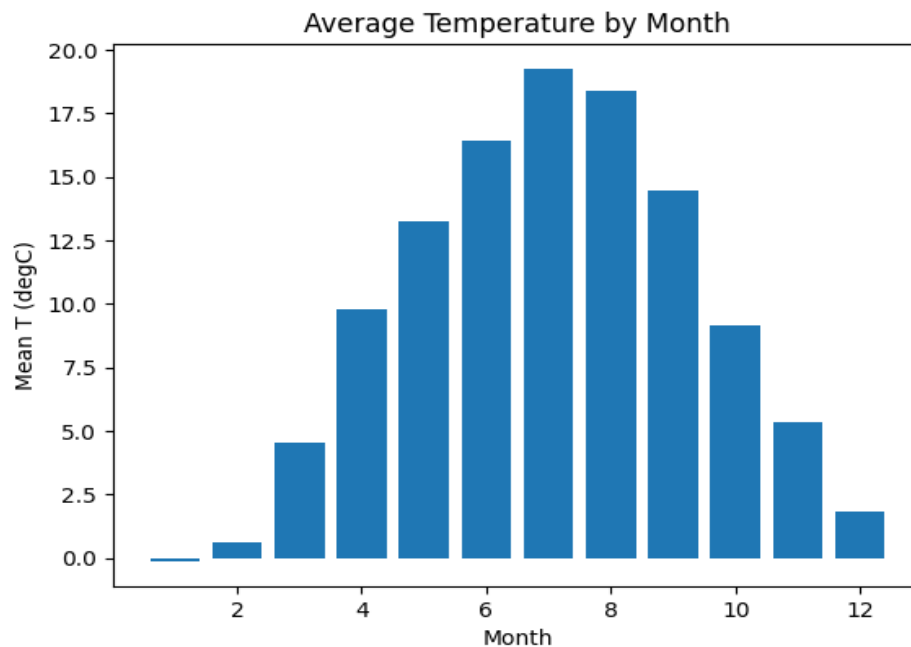


Figure 34 - Average Temperature by Month

2.5.15 Relationship Between the Target Variable and the Remaining Features

Scatter plots were generated to visualize the relationship between the target variable, T (degC), and the remaining numerical features. These plots help assess the strength, direction, and functional form of the dependencies.

Very strong relationships were observed for variables such as T_{pot} (K), T_{dew} (degC), VP_{max} (mbar), and ρ (g/m³). T_{pot} (K) shows an almost perfectly linear positive relationship with temperature, while ρ (g/m³) exhibits a very strong negative relationship, consistent with the physical behavior of air density.

Other variables, such as VP_{max} (mbar), VP_{act} (mbar), sh (g/kg), and H_2OC (mmol/mol), display strong but clearly non-linear relationships with temperature. These patterns reflect the physical dependence between temperature and atmospheric moisture-related quantities.

In contrast, wind-related variables (wv , $max. wv$, and wd) show much more diffuse scatter patterns, suggesting weaker direct linear relationships with the target variable.

Overall, these plots indicate that temperature is strongly related to several atmospheric variables, often through non-linear dependencies (figures 35 – 47). This supports the use of

deep learning models capable of capturing complex relationships beyond simple linear effects.

Although all variables were analyzed during exploration data analysis, only the subset specified in the project guidelines was retained for modelling.

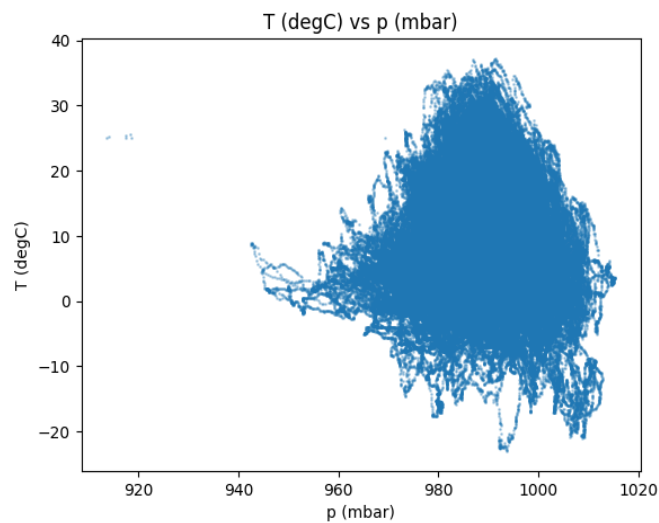


Figure 35 - Relationship Between Air Temperature (T) and Atmospheric Pressure (p)

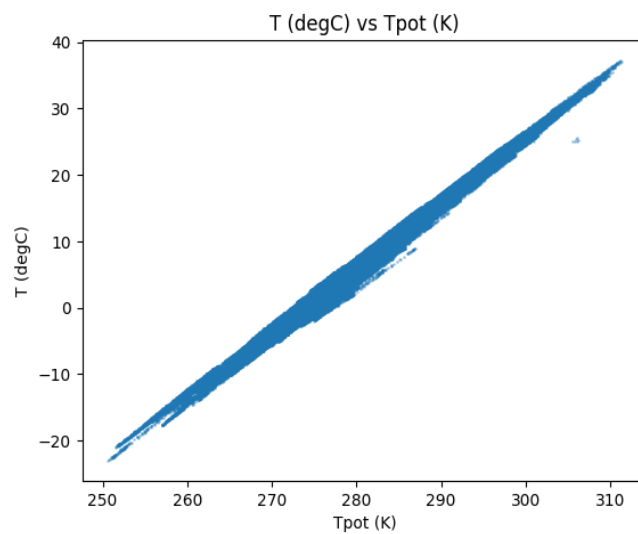


Figure 36 - Relationship Between Air Temperature (T) and Potential Temperature (Tpot)

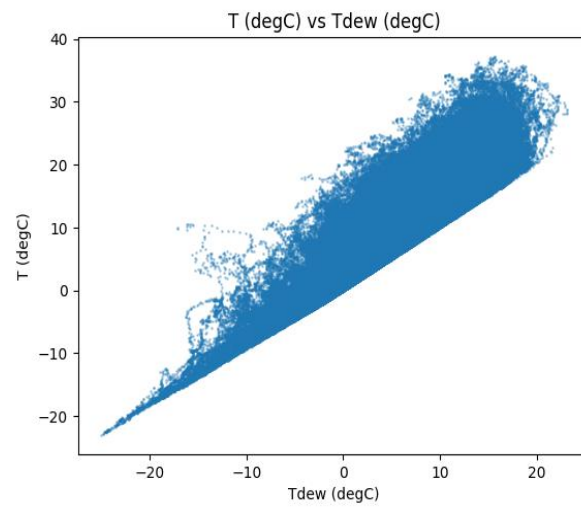


Figure 37 - Relationship Between Air Temperature (T) and Dew Point Temperature (Tdew)

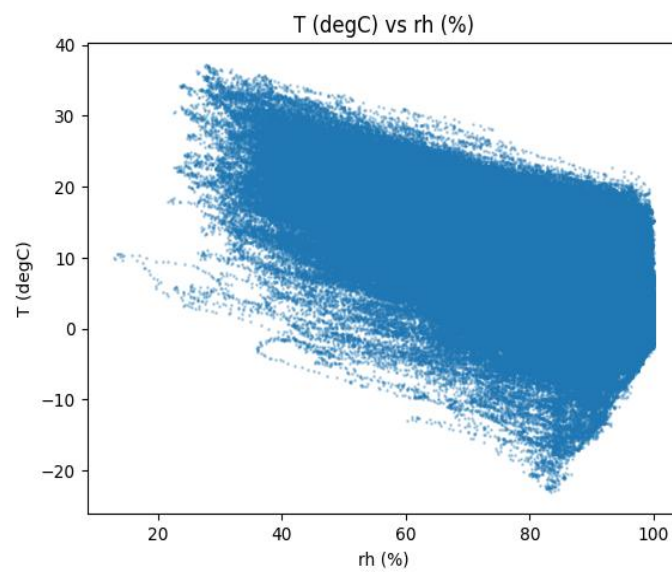


Figure 38 – Relationship Between Air Temperature (T) and Relative Humidity (rh)

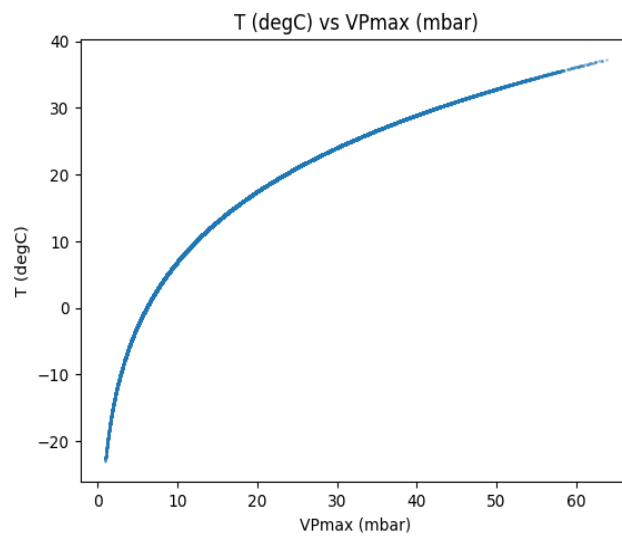


Figure 39 - Relationship Between Air Temperature (T) and Maximum Vapor Pressure (VPmax)

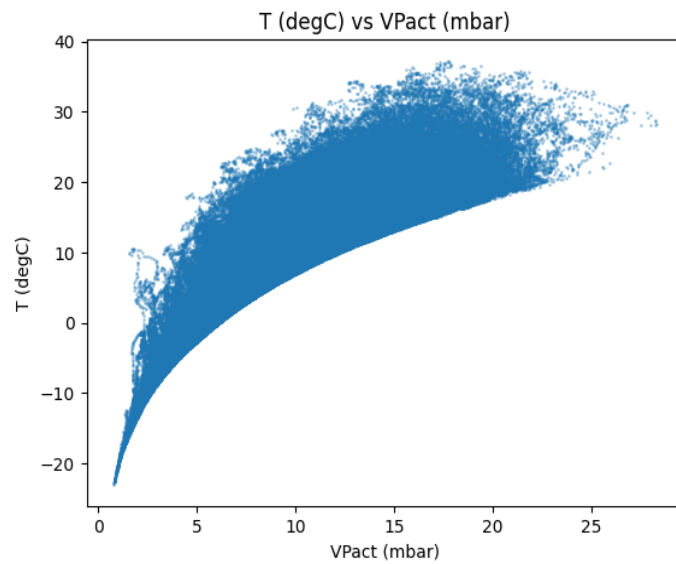


Figure 40 - Relationship Between Air Temperature (T) and Actual Vapor Pressure (VPact)

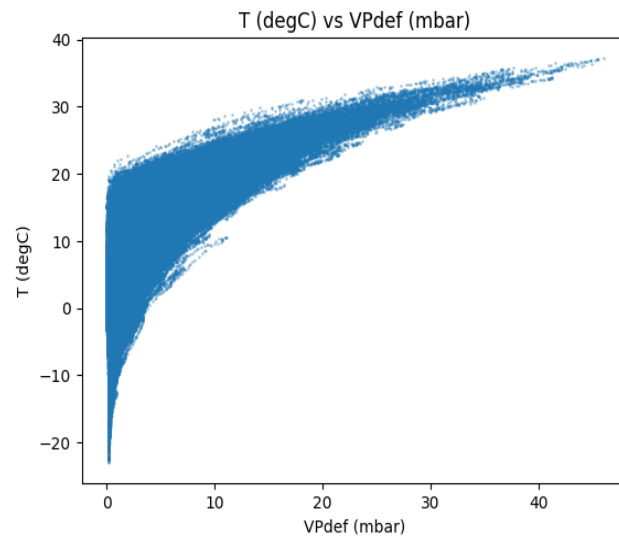


Figure 41 - Relationship Between Air Temperature (T) and Vapor Pressure Deficit (VPdef)

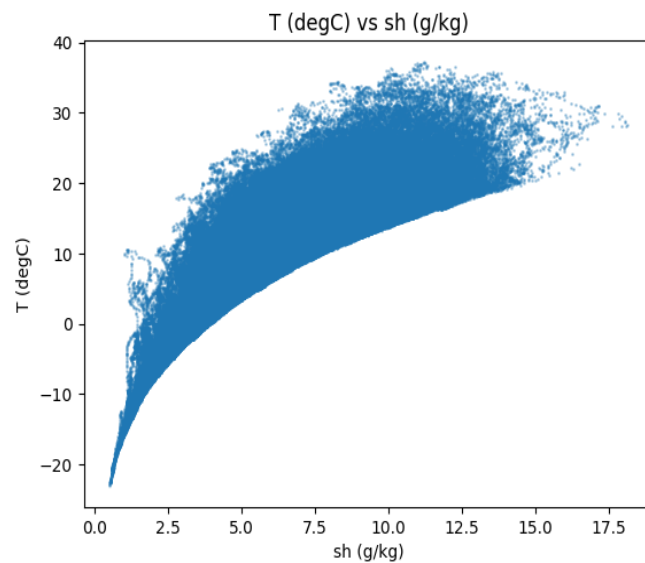


Figure 42 - Relationship Between Air Temperature (T) and Specific Humidity (sh)

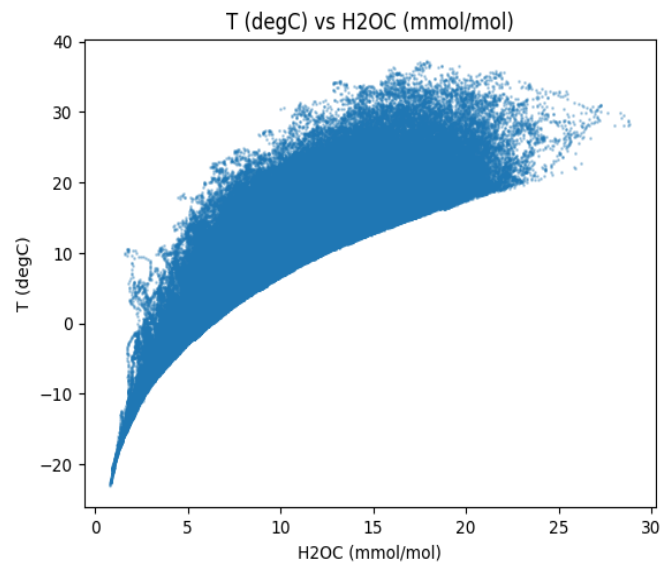


Figure 43 - Relationship Between Air Temperature (T) and Water Vapor Concentration (H2OC)

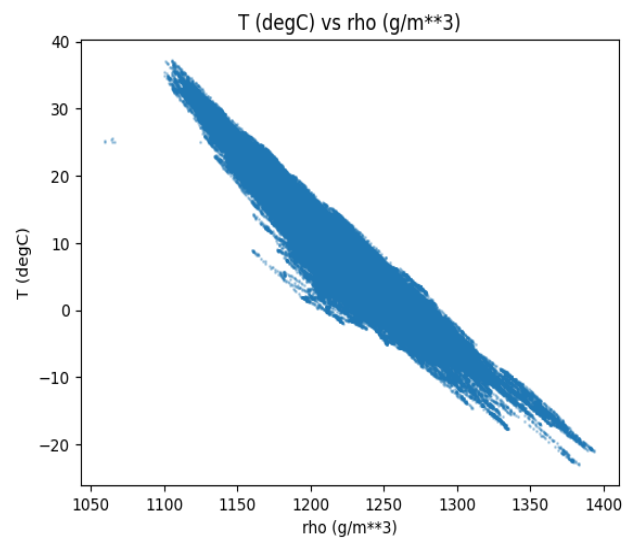


Figure 44 - Relationship Between Air Temperature (T) and Air Density (rho)

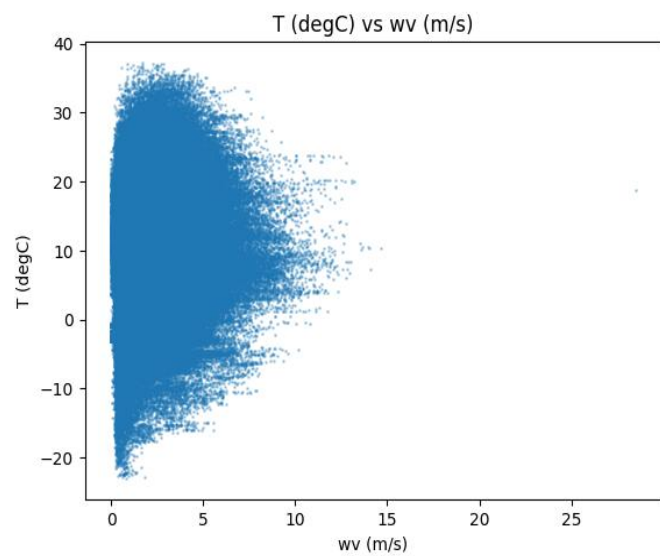


Figure 45 - Relationship Between Air Temperature (T) and Wind Velocity (wv)

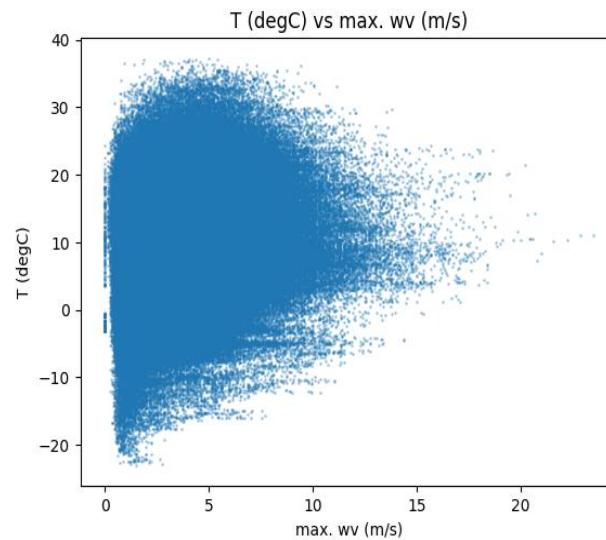


Figure 46 - Relationship Between Air Temperature (T) and Maximum Wind Velocity (max. wv)

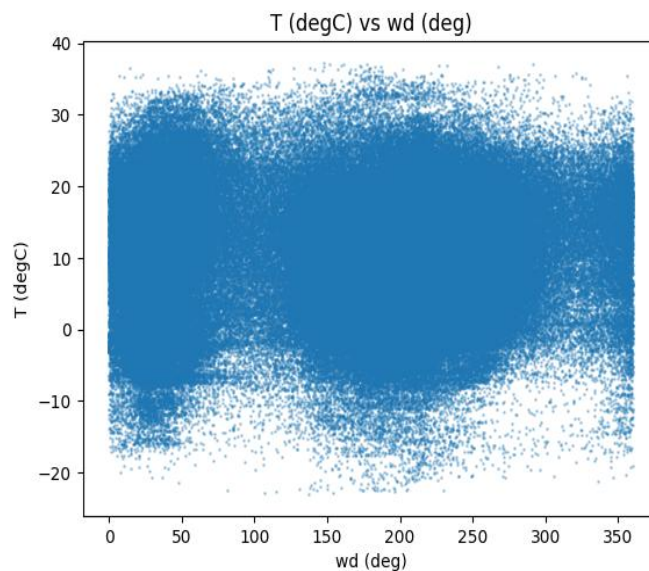


Figure 47 - Relationship Between Air Temperature (T) and Wind Direction (wd)

2.5.16 Target Mean Across Binned Variables

To further analyze the functional relationship between temperature and the remaining variables, each feature was divided into quantile-based bins and the mean temperature was computed within each bin (figures 48 – 60). This approach helps reveal the underlying trend between variables while reducing the effect of noise present in raw scatter plots.

Several clear patterns emerge:

- Tpot (K) and Tdew (degC) show almost perfectly linear relationships with temperature.
- VPmax (mbar), VPact (mbar), sh (g/kg), and H2OC (mmol/mol) exhibit strong but clearly non-linear increasing relationships with temperature.
- rho (g/m^3) shows a strong monotonic negative relationship with temperature, consistent with the physical relationship between temperature and air density.

- Relative humidity (rh) displays a decreasing relationship with temperature.
- Wind-related variables (wv, max. wv, wd) show relatively weak and noisy patterns, suggesting limited direct predictive power.

These results confirm that temperature is strongly related to several atmospheric variables, often through non-linear relationships. This further motivates the use of deep learning models capable of capturing complex non-linear dependencies in time series forecasting.

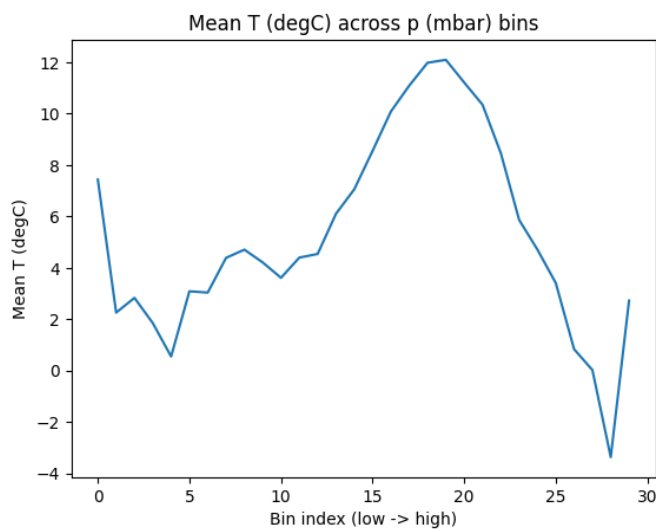


Figure 48 - Mean Air Temperature (T) across Atmospheric Pressure (p) bins

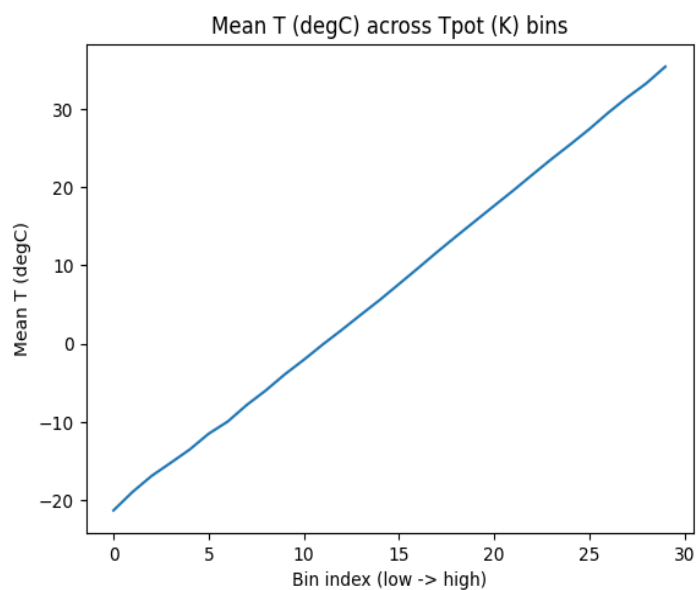


Figure 49 - Mean Air Temperature (T) across Potential Temperature (Tpot) bins

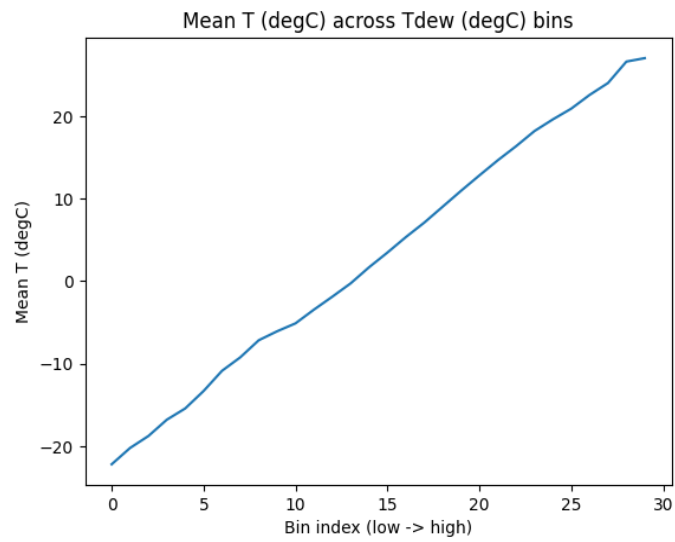


Figure 50 - Mean Air Temperature (T) across Dew Point Temperature (Tdew) bins

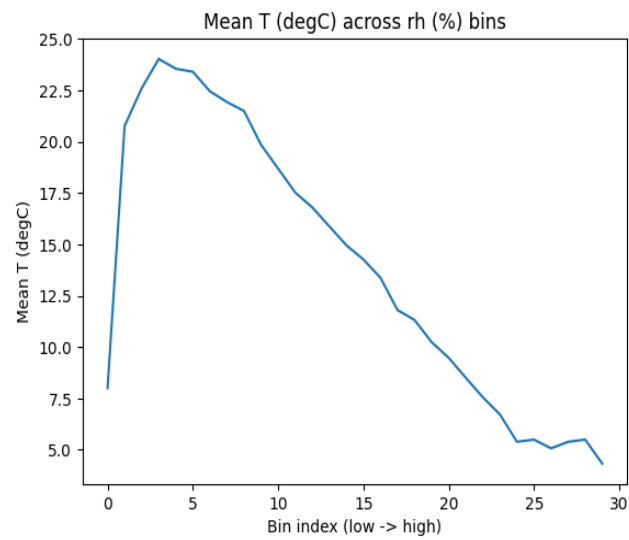


Figure 51 - Mean Air Temperature (T) across Relative Humidity (rh) bins

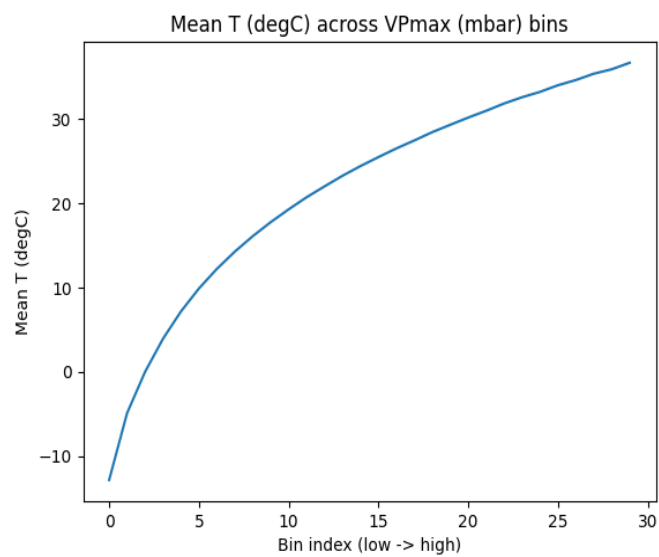


Figure 52 - Mean Air Temperature (T) across Maximum Vapor Pressure (VPmax) bins

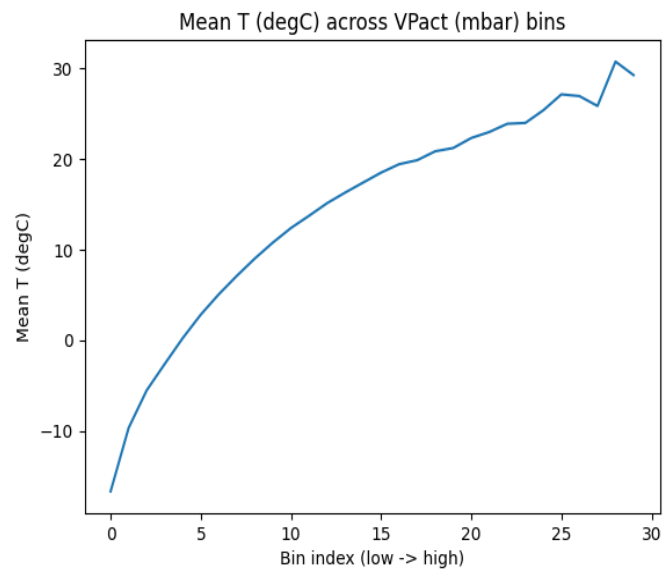


Figure 53 - Mean Air Temperature (T) across Actual Vapor Pressure (VPact) bins

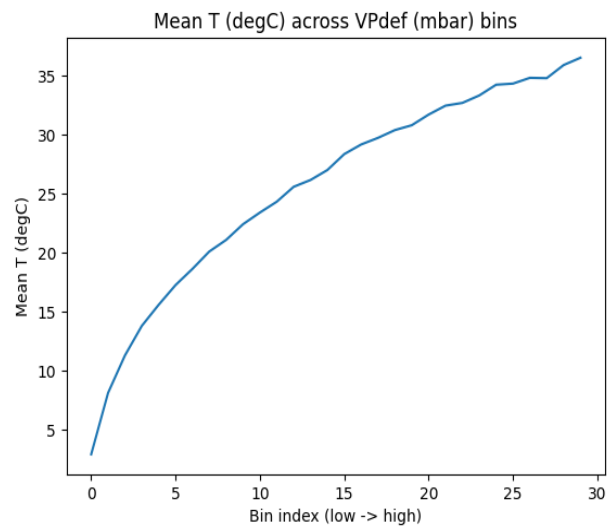


Figure 54 - Mean Air Temperature (T) across Vapor Pressure Deficit (VPdef) bins

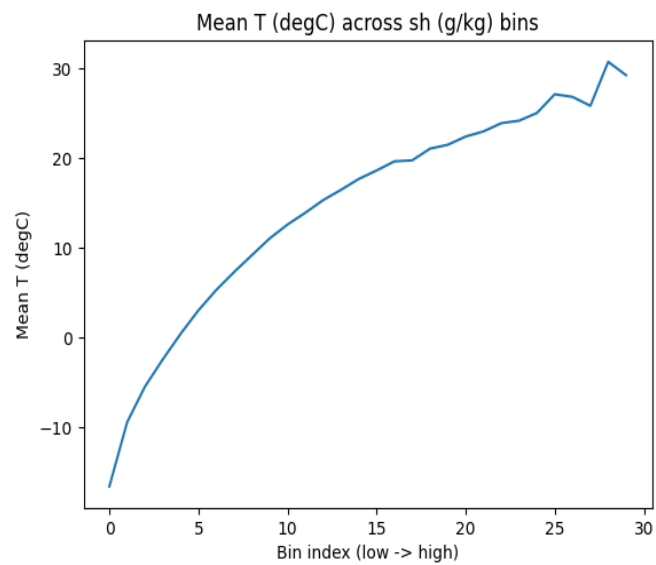


Figure 55 - Mean Air Temperature (T) across Specific Humidity (sh) bins

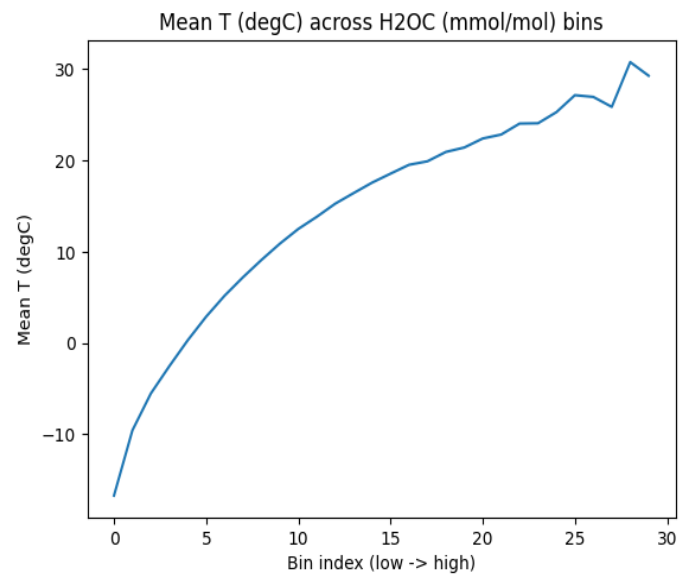


Figure 56 - Mean Air Temperature (T) across Water Vapor Concentration (H_2OC) bins

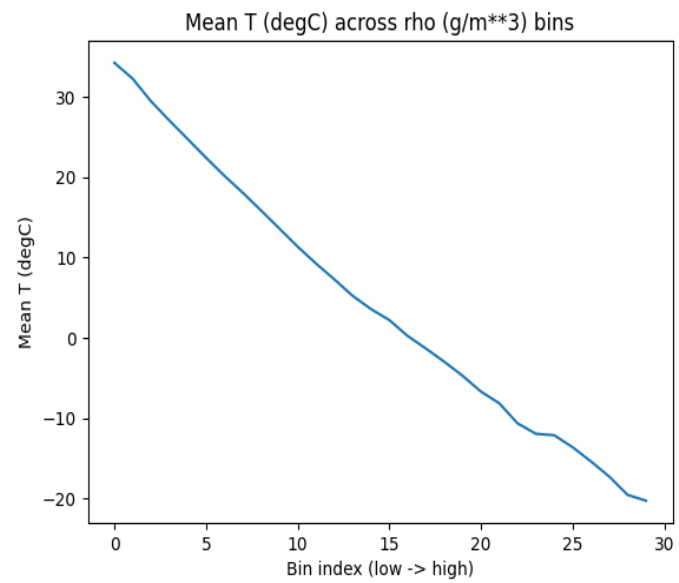


Figure 57 - Mean Air Temperature (T) across Air Density (ρ) bins

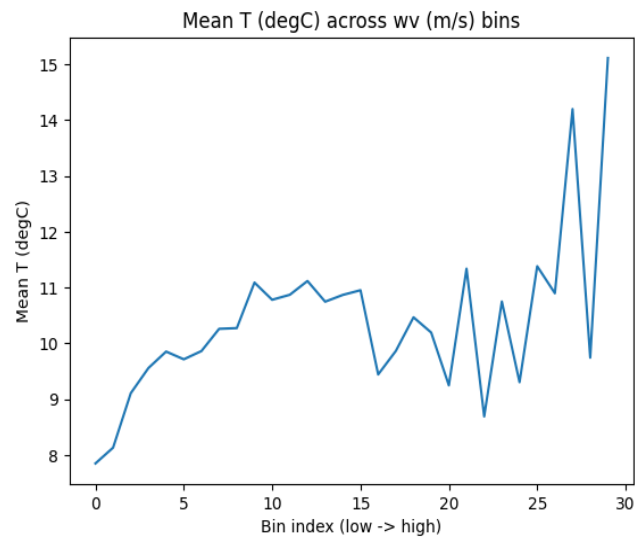


Figure 58 - Mean Air Temperature (T) across Wind Velocity (wv) bins

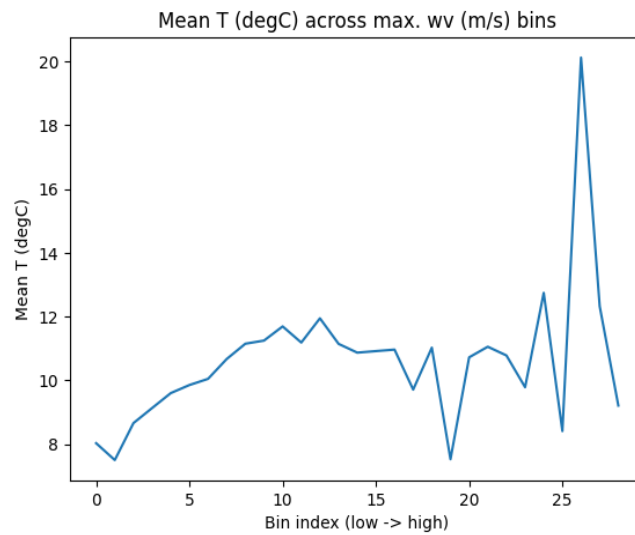


Figure 59 - Mean Air Temperature (T) across Maximum Wind Velocity (max wv) bins

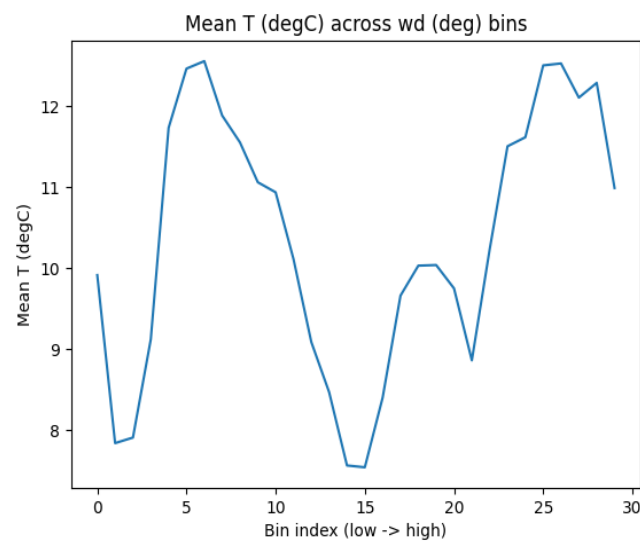


Figure 60 - Mean Air Temperature (T) across Wind Direction (wd) bins

2.5.17 Variables Mean by Hour of Day

The mean value of selected variables (temperature, relative humidity, pressure and wind speed) was computed for each hour of the day to characterize diurnal patterns (figures 61 – 64).

Temperature follows a typical daily cycle, with minimum values in the early morning and maximum values in the mid-afternoon. Relative humidity shows the opposite behavior, peaking around dawn and decreasing throughout the day. Pressure and wind speed also exhibit systematic daily variations, although with smaller amplitude. These results confirm the presence of strong intra-day seasonality in the dataset.

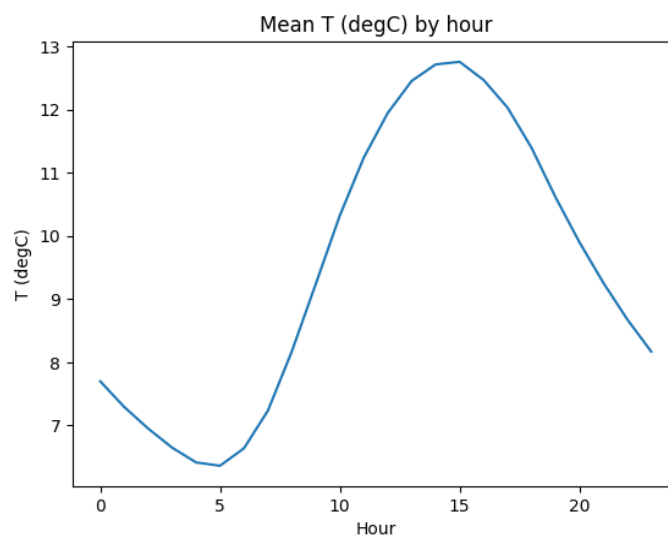


Figure 61 – Mean Air Temperature (T) by hour of the day

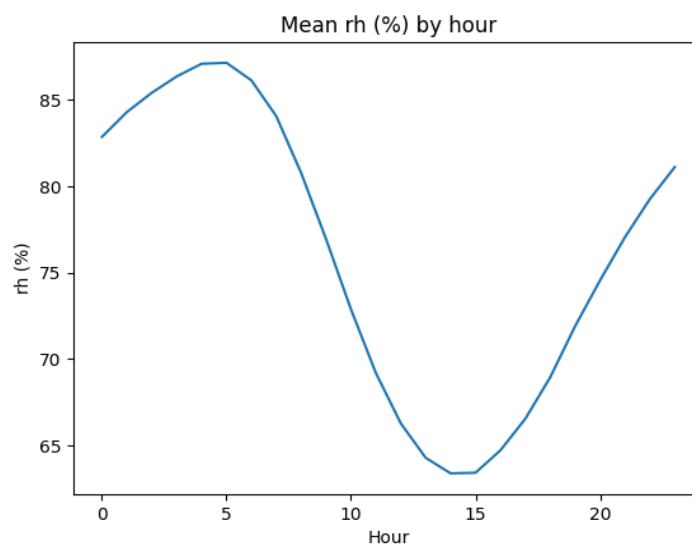


Figure 62 – Mean Relative Humidity (rh) by hour of the day

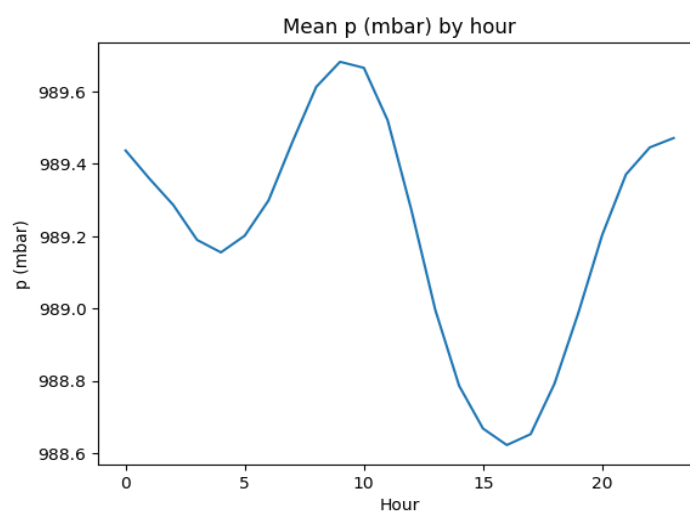


Figure 63 – Mean Atmospheric Pressure (p) by hour of the day

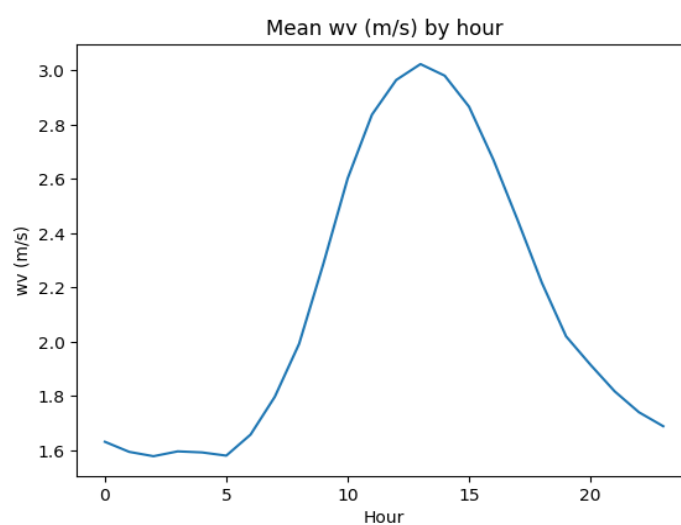


Figure 64 – Mean Wind Velocity (wv) by hour of the day

3. Data Understanding – Detailed Analysis

This section provides a deeper examination of the temporal structure and statistical characteristics of the dataset, focusing on aspects that are particularly relevant for time-series forecasting.

The dataset is first sorted chronologically using the Date Time column to ensure the correct temporal ordering of observations. The forecasting target is defined as air temperature in degrees Celsius, T (degC), while the remaining meteorological variables are considered potential predictors.

The following subsections analyze key properties of the time series, including temporal regularity, seasonality patterns, resampling behavior, and the presence of missing values. Understanding these aspects is essential for designing appropriate preprocessing steps and for selecting suitable modelling approaches for multivariate time-series forecasting.

3.1 Temporal Regularity and Gaps

To assess the temporal consistency of the dataset, the time differences between consecutive observations were computed. The most common interval between measurements is 10 minutes, confirming that the dataset follows a regular 10-minute sampling frequency (table 3).

A small number of irregular intervals were detected. In total, only five gaps larger than the expected sampling interval were observed. These include short gaps of 20–30 minutes as well as two larger gaps (16 hours and approximately 3 days), which likely correspond to temporary sensor outages or data logging interruptions.

The small number of irregular intervals indicates that the dataset is highly consistent from a temporal perspective. Nevertheless, identifying these gaps is important before performing resampling operations or applying time-series models that assume uniform temporal spacing (table 4).

Time Interval (Delta)	Count
00:10:00	420218
00:20:00	2
00:30:00	1
16:00:00	1
3 days 02:20:00	1

Table 3 - Frequency of Time Intervals in the dataset

Index	Date Time	Gap Duration
40378	2009-10-08 10:10:00	00:30:00
229875	2013-05-16 09:10:00	00:20:00
293229	2014-07-30 08:20:00	00:20:00
301346	2014-09-25 09:00:00	16:00:00

Index	Date Time	Gap Duration
410940	2016-10-28 12:50:00	3 days 02:20:00

Table 4 - Detected Timestamp Gaps and Their Durations

3.2 Seasonal Patterns

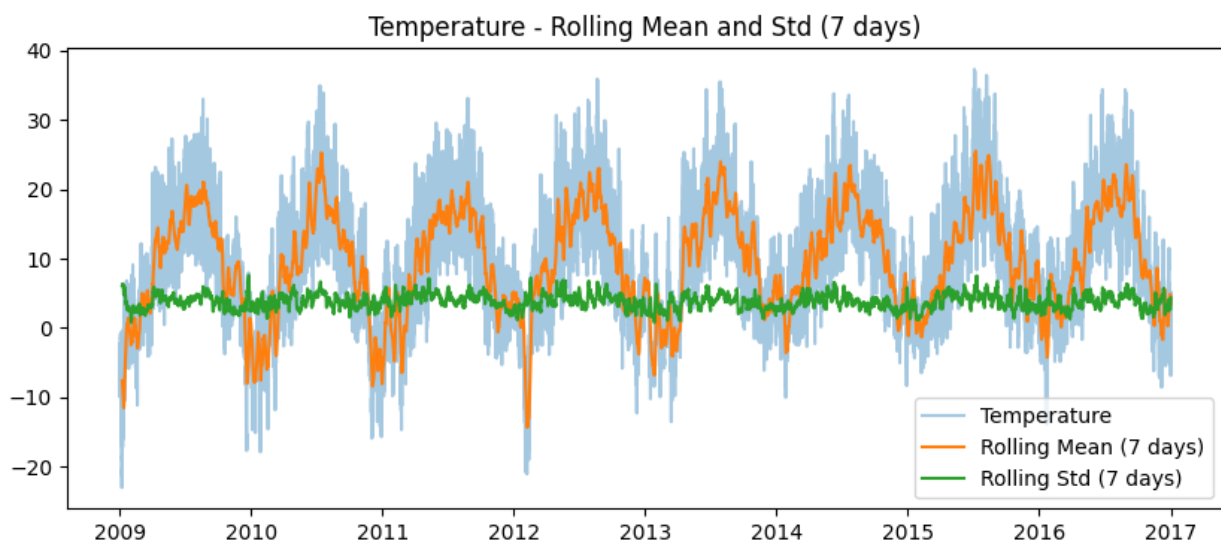
To better characterize the temporal dynamics of the dataset, seasonal patterns at different time scales are analyzed. First, rolling statistics are used to assess the stationarity of the temperature series over a 7-day window. Then, a preview of hourly resampling is performed to evaluate how aggregating 10-minute measurements into hourly means affects the structure of the data. These analyzes help to understand the dominant seasonal components and guide the choice of an appropriate modelling strategy.

3.2.1 Stationarity (Rolling Mean and Standard Deviation)

To assess the stationarity of the temperature series, a rolling window of 7 days was applied to compute the rolling mean and rolling standard deviation (figure 65).

The results reveal clear seasonal fluctuations in the rolling mean, reflecting the annual temperature cycle. The rolling standard deviation also varies over time, indicating changes in variability across different periods of the year.

These observations suggest that the temperature series is non-stationary due to strong seasonal patterns rather than a long-term structural trend. This behavior is typical in meteorological time series and highlights the importance of using forecasting models capable of handling seasonal and non-stationary data.

**Figure 65** - Temperature Rolling Mean and Standard Deviation (7-Day Window)

3.2.2 Preview of Hourly Resampling

The dataset was resampled to an hourly frequency using the Mean of all measurements within each hour. This aggregation reduces short-term noise and redundancy while preserving the overall temporal structure of the meteorological variables.

The resulting hourly dataset contains 70,129 observations and maintains the same set of variables as the original dataset. A verification of the timestamp sequence confirmed that no hourly timestamps are missing, indicating that the time index remains continuous after resampling.

However, some missing values appear after the aggregation step. These NaN values correspond to hours where no valid sensor measurements were available within the original 10-minute data. The number of missing values detected in each variable is summarized in Table 5. These missing observations must be addressed before model training to ensure that the forecasting models receive complete input sequences.

Variable	Number of NaN Values
p (mbar)	88
T (degC)	88
Tpot (K)	88
Tdew (degC)	88
rh (%)	88
VPmax (mbar)	88
VPact (mbar)	88
VPdef (mbar)	88
sh (g/kg)	88
H2OC (mmol/mol)	88
rho (g/m ³)	88
wv (m/s)	90
max. wv (m/s)	91
wd (deg)	88

Table 5 - Number of Missing Values per Variable

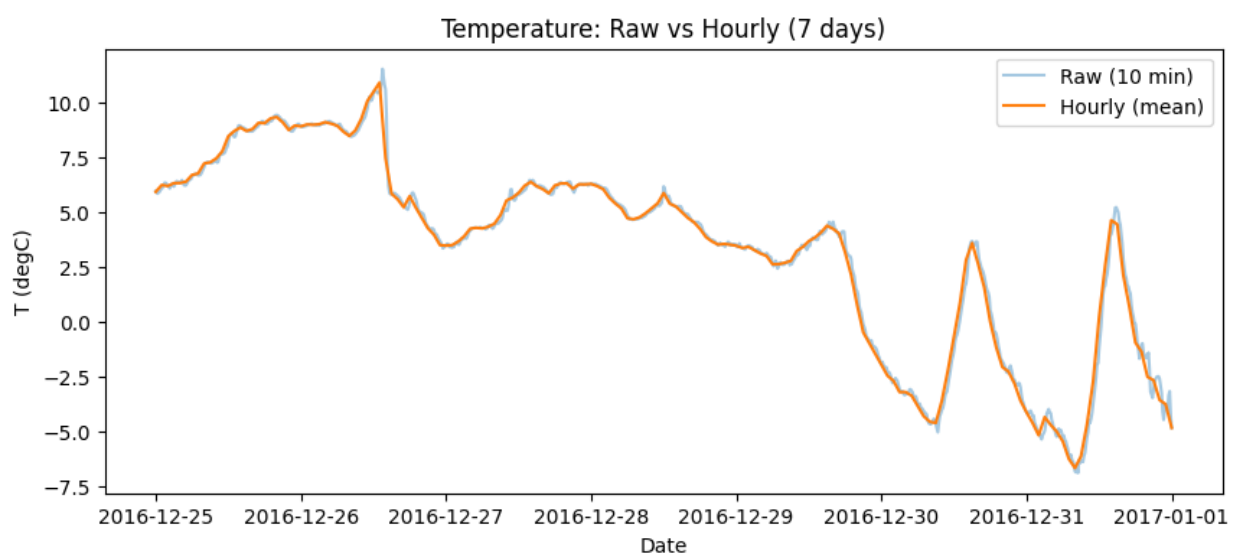


Figure 66 - Temperature: Raw (10-min) vs Hourly Mean (7-Day Window)

3.3 Identification of Missing Hourly Observations

Rows containing missing values in the hourly dataset were identified using a row-wise NaN check. In total, 88 hourly rows contain missing values across the meteorological variables.

Inspection of these rows shows that the missing values occur simultaneously across most variables, indicating periods where the original 10-minute dataset contained no valid measurements. These rows appear in contiguous blocks, corresponding to short intervals where the weather station temporarily stopped recording data.

Since these rows contain no usable information for modelling, they must be handled before further analysis. In this study, the affected rows were removed to ensure that the resulting time series contains only complete observations.

3.3.1 Removal of Missing Hourly Rows

To ensure data consistency, all hourly rows containing NaN values were removed from the dataset. This cleaning step reduced the number of observations from 70,129 to 70,038 rows.

A final verification confirmed that no missing values remain in the cleaned hourly dataset. This cleaned version of the hourly data is used in all subsequent modelling steps (table 6).

Dataset Stage	Rows	Columns
Hourly dataset (before cleaning)	70,129	15
Hourly dataset (after removing NaN)	70,038	15
Remaining missing values	0	–

Table 6 - Missing Values per Variable (Hourly Dataset)

4. Data Preparation

This section describes the steps required to prepare the dataset for modelling. The process includes selecting the relevant variables, confirming data integrity, defining the temporal frequency, setting the datetime index, splitting the data into training, validation and test sets, engineering additional time-based features, scaling the inputs, and finally constructing sliding windows for supervised learning.

4.1 Dataset Scope and Columns

The forecasting target selected for this study is air temperature in degrees Celsius, T (degC). The input variables were defined according to the specifications of the project guidelines, which require the use of a subset of meteorological variables from the Jena Climate dataset.

All variables were extracted from the cleaned hourly-resampled dataset, ensuring that the data share a consistent temporal resolution and contain no missing values. This dataset serves as the working base for all subsequent modelling steps.

4.1.1 Define Target, Inputs and Frequency

The target variable is defined as the hourly air temperature T (degC).

The input variables used for forecasting include:

- Atmospheric pressure (p (mbar))
- Relative humidity (rh (%))
- Wind speed (wv (m/s))
- Maximum wind speed ($max. wv$ (m/s))
- Wind direction (wd (deg))

These variables were extracted from the cleaned hourly dataset to construct the modelling dataframe. Using a consistent hourly frequency ensures that all predictors are temporally aligned with the target variable and suitable for time-series forecasting.

4.1.2 Sanity Checks

A validation step was performed to ensure that the working dataset contains no missing values and that all columns have the correct data types. All variables were confirmed to be numerical, and the datetime column was correctly stored as a Datetime Index. These checks guarantee that the dataset is ready for temporal indexing and model training.

4.1.3 Confirm Frequency is Hourly

The time differences between consecutive observations were computed to verify the sampling frequency after the resampling step. The most common interval between observations is exactly 1 hour, confirming that the dataset is uniformly spaced in time.

A very small number of larger intervals were also detected, corresponding to previously identified data gaps in the original dataset (table 7). These rare gaps appear as longer time differences but do not affect the overall regular hourly structure of the cleaned dataset.

Maintaining a uniform temporal frequency is essential for constructing sliding windows and training sequence models such as recurrent neural networks and transformers.

Time Difference Between Consecutive Observations	Count
1 hour	70,034
16 hours	1
4 hours	1

Table 7 - Time Differences Between Consecutive Observations (Hourly Dataset)

4.1.4 Set Date Time as Index

The Date Time column was set as the dataframe index, converting it into a Datetime Index. This enables efficient slicing, alignment, resampling and windowing operations, which are fundamental for time-series forecasting workflows.

4.2 Temporal Split (Train / Validation / Test)

To evaluate the forecasting models in a realistic setting, the dataset was split into training, validation and test sets using a purely temporal partition. This avoids information leakage from the future into the past and mimics real-world deployment conditions.

4.2.1 Split Proportions

The dataset was divided into three subsets using the following proportions: 70% for training, 15% for validation, and 15% for testing.

Since this is a time-series forecasting problem, the split was performed chronologically rather than randomly, ensuring that the training data precede the validation and test sets in time. This prevents information leakage from future observations.

Based on the total number of hourly samples, the resulting dataset sizes are shown in Table 8. These splits provide sufficient data for model training, hyperparameter tuning, and final performance evaluation.

Dataset Split	Number of Samples	Proportion
Training set	49,026	70%
Validation set	10,505	15%
Test set	10,507	15%
Total	70,038	100%

Table 8 - Dataset Split Proportions (Train, Validation, Test)

4.2.2 Contiguous Temporal Splits

The dataset was split into three contiguous time blocks while preserving the chronological order of the observations. The training set contains the earliest portion of the series, followed by the validation set, and finally the test set, which represents the most recent period of the dataset.

This chronological splitting strategy ensures that the model is always trained using past observations and evaluated on unseen future data, which reflects a realistic forecasting scenario. The temporal coverage of each subset is summarized in Table 9, and the corresponding dataset sizes are reported in Table 10.

A visual inspection of the temperature series across the three subsets confirms that all seasonal patterns are represented and that no temporal overlap occurs between the training, validation, and test datasets.

Dataset Split	Start Date	End Date
Training set	2009-01-01 00:00	2014-08-05 17:00
Validation set	2014-08-05 18:00	2015-10-18 04:00
Test set	2015-10-18 05:00	2017-01-01 00:00

Table 9 - Dataset Split: Temporal Ranges (Train, Validation, Test)

Dataset Split	Rows	Columns
Training set	49,026	6
Validation set	10,505	6
Test set	10,507	6

Table 10 - Dataset Split: Rows and Columns per Subset

4.2.3 Visual target over splits

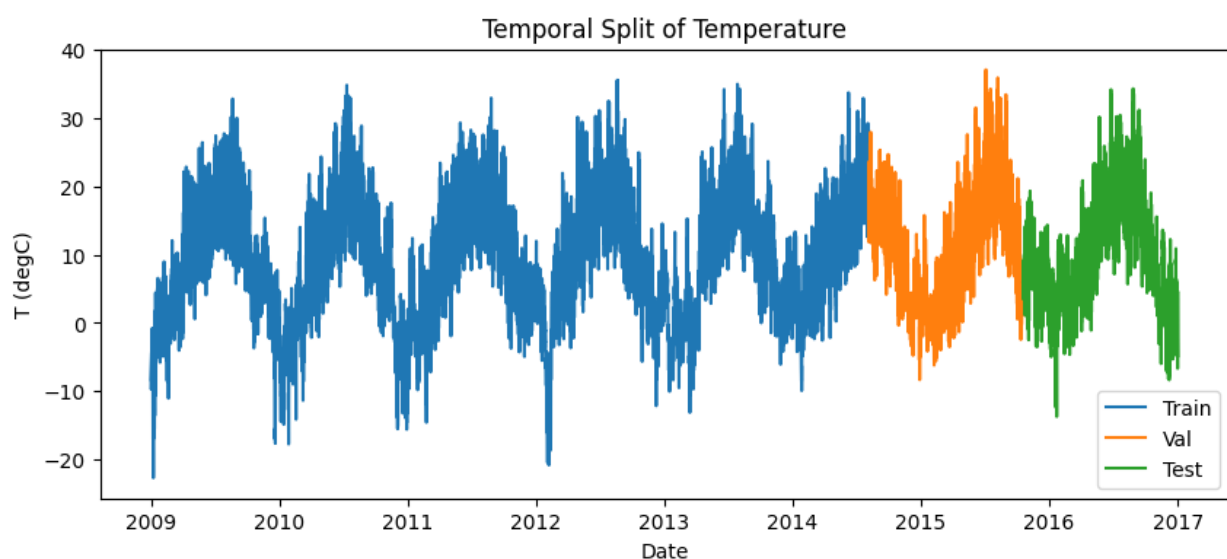


Figure 67 - Temporal Split of Temperature (Train, Validation, Test)

4.3 Feature Engineering

Additional time-based features were engineered to help the models capture periodic patterns present in the meteorological data.

From the datetime index, the hour of day** and day of year were extracted and encoded using sine and cosine transformations. This cyclical encoding preserves the circular nature of time variables, ensuring that values such as 23:00 and 00:00 remain close in the feature space. These features allow the model to capture daily and annual seasonality.

Wind direction is also a circular variable ranging from 0° to 360°. Therefore, it was transformed into sine and cosine components to correctly represent its angular structure. The original wind direction variable was then removed to avoid redundancy.

After applying these transformations, the resulting feature set contains 11 variables, including the original meteorological variables and the newly engineered cyclical features (table 11). These additional predictors provide explicit representations of periodic behavior that can improve the performance of sequence forecasting models.

Feature	Description
T (degC)	Target temperature
p (mbar)	Atmospheric pressure
rh (%)	Relative humidity
wv (m/s)	Wind speed
max. wv (m/s)	Maximum wind speed
hour_sin	Sine encoding of hour of day
hour_cos	Cosine encoding of hour of day
doy_sin	Sine encoding of day of year
doy_cos	Cosine encoding of day of year
wd_sin	Sine encoding of wind direction
wd_cos	Cosine encoding of wind direction

Table 11 - Features Used and Their Descriptions

4.4 Feature Scaling

All input features, including the engineered time-based variables, were standardized using the StandardScaler method. This transformation rescales each feature so that it has zero mean and unit variance, which helps stabilize gradient-based optimization during neural network training.

To prevent data leakage, the scaler was fitted exclusively on the training dataset. The same transformation parameters were then applied to the validation and test sets, ensuring that no information from future observations influenced the training process.

Feature scaling is particularly important for deep learning models such as recurrent neural networks and transformers, as it ensures that variables with different numerical ranges contribute proportionally during model optimization.

A final verification confirmed that no NaN values were introduced during the scaling process.

4.5 Windowing (L = 120, H = 24)

To cast the forecasting problem into a supervised learning setting, sliding windows were constructed over the scaled time series data. Each input sample consists of the previous L = 120 hours (5 days) of observations, while the corresponding target consists of the next H = 24 hours of air temperature.

This formulation defines a multivariate-input, univariate-output, multi-step forecasting problem, where the model receives a sequence of past meteorological observations and directly predicts the full 24-hour temperature profile ahead.

Since the windowing procedure was applied after feature scaling, both the input sequences and the target values were represented in the scaled space during training. This is beneficial for neural network optimization, as it stabilizes the magnitude of the learning signal.

The resulting datasets are summarized in Table 12.

Dataset	X Shape	y Shape
Training	(48,882, 120, 11)	(48,882, 24)
Validation	(10,361, 120, 11)	(10,361, 24)
Test	(10,363, 120, 11)	(10,363, 24)

Table 12 - Dataset Shapes After Windowing

4.5.1 Baseline: Persistence

As a simple baseline, a persistence model was implemented. For each window, the model predicts that the temperature for all 24 forecast hours will be equal to the last observed temperature in the input window. Although naive, this baseline provides a meaningful lower bound on performance and helps to contextualize the results of more complex models.

4.5.2 Evaluation Metrics

The performance of the multi-horizon forecasts was evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), computed over all horizons and all samples. These metrics quantify the average magnitude of the prediction errors and penalize larger deviations more strongly in the case of RMSE. The persistence baseline achieves an MAE of approximately 0.36 (scaled units) and an RMSE of about 0.49, which serves as a reference for subsequent models.

4.5.3 MAE by Horizon

To analyze how forecast accuracy degrades with time, the MAE was computed separately for each forecast horizon from 1 to 24 hours ahead (figure 68). The error profile shows that the MAE increases for intermediate horizons and then slightly decreases for the farthest horizons, reflecting the interaction between daily cycles and the persistence assumption. This horizon-wise analysis provides a more detailed view of model behavior than a single aggregated metric.

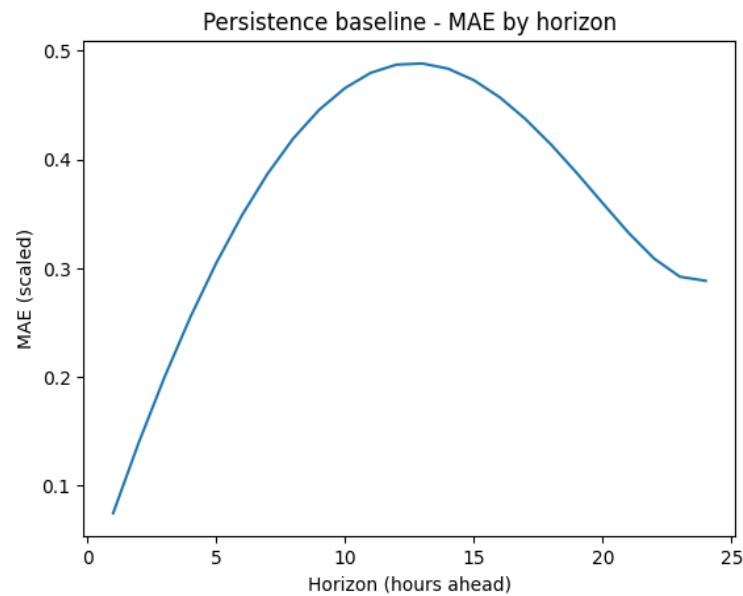


Figure 68 - Persistence Baseline: MAE by Forecast Horizon

5. GRU

This section describes the construction, configuration, and optimization of a GRU-based neural network for multi-horizon temperature forecasting. The workflow includes GPU setup, reproducibility settings, model architecture definition, training procedures, and hyperparameter optimization.

Hyperparameter optimization was performed using Optuna, an automatic hyperparameter search framework designed for efficient optimization of machine learning models (table 13). Optuna employs a sampling strategy that explores the hyperparameter space iteratively, evaluating model performance and updating its search strategy based on previous trials.

During the optimization process, multiple training configurations are evaluated by varying parameters such as the number of GRU units, number of layers, dropout rate, learning rate, batch size, and regularization strength. Each configuration corresponds to a trial, where the model is trained and evaluated on the validation set.

Optuna uses techniques such as Tree-structured Parzen Estimators (TPE) to guide the search towards promising regions of the hyperparameter space, allowing it to identify high-performing configurations more efficiently than traditional grid search methods.

The objective function minimizes the validation loss, and the best-performing configuration is selected as the final model setup for subsequent training and evaluation.

Hyperparameter	Description
units	Number of GRU units
n_layers	Number of GRU layers
dropout	Dropout rate for regularization
recurrent_dropout	Dropout applied to recurrent connections
learning_rate	Learning rate of the optimizer
batch_size	Number of samples per training batch
l2	L2 weight regularization

Table 13 - GRU Hyperparameters and Their Descriptions

5.1 Reproducibility Setup

To ensure consistent and repeatable results across runs, random seeds were fixed for both TensorFlow and NumPy. The input window length (L), forecast horizon (H) and number of features were also extracted from the training data to parameterize the model architecture.

5.2 Model Architecture and Hyperparameters

A flexible GRU-based architecture was implemented to model the temporal dependencies in the meteorological time series (table 14). The network supports between one and three stacked GRU layers, allowing the model capacity to be adjusted depending on the hyperparameter configuration.

Each GRU layer processes sequential input data and learns temporal representations from past observations. Dropout regularization is applied within the recurrent layers to reduce overfitting, while optional L2 weight regularization can be used to constrain the model parameters.

An optional Gaussian noise layer can be applied to the inputs during training, which acts as a form of regularization by improving model robustness to small perturbations in the input data.

The architecture also supports an optional dense layer before the output layer, enabling additional non-linear feature transformations. Different activation functions can be used in this layer depending on the configuration explored during hyperparameter optimization.

The final output layer produces a vector of length $H = 24$, corresponding to the 24-hour forecasting horizon. This implements a direct multi-step forecasting strategy, where the model predicts the entire temperature trajectory for the next day in a single forward pass.

Model training is performed using gradient-based optimization with either the Adam or AdamW optimizer. Gradient clipping is applied to stabilize training, and several loss functions can be selected (MSE, MAE, or Huber loss) depending on the optimization configuration.

Hyperparameter	Description
units1, units2, units3	Number of GRU units per layer
n_layers	Number of stacked GRU layers
dropout	Dropout rate applied to recurrent layers
l2	L2 weight regularization strength
dense_units	Size of optional dense layer
dense_activation	Activation function in dense layer
learning_rate	Learning rate of the optimizer
optimizer_name	Optimizer type (Adam or AdamW)
weight_decay	Weight decay parameter for AdamW
clipnorm	Gradient clipping threshold
loss_name	Training loss function (MSE, MAE, Huber)
gaussian_noise_std	Standard deviation of input Gaussian noise

Table 14 - Hyperparameters and Their Descriptions

5.3 Training Wrapper and Callbacks

To facilitate experimentation with multiple hyperparameter configurations, a dedicated training wrapper function was implemented. This function builds the GRU model according to a given configuration, trains the model, and returns the resulting performance metrics.

The wrapper enables consistent training procedure across all experiments by automatically constructing the model architecture, applying the selected hyperparameters, and executing the training process.

Two callback mechanisms are used during training to improve optimization stability and prevent overfitting:

- **EarlyStopping**, which monitors the validation loss and stops training if no improvement is observed for several epochs. The best model weights are automatically restored.
- **ReduceLROnPlateau**, which reduces the learning rate when the validation loss stagnates, allowing the optimizer to refine the solution.

After training, the function extracts the best validation metrics obtained during the training process, including the minimum validation loss and the corresponding validation MAE. These metrics are used to compare different configurations during the hyperparameter optimization stage.

This modular training setup allows efficient evaluation of multiple hyperparameter configurations during the Optuna optimization process.

5.4 Hyperparameter Optimization with Optuna

Hyperparameter optimization was performed using the Optuna framework, which implements efficient Bayesian optimization for machine learning models. Optuna uses a Tree-structured Parzen Estimator (TPE) sampler to explore the hyperparameter space and identify promising configurations based on previous trials.

During the optimization process, each trial samples a set of architectural and training hyperparameters, including the number of GRU layers, number of units per layer, dropout rate, learning rate, batch size, optimizer type, regularization strength, and loss function (table 15).

A custom pruning callback was implemented to integrate Optuna with the Keras training loop. After each epoch, the validation loss is reported to Optuna, allowing the Hyperband pruner to terminate trials that are unlikely to outperform the current best configuration. This significantly reduces computational cost by avoiding unnecessary training of poorly performing models.

For each trial, the GRU model is trained using the sampled hyperparameters and evaluated on the validation dataset. The objective function minimizes the validation loss, while the corresponding validation MAE is also recorded for analysis.

The study continuously tracks the best-performing configuration and stores the associated model, training history, and hyperparameter settings. This enables the final model selection to be based on the configuration achieving the lowest validation loss during the optimization process.

Hyperparameter	Values Explored
n_layers	1, 2, 3
units1	64, 96, 128, 192
units2	32, 64, 96, 128
units3	32, 64, 96
dropout	0.0, 0.1, 0.2, 0.3, 0.4
l2	0.0, 1e-6, 1e-5, 1e-4
dense_units	0, 64, 128, 256
dense_activation	Relu, Gelu, Elu, leaky_relu
learning_rate	2e-3, 1e-3, 5e-4, 3e-4, 2e-4, 1e-4
batch_size	128, 256, 512
clipnorm	0.5, 1.0, 2.0, 5.0
optimizer	Adam, AdamW
weight_decay	0.0, 1e-6, 1e-5, 1e-4
loss_name	MSE, MAE, huber1, huber2
gaussian_noise_std	0.0, 0.01, 0.05

Table 15 - Hyperparameters and Search Space Explored with Optuna

5.5 Hyperparameter Optimization Run

The hyperparameter optimization was executed using Optuna with a Tree-structured Parzen Estimator (TPE) sampler and a Hyperband pruning strategy. The TPE sampler performs Bayesian optimization by modelling the distribution of promising hyperparameter configurations based on previously evaluated trials, allowing the search process to focus on regions of the parameter space likely to produce better results.

To reduce computational costs, the Hyperband pruner was used to terminate unpromising trials early. Each trial was required to train for a minimum number of epochs before pruning decisions were made, ensuring that early training noise did not prematurely eliminate potentially good configurations.

A custom callback was implemented to provide live visual feedback during the optimization process, displaying the validation loss progression across completed trials as well as the duration of each trial. This allowed real-time monitoring of optimization progress and computational efficiency.

In total, 150 trials were executed. Among these, 92 trials were completed successfully, while 58 trials were pruned early by the Hyperband strategy. The best configuration achieved a validation loss of 0.0337 (MSE on scaled data, v. Table 16).

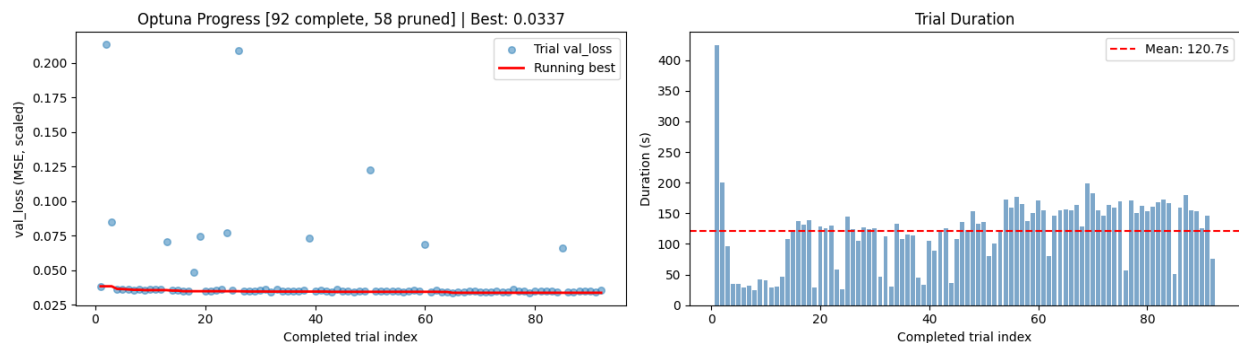
Metric	Value
Total trials	150
Completed trials	92
Pruned trials	58
Best validation loss	0.0337
Optimization method	TPE (Bayesian)
Pruning strategy	Hyperband

Table 16 - Summary of the Hyperparameter Optimization Run

The optimization results show that the pruning strategy effectively reduced the computational cost by terminating nearly 40% of trials early, while still identifying high-performing configurations within the search space (figure 69).

[150/150] Trial #149 → PRUNED

Complete: 92 | Pruned: 58 | Best so far: 0.0337



Optimization complete!

Best trial #113 – val_loss: 0.0337

Best params: {'n_layers': 2, 'units1': 96, 'units2': 64, 'units3': 96, 'dropout': 0.0, 'l2': 1e-06, 'dense_units': 256, 'dense_activation': 'relu', 'lr': 0.0002, 'batch_size': 128, 'clipnorm': 2.0, 'optimizer': 'adamw', 'weight_decay': 1e-06, 'loss_name': 'huber1', 'gaussian_noise_std': 0.0}

Figure 69 - Optuna Optimization Progress and Trial Durations

5.6 Best Model Configuration

After completing the hyperparameter optimization process, the best-performing configuration was obtained in trial 113, achieving a validation loss of 0.0337 (MSE on scaled data).

The optimal model uses a two-layer GRU architecture with moderate hidden sizes and no dropout regularization. Instead, a small amount of L2 weight regularization and AdamW weight decay are used to control overfitting. An additional dense layer with 256 units is applied before the output layer to increase the model's capacity for non-linear feature transformations.

The model was trained using the AdamW optimizer with a low learning rate and Huber loss, which provides robustness to occasional large prediction errors.

The full set of hyperparameters for the best configuration is shown in Table 17.

Hyperparameter	Best Value
n_layers	2
units1	96
units2	64
units3	96
dropout	0.0
l2	1e-6
dense_units	256
dense_activation	relu
learning_rate	0.0002
batch_size	128
clipnorm	2.0
optimizer	adamw
weight_decay	1e-6
loss_function	huber ($\delta = 1$)
gaussian_noise_std	0.0

Table 17 - Best Hyperparameter Configuration

The results suggest that a moderately sized GRU architecture is sufficient to capture the temporal dynamics of the meteorological variables. The optimization process favored deeper representations combined with a dense projection layer rather than heavy recurrent regularization.

The selection of Huber loss indicates that robustness to occasional extreme temperature variations may improve forecasting stability compared to purely quadratic loss functions.

5.6.1 Analysis of Hyperparameter Trials

Beyond identifying the best-performing configuration, it is important to analyze how different hyperparameter choices influence model performance. The Optuna optimization process evaluated a wide range of architectural and training configurations, allowing several patterns to emerge across the completed trials.

Out of the 150 optimization trials executed, 92 trials completed successfully while 58 were pruned early. Approximately 39% of trials were pruned before completing training, indicating that the Hyperband pruning strategy effectively reduced computational cost by terminating configurations that were unlikely to outperform the current best model.

- **Architecture Depth:** Trials with two GRU layers generally performed better than single-layer models, suggesting that additional temporal abstraction helps capture complex patterns in the meteorological time series. However, increasing the depth to three layers did not consistently improve performance, likely due to increased model complexity and training instability.
- **Number of Units:** Configurations with moderate hidden sizes (e.g., 96–64 units) tended to outperform both very small and very large architectures. Smaller models may lack sufficient capacity to model temporal dynamics, while excessively large models risk overfitting or slower convergence.
- **Regularization:** Interestingly, the best-performing configuration used no dropout, but relied on mild L2 regularization together with weight decay through the AdamW optimizer. This suggests that with a sufficiently large dataset, heavy dropout regularization may not be necessary to control overfitting.
- **Loss Functions:** Different loss functions were explored during optimization. Trials using Huber loss frequently achieved competitive results, likely because this loss function is less sensitive to occasional large prediction errors caused by extreme temperature fluctuations.
- **Learning Rate and Optimizer:** Lower learning rates (around 2×10^{-4}) consistently produced more stable training behavior. The AdamW optimizer also appeared in several strong-performing trials, indicating that decoupled weight decay can provide more stable optimization than standard Adam in some configurations.
- **Dense Projection Layer:** Many of the best-performing trials included an additional dense layer before the output. This layer enables further non-linear transformations before producing the 24-hour forecast vector, improving the model's ability to capture complex relationships between the input variables and the predicted temperature trajectory.

Taken together, the optimization experiments indicate that moderately sized GRU architectures combined with stable optimization settings provide reliable forecasting performance for this task (table 18).

Rank	Best Val Loss	Layers	L2	Dense Units	Dense Act	LR	Batch	Optimizer	Weight Decay
1	0.033656	2	1e-6	256	relu	2e-4	128	adamw	1e-6
2	0.033723	2	1e-6	256	relu	2e-4	128	adamw	1e-6
3	0.033961	2	1e-6	256	relu	2e-4	128	adamw	1e-6
4	0.034121	2	1e-6	256	relu	2e-4	128	adamw	1e-6
5	0.034135	2	1e-6	256	relu	2e-4	128	adamw	1e-6
6	0.034169	2	1e-6	256	relu	2e-4	128	adamw	1e-6
7	0.034188	2	1e-6	256	leaky_relu	2e-4	128	adamw	1e-6

Rank	Best Val Loss	Layers	L2	Dense Units	Dense Act	LR	Batch	Optimizer	Weight Decay
8	0.034245	2	1e-6	256	relu	2e-4	128	adamw	1e-6
9	0.034247	2	1e-6	256	leaky_relu	2e-4	128	adamw	1e-6
10	0.034269	2	1e-6	256	leaky_relu	2e-4	128	adamw	1e-6

Table 18 - Best Hyperparameter Values from Optuna Optimization

5.7 Training Curves (Best GRU)

The training dynamics of the best-performing GRU model (trial #113) were analyzed using the evolution of the training and validation metrics across epochs (figures 70–71).

Both the loss (MSE) and MAE curves show a rapid decrease during the first few epochs, indicating that the model quickly learns the main temporal patterns present in the meteorological time series. After this initial phase, the improvement becomes more gradual, eventually stabilizing after approximately 15–18 epochs.

The validation curves closely follow the training curves throughout the optimization process, with only a small gap between them. This behavior suggests that the model generalizes well to unseen data and does not exhibit strong signs of overfitting.

Additionally, the validation loss reaches a plateau toward the final epochs, which indicates that the model has reached a stable solution and that further training would likely provide only marginal improvements. This behavior is consistent with the early stopping strategy used during training.

The training curves confirm that the selected hyperparameter configuration leads to stable optimization and reliable convergence behavior for the GRU forecasting model.

The smooth behavior of the curves also indicates that the chosen learning rate and gradient clipping settings provided stable optimization without oscillations during training.

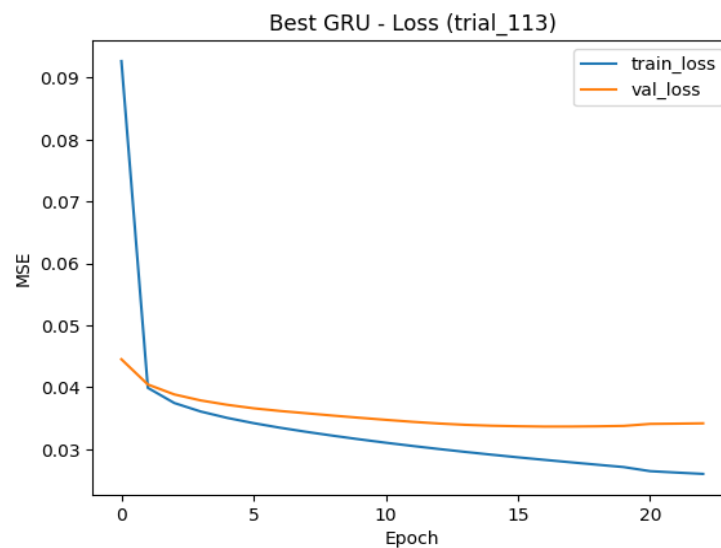


Figure 70 - Training and validation loss (MSE) across epochs for the best GRU model (trial 113)

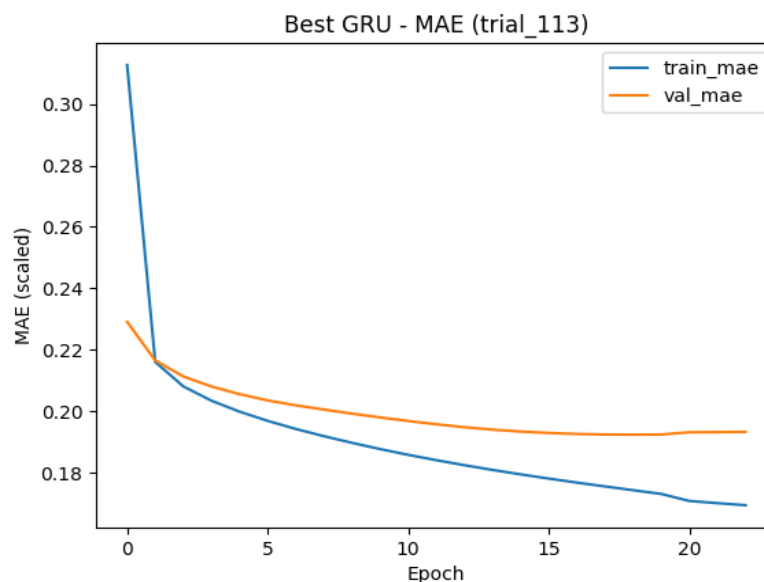


Figure 71 - Training and validation mean absolute error (MAE) across epochs for the best GRU model (trial 113)

5.8 Evaluation of Test Set (Scaled)

The best GRU model was evaluated on the test dataset using the scaled target values produced during preprocessing. The predictions were compared with the ground truth values in the scaled space, and several evaluation metrics were computed to assess model performance.

The model achieved low error values and a high coefficient of determination (R^2), indicating that it captures most of the variance in the temperature series and produces accurate multi-step forecasts (table 19).

Since the values are still in the scaled space, the Mean Absolute Percentage Error (MAPE) was not computed at this stage, as percentage-based metrics are not meaningful before transforming the predictions back to the original temperature units.

The evaluation results are summarized in Table 19.

Metric	Value (scaled)
MAE	0.1885
RMSE	0.2493
R^2	0.9239

Table 19 - Final Model Performance Metrics

5.9 Inverse Scaling and Metrics in °C

To obtain interpretable evaluation metrics, the predicted and ground-truth temperature values were transformed back to the original temperature scale (degrees Celsius). This was done by applying the inverse of the standardization used during preprocessing.

Since the data were standardized using `StandardScaler`, the inverse transformation was performed using the mean and standard deviation associated with the temperature feature. Only the target variable was inverse scaled, while the remaining input features remained in the scaled space.

After converting the predictions and targets back to degrees Celsius, evaluation metrics were recomputed to assess the forecasting accuracy in physically meaningful units.

While the scaled metrics provide useful information about optimization behavior during training, evaluating the model in degrees Celsius allows a clearer interpretation of the forecasting error.

5.9.1 Metrics in °C

After applying the inverse transformation, evaluation metrics were computed in degrees Celsius to obtain physically interpretable results.

The GRU model achieves a Mean Absolute Error (MAE) of 1.63 °C and a Root Mean Squared Error (RMSE) of 2.15 °C on the test set. These values indicate that, on average, the model's predictions deviate by roughly one to two degrees Celsius from the observed temperature.

The coefficient of determination ($R^2 = 0.92$) shows that the model explains more than 92% of the variance in the temperature series, indicating that the dominant temporal patterns are successfully captured.

The Mean Absolute Percentage Error (MAPE) was not used as an evaluation metric. Temperature values in the dataset frequently approach or cross zero degrees Celsius, which makes percentage-based errors unstable due to division by values close to zero. In such situations, MAPE can produce extremely large or misleading values. For this reason, MAE and RMSE are considered more reliable metrics for evaluating temperature forecasting models.

5.10 MAE by Horizon

To analyze how forecast accuracy evolves with increasing prediction horizon, the Mean Absolute Error (MAE) was computed separately for each of the 24 forecast hours.

The results show a gradual increase in error as the forecasting horizon expands. The MAE starts at approximately 0.6 °C for the first hour ahead* reflecting very accurate short-term predictions. As the horizon increases, the error progressively grows, reaching around 2.1–2.2 °C at the 24-hour horizon.

This behavior is expected in multi-step forecasting tasks. When predictions are made further into the future, the model must rely on increasingly uncertain temporal patterns, which naturally leads to larger prediction errors.

Despite this gradual increase, the error growth remains smooth and stable, indicating that the model maintains reliable predictive performance across the full 24-hour forecasting window. The absence of sudden jumps in error suggests that the GRU architecture successfully captures the temporal dynamics required for medium-term temperature forecasting.

This result confirms that the model remains robust even for longer forecasting horizons, which is particularly important for practical meteorological applications where predictions for the next 24 hours are required (figure 72).

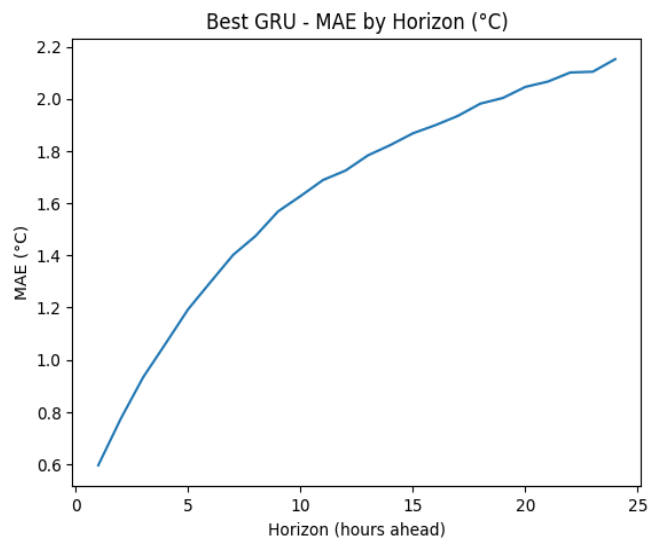


Figure 72 - Best GRU: MAE by Forecast Horizon (°C)

6. Overview of Hyperparameter Configurations

This section summarizes the hyperparameter configurations explored during the optimization process for the GRU forecasting model. Each configuration represents a specific combination of architectural parameters, regularization strategies and optimization settings sampled during the Optuna search procedure.

For each trial, the table reports the validation loss and the corresponding validation MAE obtained during training. These metrics allow direct comparison between different configurations and help identify which design choices lead to better predictive performance.

The results are sorted by validation loss, where the top rows correspond to the most accurate models and the bottom rows represent the least effective configurations. In total, 150 trials were executed during the optimization process, of which 92 were completed successfully while 58 were pruned early by the Hyperband strategy.

The best-performing configuration achieved a validation loss of 0.0337 and a validation MAE of 0.1924, demonstrating strong predictive performance for the multi-horizon temperature forecasting task.

Examining the top-performing models reveals a consistent pattern in the hyperparameter choices. Most of the best trials share the following characteristics:

- Two GRU layers, suggesting that moderate architectural depth is sufficient to capture temporal dependencies.
- Hidden sizes around 96–64 units, indicating that medium-capacity models perform better than either very small or excessively large networks.
- No dropout, implying that strong regularization through dropout was not necessary given the size of the dataset.
- AdamW optimizer with small weight decay, which appears to provide stable optimization behavior.
- Low learning rates (around 2×10^{-4}), allowing smoother convergence during training.
- Huber loss, which is robust to occasional large prediction errors.

In contrast, the worst-performing configurations often include one or more of the following characteristics:

- High dropout values (0.3–0.4), which may excessively regularize the model and limit its ability to learn meaningful temporal patterns.
- Large Gaussian noise levels, which can destabilize training.
- Higher learning rates, which may lead to less stable optimization.
- Different loss functions such as MAE or MSE, which in some trials produced less stable convergence compared to Huber loss.

The comparison between the best and worst trials highlights the importance of balanced architectural complexity and stable optimization settings when training recurrent neural networks for time-series forecasting.

6.1 Distribution of Validation Performance

To better understand the variability of model performance across the explored configurations, histograms were generated for the best validation loss and best validation MAE obtained during training.

The distribution of validation loss shows a strong concentration of trials in the lower error region, particularly between 0.033 and 0.040 MSE. This indicates that many hyperparameter configurations produce reasonably accurate models once the optimization process converges.

However, a small number of configurations result in significantly higher errors, with validation loss values extending beyond 0.07 and up to approximately 0.21. These outliers correspond to less effective combinations of hyperparameters, such as high dropout values, unstable optimization settings, or insufficient model capacity.

A similar pattern can be observed in the distribution of validation MAE. Most trials cluster around 0.19–0.20, suggesting that a large portion of the explored configurations achieve comparable predictive performance. A few configurations, however, produce noticeably larger errors, indicating weaker generalization.

These distributions highlight two important aspects of the optimization process. First, the search space contains multiple viable configurations capable of achieving strong forecasting performance. Second, certain hyperparameter combinations clearly degrade model quality, reinforcing the value of systematic hyperparameter optimization.

The presence of relatively few high-error trials also reflects the effectiveness of the Hyperband pruning strategy, which terminated many underperforming configurations early and prevented unnecessary training of poor models.

The narrow spread among the best-performing trials suggests that the optimization process converged to a stable region of the hyperparameter space where several configurations yield very similar performance (figures 73 – 74).

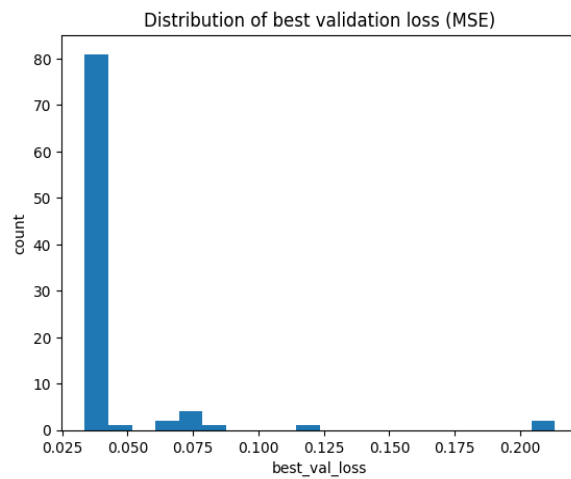


Figure 73 - Distribution of the best validation loss (MSE) obtained across the hyperparameter optimization trials

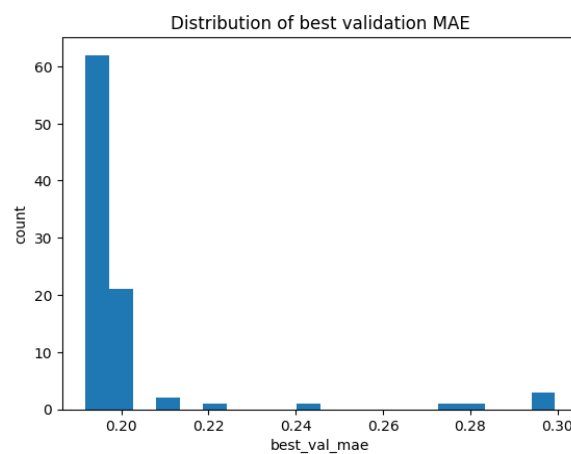


Figure 74 – Distribution of the best validation mean absolute error (MAE) obtained across the hyperparameter optimization trials

6.2 Hyperparameter Search Progress and Learning Rate Analysis

The hyperparameter optimization progress is illustrated in Figure 75. Each point represents the best validation loss obtained from a completed trial, while the red line shows the running best value observed during the optimization process. The vertical dashed line indicates the trial where the best-performing configuration was discovered.

The results show that a strong configuration was identified relatively early in the optimization process. After this point, most additional trials produced validation losses very close to the current best value, indicating that the search quickly converged to a stable region of the hyperparameter space.

The distribution of validation loss across different learning rates provides additional insight into the sensitivity of the model to this parameter. The boxplot analysis shows that learning rates around 0.0002 consistently produce the lowest validation errors and the most stable training behavior.

In contrast, higher learning rates such as 0.0003 display much larger variability in validation performance, suggesting that the optimization process becomes less stable at these values. Very small or very large learning rates also tend to produce slightly worse results.

Overall, this analysis confirms that the learning rate is one of the most influential hyperparameters in training the GRU model. A moderately small learning rate enables stable gradient updates and allows the model to converge to a better-performing solution.

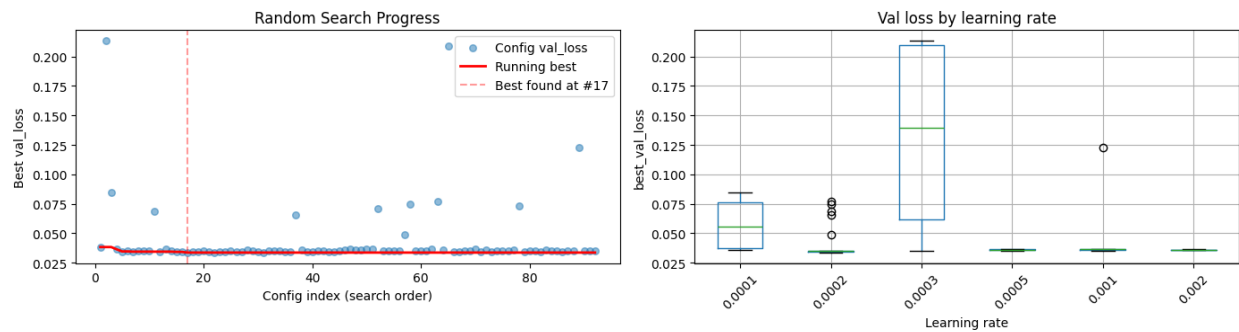


Figure 75 - Random Search Progress and Validation Loss by Learning Rate

6.3 Performance vs Training Time

The scatter plot in figure 76 illustrates the relationship between training time and validation performance for the different hyperparameter configurations evaluated during the optimization process.

Each point represents a single configuration. The horizontal axis corresponds to the total training time (in seconds), while the vertical axis shows the best validation loss obtained by that model. Since lower validation loss indicates better predictive performance, points located closer to the bottom of the plot correspond to more accurate models.

The results reveal that longer training times do not necessarily translate into better model performance. Many of the best-performing configurations are concentrated within a moderate training-time range of approximately 120–180 seconds, where several models achieve validation losses close to the optimal value.

In contrast, some configurations with very short training times produce higher validation losses, indicating insufficient model capacity or unstable optimization. At the same time, a few models with longer training times do not provide additional performance improvements, suggesting diminishing returns from increased computational cost.

This analysis highlights the importance of balancing model complexity and computational efficiency. Rather than simply increasing training time or model size, carefully tuned configurations can achieve strong predictive performance while maintaining reasonable computational cost. Such configurations represent favorable trade-offs between accuracy and efficiency.

This type of analysis provides a Pareto-style interpretation of the optimization results, helping identify configurations that offer strong predictive accuracy while maintaining efficient training times.

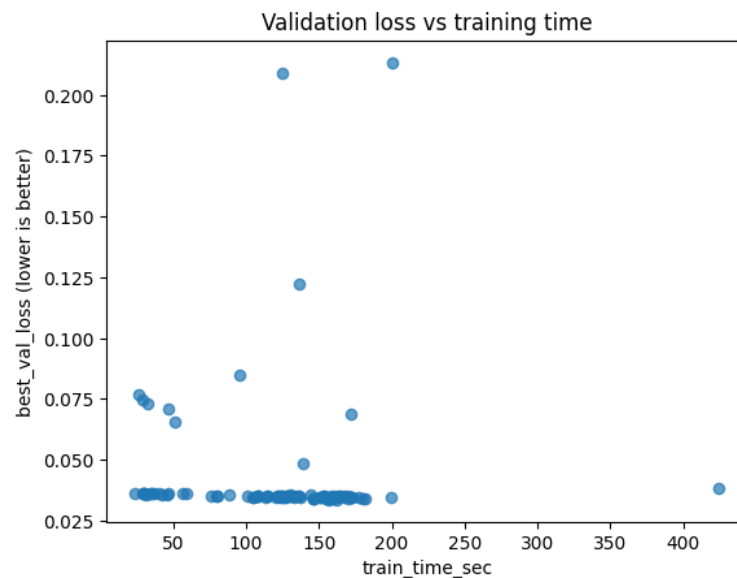


Figure 76 - Validation Loss vs Training Time

6.4 Important Hyperparameters

To better understand how different architectural choices influence model performance, additional analysis was conducted focusing on key hyperparameters of the GRU architecture. We examine how validation performance varies with respect to the number of GRU layers, dropout rate, and optimizer choice.

These analyses provide insight into which hyperparameters have the strongest impact on forecasting performance and help guide the selection of more effective model configurations. By studying the distribution of validation loss across different hyperparameter values, we can identify patterns that reveal which architectural or training choices tend to produce more stable and accurate models.

6.4.1 Effect of the Number of GRU Layers

The boxplot in figure 77 illustrates the distribution of validation loss across models with different numbers of GRU layers.

Models with two GRU layers consistently achieve the lowest validation losses, including the best-performing configurations identified during the optimization process. This suggests that a moderate level of architectural depth helps the model capture more complex temporal dependencies present in the meteorological time series.

Models with a single GRU layer also achieve relatively low validation losses, although their performance is slightly less consistent compared to the two-layer architecture. This indicates that shallow architectures are still capable of modelling the underlying temporal structure but may lack the representational capacity of deeper models.

In contrast, configurations with three GRU layers exhibit substantially higher validation losses. This suggests that deeper architectures introduce unnecessary complexity for this task, potentially making optimization more difficult and increasing the risk of unstable training.

The results indicate that moderately deep architectures provide the best trade-off between representational capacity and training stability, with two-layer GRU models offering the most reliable forecasting performance.

This behavior suggests the existence of a capacity “sweet spot”, where the model is sufficiently expressive to capture temporal patterns without introducing unnecessary complexity.

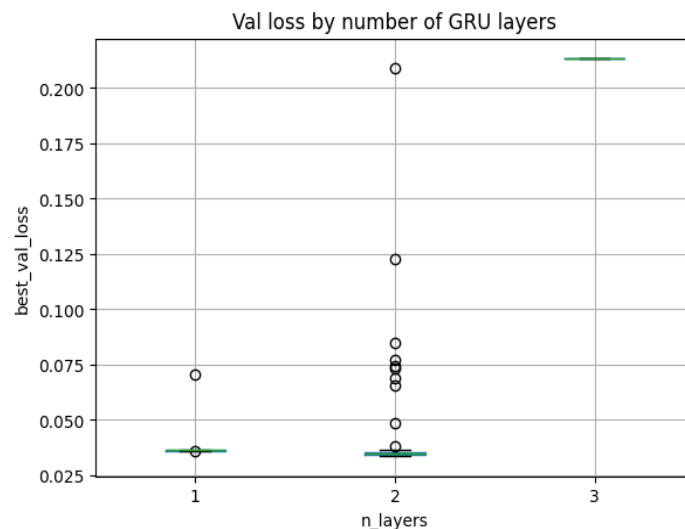


Figure 77 - Validation Loss by Number of GRU Layers

6.4.2 Effect of Dropout Regularization

The boxplot in figure 78 illustrates how validation performance varies across different dropout rates used during training.

The results show that configurations without dropout (0.0) consistently achieve the lowest validation losses. These models not only achieve the best overall performance but also display very low variability across configurations, indicating stable and reliable training behavior.

As the dropout rate increases, the validation loss generally becomes larger and more variable. Models with dropout values of 0.1 and above exhibit substantially higher validation losses and greater dispersion, suggesting that stronger regularization negatively affects the model’s ability to learn useful temporal representations.

Higher dropout values remove a larger fraction of the hidden activations during training, which can significantly reduce the effective capacity of the recurrent network. In this forecasting task, where the dataset is relatively large and the temporal structure is complex, excessive dropout appears to hinder the learning process.

Overall, the results indicate that little or no dropout regularization is preferable for this task, allowing the GRU architecture to fully exploit the temporal information present in the meteorological time series.

This observation is consistent with the behavior of the best-performing configuration, which also used no dropout regularization .

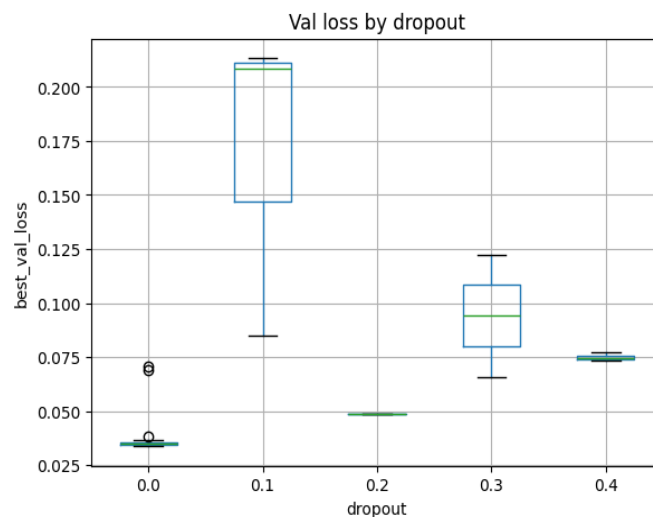


Figure 78 - Validation Loss by Dropout Rate

6.4.3 Effect of the Optimizer

The final analysis compares model performance across the different optimization algorithms used during training.

The boxplot in figure 79 shows that configurations trained with AdamW consistently achieve lower validation losses and exhibit much lower variability compared to those using the standard Adam optimizer. Most of the best-performing models are clustered near the lowest validation loss values when AdamW is used.

In contrast, models trained with Adam display substantially greater dispersion in validation loss, with several configurations producing significantly worse results. This indicates that the optimization process is less stable when using the standard Adam optimizer for this task.

A likely explanation is that AdamW applies decoupled weight decay, which improves regularization by separating weight decay from the gradient-based update rule. This mechanism often leads to more stable optimization and better generalization performance in deep learning models.

The results suggest that AdamW provides a more reliable and effective optimization strategy for training the GRU-based forecasting model on the Jena climate dataset.

This observation is consistent with the best-performing configuration identified during the hyperparameter optimization process, which also used the AdamW optimizer.

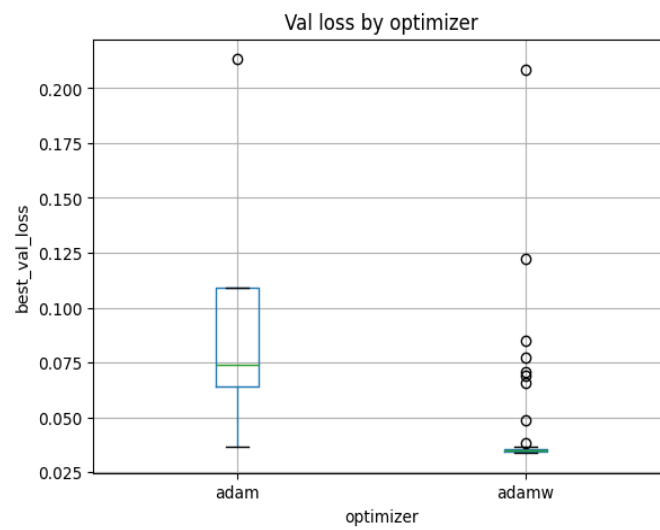


Figure 79 - Validation Loss by Optimizer Type

7. Model Evaluation and Forecast Analysis

This section presents a detailed evaluation of the forecasting performance of the proposed GRU model on the test dataset. Beyond aggregate performance metrics, the analysis focuses on how the model behaves across different prediction horizons and how effectively it captures the temporal dynamics of the temperature series.

Several complementary analyses are used to assess the robustness and reliability of the model. These include a comparison with a simple baseline model, an examination of forecast errors across the 24-hour horizon, and visual inspection of predicted and observed temperature trajectories.

These analyses provide a more complete understanding of the strengths and limitations of the GRU architecture for multi-step temperature forecasting.

7.1 Horizon-wise Comparison: Baseline vs Best GRU

To evaluate the effectiveness of the GRU forecasting model, its performance was compared against a persistence baseline across all 24 forecast horizons. The persistence model assumes that future temperature remains equal to the most recent observed value, making it a simple but relevant benchmark for short-term forecasting.

Figure 80 shows the Mean Absolute Error (MAE) in degrees Celsius for each horizon from 1 to 24 hours ahead. At the shortest horizon, both models achieve similar performance, which is expected since very near-future temperatures are often close to the most recent observation.

However, as the forecasting horizon increases, the persistence baseline deteriorates rapidly, with the MAE rising sharply and remaining substantially higher than that of the GRU model. This behavior reflects the inability of the naive baseline to capture the temporal evolution of temperature across the day.

In contrast, the GRU model maintains consistently lower error across the full forecasting horizon. Although its error also increases gradually with horizon, the increase is smooth and much more controlled. This indicates that the GRU successfully learns meaningful temporal dependencies from the historical meteorological observations and can generate more reliable multi-step forecasts.

The comparison demonstrates that the GRU architecture provides a substantial improvement over a naive persistence strategy, particularly for medium- and longer-range horizons, where modelling temporal dynamics becomes essential.

The increasing performance gap between the two models as the horizon expands highlights the added value of learned temporal representations for multi-step forecasting.

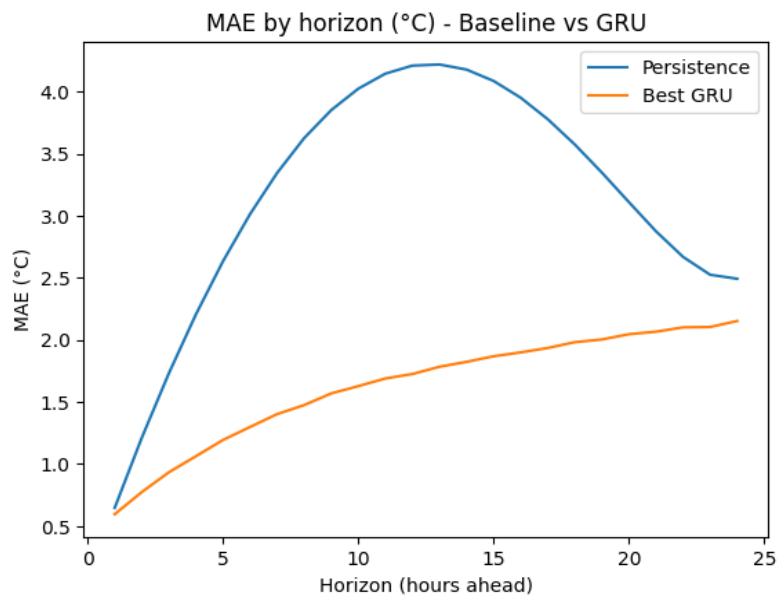


Figure 80 - Horizon-wise MAE Comparison: Baseline vs Best GRU

7.2 Error Distribution (Residual Analysis)

To further analyze the forecasting performance of the GRU model, the distribution of prediction residuals on the test set was examined (figures 81 – 82). Residuals are defined as the difference between the true temperature values and the predicted values produced by the model.

The residual histogram is approximately centered around zero, indicating that the model does not exhibit strong systematic bias in its predictions. This suggests that the GRU model is not consistently overestimating or underestimating temperature values across the dataset.

The shape of the distribution is roughly symmetric and resembles a bell-shaped curve, which is typical of well-behaved regression residuals. Most residuals are concentrated close to zero, indicating that most predictions deviate only slightly from the true temperature values.

The absolute error distribution provides additional insight into the magnitude of prediction errors. The histogram shows that most absolute errors are relatively small, with a large proportion of predictions falling to a few degrees Celsius of the true value. Larger errors occur less frequently and appear in the tails of the distribution.

Overall, the residual analysis confirms that the GRU model produces stable and generally accurate predictions, with most errors remaining within a reasonable range for short- to medium-term temperature forecasting.

Most absolute prediction errors are concentrated below approximately 3 °C, with only a small number of larger deviations.

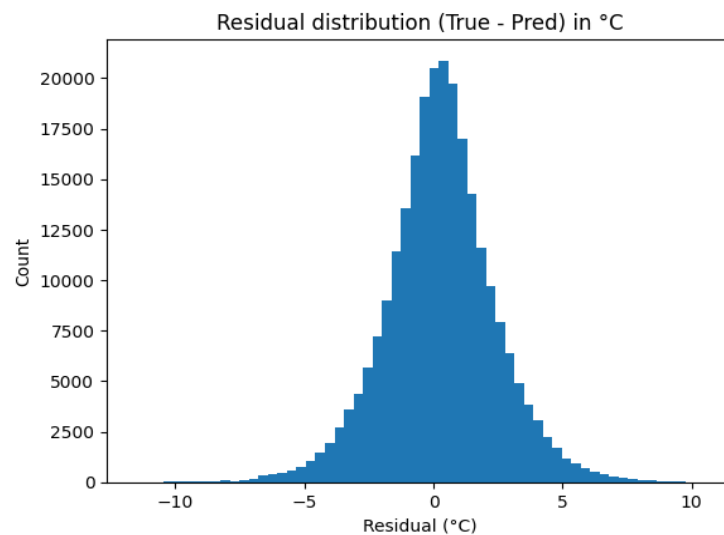


Figure 81 - Distribution of residuals (True – Predicted) in degrees Celsius (°C)

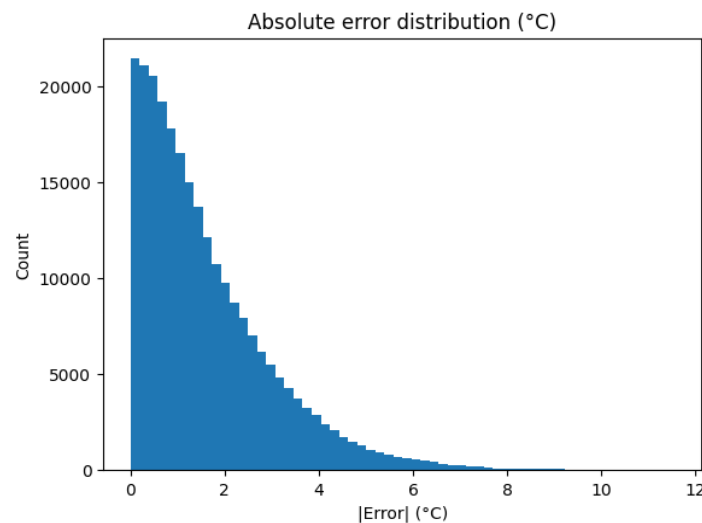


Figure 82 - Distribution of absolute prediction errors in degrees Celsius (°C)

7.3 One-Step-Ahead Forecast Visualization

To visually assess the quality of the GRU forecasts, a one-week slice of the test set was plotted, comparing the true temperature values with the corresponding 1-hour-ahead predictions (figure 83).

The figure shows that the predicted series closely follows the temporal dynamics of the observed signal. The model successfully captures the main oscillatory behavior of the temperature series, including most local increases, decreases, peaks, and troughs.

Although small deviations can be observed at some turning points, the overall agreement between predicted and true values remains strong. In particular, the model tends to slightly smooth some of the sharpest fluctuations, which is a common behavior in neural forecasting models.

This visual comparison confirms that the GRU model provides accurate short-term forecasts and can track the local evolution of temperature with high fidelity.

The close alignment between the two curves supports the quantitative results previously obtained for the short forecasting horizons.

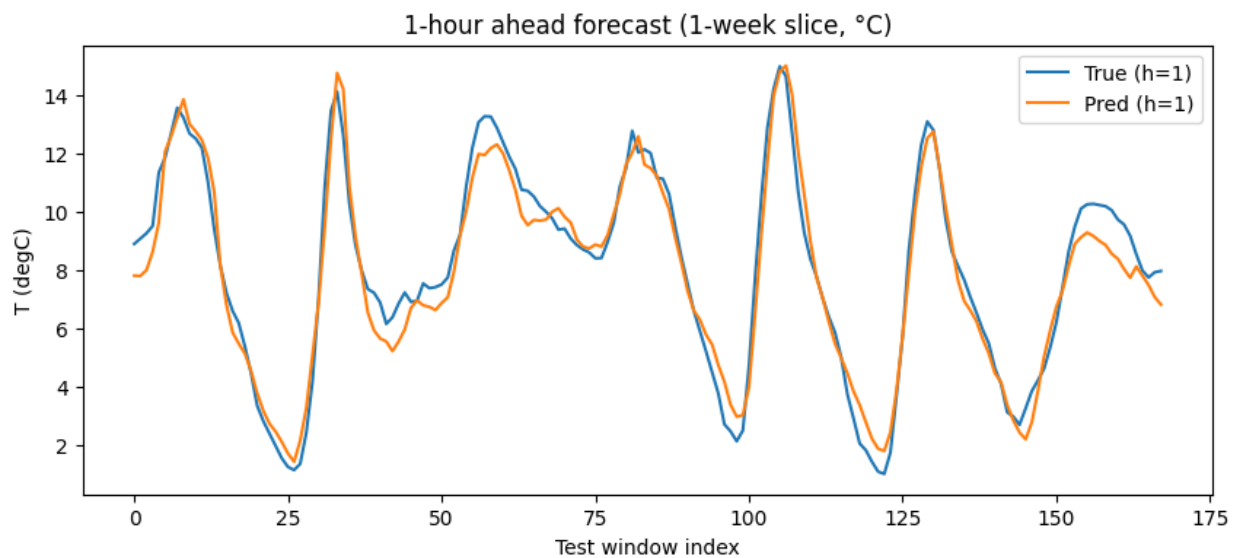


Figure 83 - One-Step-Ahead Forecast: True vs Predicted Temperatures

7.4 Multi-Step Forecast Trajectories

To better understand the model's behavior across longer prediction horizons, several 24-hour forecast trajectories were visualized for different examples in the test set.

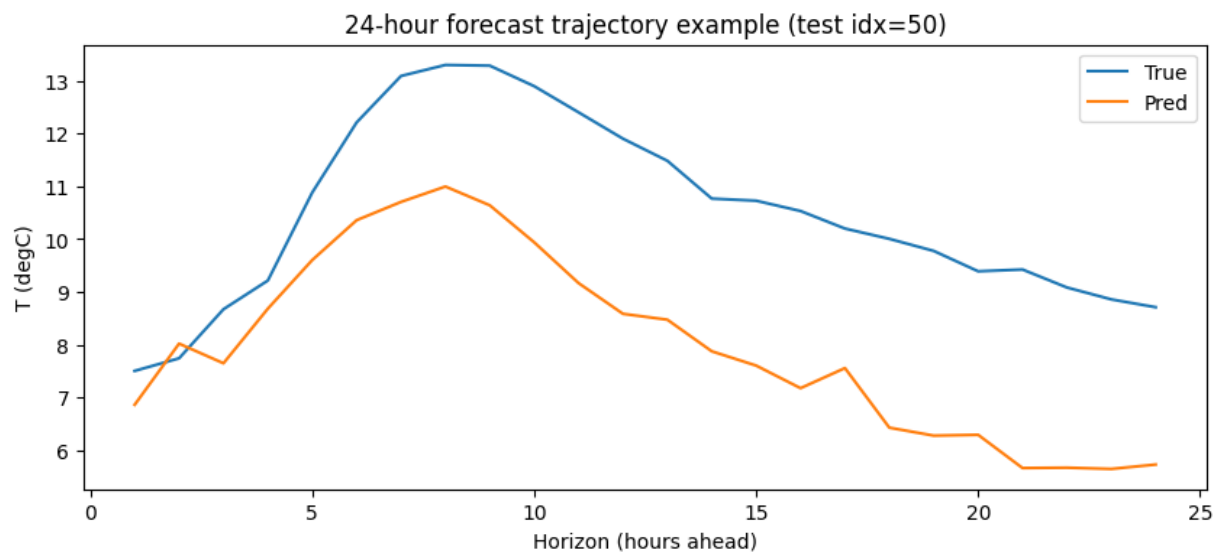
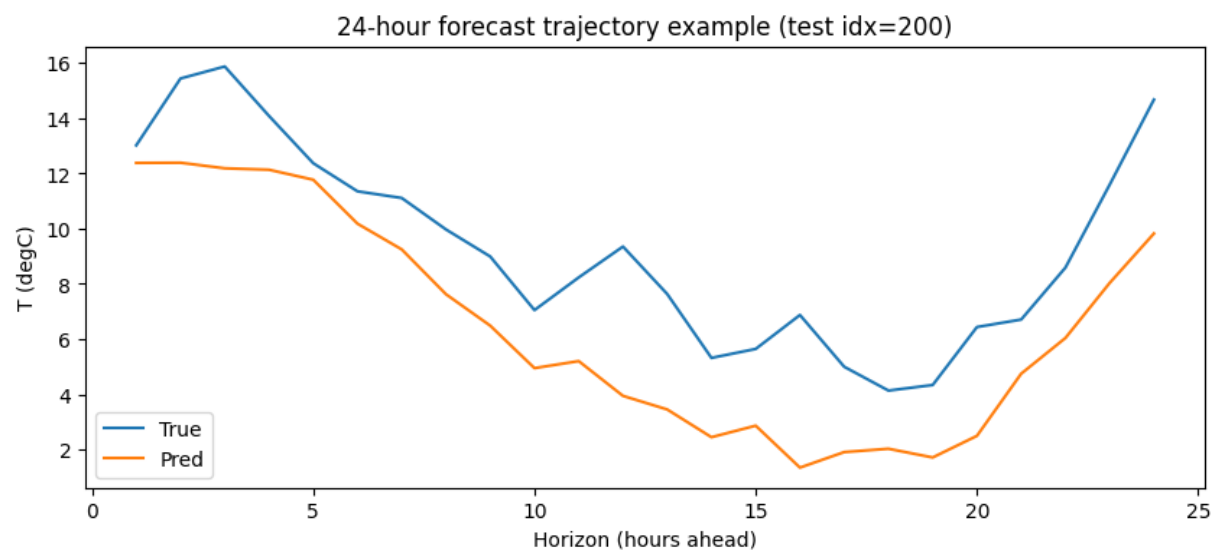
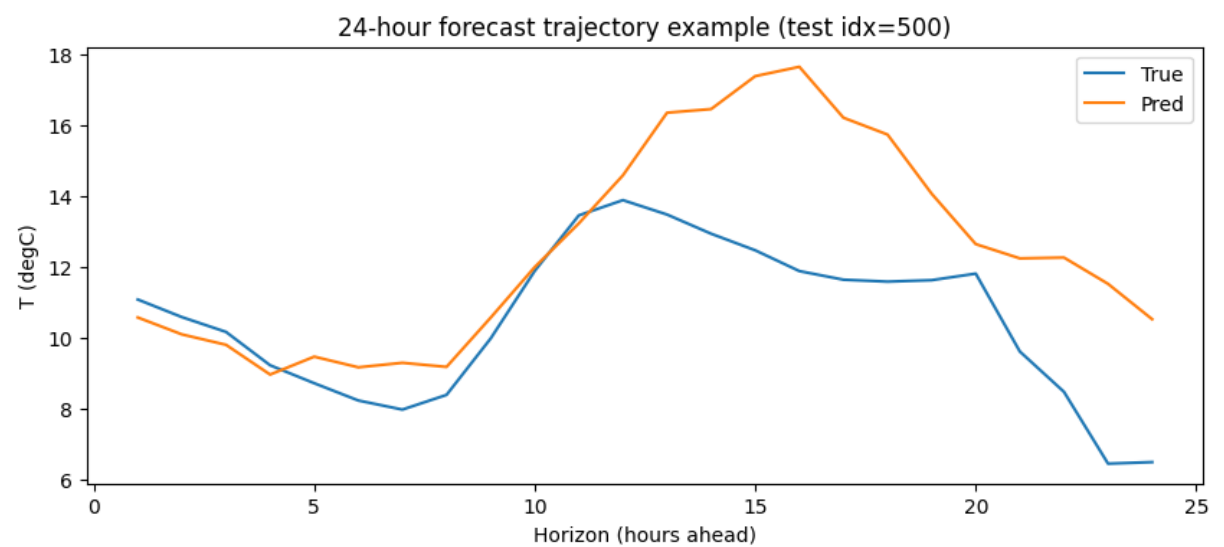
Each plot compares the true temperature evolution with the sequence of predictions produced by the GRU model for the full 24-hour forecasting horizon. These examples illustrate how the model performs when predicting multiple future time steps simultaneously.

Across the different cases, the predicted trajectories generally follow the overall trends of the true temperature curves. The model successfully captures the broad evolution of temperature, including rising and falling patterns over the forecast horizon.

However, deviations between predicted and true values become more noticeable as the horizon increases. In some examples, the model slightly underestimates temperature peaks or smooths sharper transitions in the true signal. This behavior is common in multi-step forecasting tasks, where prediction uncertainty accumulates as the forecast extends further into the future.

Despite these deviations, the GRU model consistently reproduces the main temporal dynamics of the temperature evolution, indicating that it has learned meaningful patterns from the historical meteorological data.

These visual results are consistent with the quantitative evaluation presented earlier, where prediction error increases gradually with the forecast horizon but remains within a reasonable range (figures 84 – 86).

**Figure 84** - Example of a 24-hour forecast trajectory (test index = 50)**Figure 85** - Example of a 24-hour forecast trajectory (test index = 200)**Figure 86** - Example of a 24-hour forecast trajectory (test index = 500)

7.5 Learning Curves of the Best Configurations

To analyze the training dynamics of the most promising configurations, the learning curves of the top-performing models were examined (figures 87 – 89). These curves show the evolution of both training loss and validation loss over the training epochs.

Across all three configurations, the training loss decreases rapidly during the first few epochs, indicating that the model quickly learns the main temporal patterns present in the data. After this initial phase, the rate of improvement slows and the curves gradually converge.

The validation loss follows a similar trend and stabilizes after approximately 12–15 epochs, suggesting that the model reaches its optimal performance relatively early during training. Beyond this point, additional epochs provide only marginal improvements.

A small gap between training and validation loss can be observed toward the later epochs. This behavior is typical in neural network training and indicates mild overfitting, but the gap remains relatively small, suggesting that the model still generalizes well to unseen data.

Learning curves demonstrate stable and well-behaved training dynamics for the selected GRU configurations, confirming that the chosen hyperparameters enable efficient optimization without severe overfitting.

The similarity between the curves of the top configurations also indicates that the optimization process converged to a stable region of the hyperparameter space.

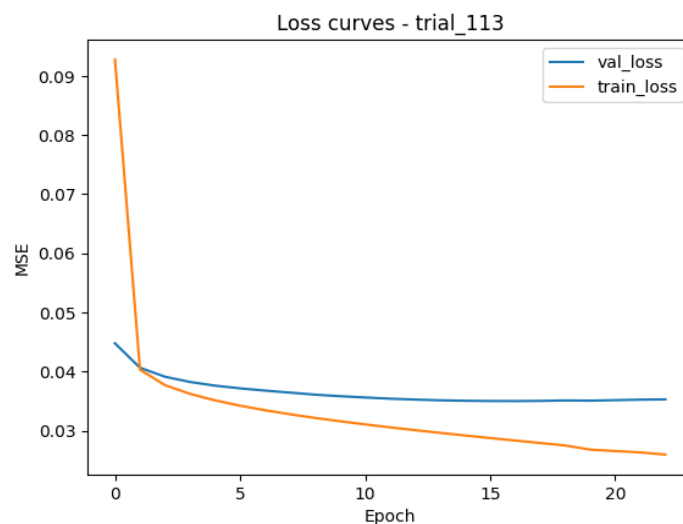


Figure 87 - Training and validation loss curves (trial 113)

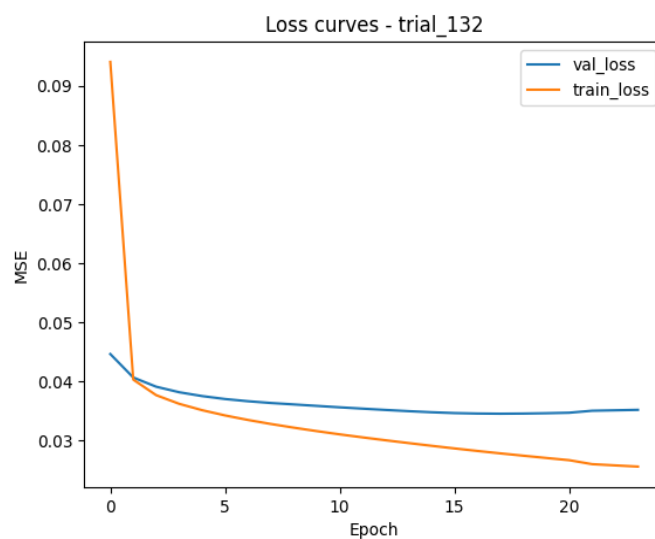


Figure 88 - Training and validation loss curves (trial 132)

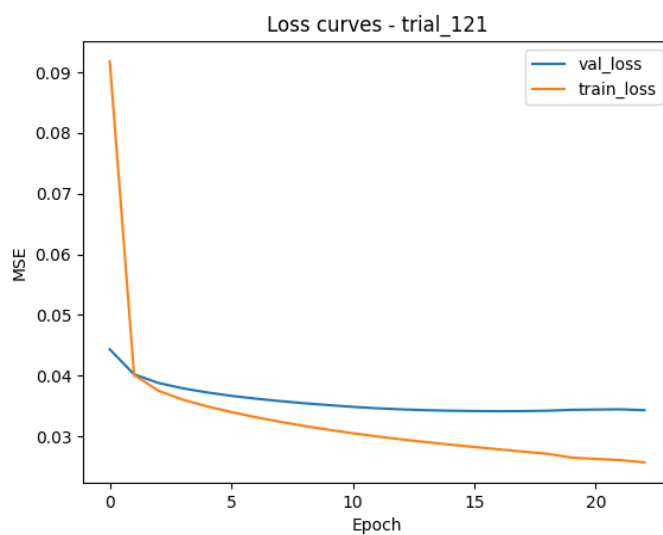


Figure 89 - Training and validation loss curves (trial 121)

8. Forecasting

Beyond quantitative evaluation, an important objective of time-series forecasting models is their ability to generate future predictions from the most recent available observations. In practical settings, forecasting systems are applied to the latest measured data to estimate the future evolution of the target variable.

This section illustrates how the trained GRU model can be used for inference. Using the last available observation window from the dataset, the model generates a direct 24-hour temperature forecast, representing the expected evolution of air temperature over the next day.

8.1 Generating a 24-Hour Temperature Forecast

To generate a future forecast, the final input window of length $L = 120$ hours was extracted from the most recent observations in the cleaned hourly dataset. The same feature engineering steps used during training were applied, including the cyclical temporal covariates and wind direction encoding.

The resulting input window was then transformed using the previously fitted scaler, ensuring consistency between training and inference. The scaled input was provided to the trained GRU model, which produced a sequence of 24 temperature predictions corresponding to the next 24 hours (table 20) (figure 90).

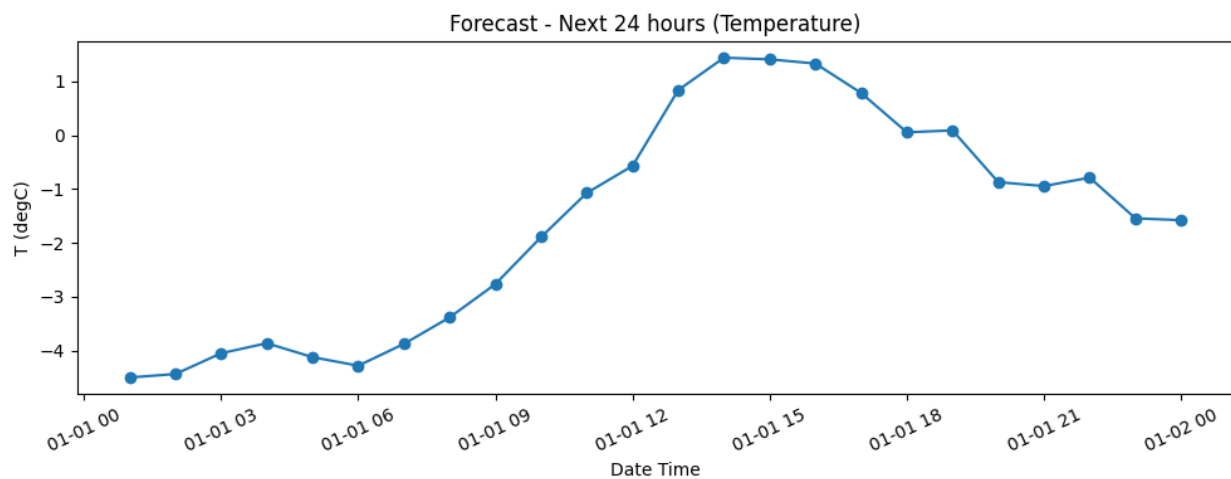
Because the model outputs predictions in the scaled space, the forecast was transformed back to degrees Celsius using the inverse of the target scaling transformation. The resulting values were then associated with their corresponding future timestamps.

The forecasted trajectory shows a plausible daily temperature cycle, with colder values during the early hours, a gradual increase toward the afternoon, and a subsequent decline during the evening. This behavior is consistent with the daily periodic structure previously identified during the exploratory data analysis.

It should be noted that this forecast extends beyond the time span covered by the dataset. Therefore, it is presented as an illustration of the model's inference capability rather than as an additional quantitative evaluation.

Date Time	T_forecast_degC
2017-01-01 01:00:00	-4.501
2017-01-01 02:00:00	-4.438
2017-01-01 03:00:00	-4.051
2017-01-01 04:00:00	-3.865
2017-01-01 05:00:00	-4.124
2017-01-01 06:00:00	-4.282
2017-01-01 07:00:00	-3.877
2017-01-01 08:00:00	-3.384

Date Time	T_forecast_degC
2017-01-01 09:00:00	-2.765
2017-01-01 10:00:00	-1.891
2017-01-01 11:00:00	-1.095
2017-01-01 12:00:00	-0.569
2017-01-01 13:00:00	0.835
2017-01-01 14:00:00	1.454
2017-01-01 15:00:00	1.412
2017-01-01 16:00:00	1.334
2017-01-01 17:00:00	0.787
2017-01-01 18:00:00	0.056
2017-01-01 19:00:00	0.091
2017-01-01 20:00:00	-0.870
2017-01-01 21:00:00	-0.943
2017-01-01 22:00:00	-0.787
2017-01-01 23:00:00	-1.543
2017-01-02 00:00:00	-1.578

Table 20 - Forecast Results for 24 Hours**Figure 90** - 24-Hour Temperature Forecast

8.2 Five-Day Forecasting

To further demonstrate the practical forecasting capability of the trained model, a longer prediction horizon is considered by generating a five-day temperature forecast (120 hours). Since the GRU model is trained to predict the next 24 hours, a rolling forecasting strategy is used to iteratively extend the prediction horizon.

At each step, the model receives the most recent input window and predicts the next 24 hours of temperature. The predicted values are then appended to the simulated dataset and

incorporated into the input window used for the following prediction step. This recursive procedure is repeated until the full five-day horizon is obtained.

For the remaining meteorological variables used as inputs (such as pressure, humidity and wind-related measurements), future values are approximated using observations from a previous time lag. This approach leverages the quasi-periodic behavior commonly observed in meteorological data and provides a reasonable approximation of unknown future conditions.

The resulting forecast represents a continuous multi-day temperature trajectory generated by the GRU model. The predicted sequence exhibits a clear daily cycle, with alternating temperature peaks and troughs that reflect the diurnal pattern previously observed in the dataset.

It is important to note that uncertainty increases as the forecasting horizon grows. Because the model recursively uses its own predictions as inputs, small prediction errors can accumulate over time, potentially leading to larger deviations in longer horizons. Despite this limitation, the model is still able to reproduce the general structure of the temperature dynamics over several consecutive days.

Figure 91 shows the five-day temperature forecast generated using the rolling prediction strategy.

8.2.1 Five-Day Temperature Forecast

To extend the forecasting horizon beyond the initial 24-hour prediction window, a rolling forecasting strategy is applied to generate a five-day temperature forecast. The model iteratively predicts the next 24 hours and then incorporates these predictions into the input sequence to produce subsequent forecasts (figure 91).

The resulting predictions are combined to form a continuous temperature trajectory covering the next 120 hours (5 days). This approach allows the model to simulate longer-term forecasts while maintaining the same temporal structure used during training.

The figure below illustrates the predicted evolution of temperature over the five-day horizon. The forecast exhibits smooth temporal patterns that resemble realistic daily temperature cycles, indicating that the GRU model captures meaningful temporal dynamics in the data.

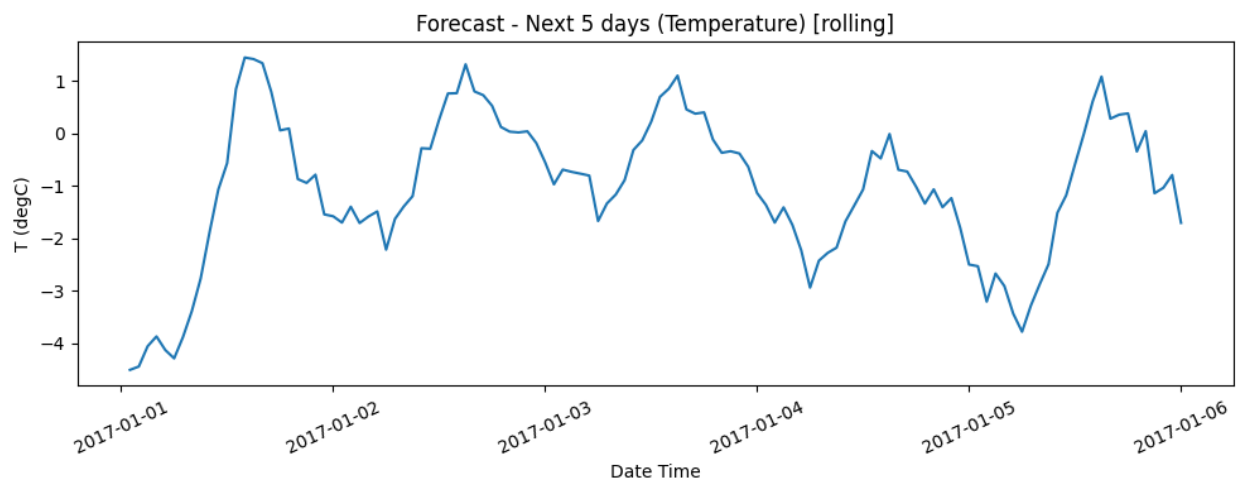


Figure 91 - Five-Day Temperature Forecast

9. Transformer

9.1 Reproducibility Setup

Transformer v3 configuration changes from v2:

- **Pruner fix:** `min_resource=5` ensures every trial survives past the 5-epoch warmup before pruning kicks in.
- **Softer pruning:** `reduction_factor=3` keeps top 50% per bracket instead of 33%.
- **Focused search space** locked in patterns that won in both v1 and v2 (huber1, adamw, noise=0.05).
- **max_epochs=60:** gives the cosine schedule more room to converge.

9.2 Model Architecture and Hyperparameters

A Transformer-based forecasting model was implemented following a PatchTST-style design, adapted for multivariate time-series forecasting. Unlike the GRU model, which processes the full sequence recurrently, the Transformer learns temporal dependencies through self-attention mechanisms, allowing it to model both short- and long-range interactions in the input sequence.

The model receives an input window of length $L = 120$ hours and produces a direct forecast for the next $H = 24$ hours.

Patch-based Input Representation

Instead of processing all time steps individually, the input sequence is divided into non-overlapping patches of fixed size. Each patch groups several consecutive time steps and is projected into a latent embedding space. This reduces the number of tokens processed by the attention layers and helps the model focus on higher-level temporal patterns while also reducing noise.

Given the configuration used here, the original input sequence is transformed into a sequence of patch embeddings, which are then used as tokens for the Transformer encoder.

Positional Encoding

Since self-attention is permutation-invariant, sinusoidal positional encoding is added to the patch embeddings to preserve temporal order. This allows the model to distinguish between earlier and later parts of the input window.

Transformer Encoder Blocks

The core of the model consists of several stacked Transformer encoder blocks. Each block contains:

- Multi-head self-attention, which enables the model to learn dependencies between different temporal regions of the input sequence.
- Feed-forward layers, which apply non-linear transformations to the learned representations.
- Residual connections and Layer Normalization, which improve optimization stability and gradient flow.

The architecture uses pre-LayerNorm formulation, where normalization is applied before the attention and feed-forward sublayers. This design is known to improve training stability in deeper Transformer models.

Output Representation

After the encoder blocks, the representation of the last patch token is selected and passed to the prediction head. This representation is then mapped to the final forecast vector of length 24, corresponding to the full multi-step forecasting horizon.

An optional dense projection layer can also be included before the output layer, enabling additional non-linear transformations.

Training Configuration

To stabilize optimization, the model uses:

- Gradient clipping
- Optional Gaussian input noise
- Configurable dropout
- Optional Adam or AdamW optimization
- Selectable loss functions (MSE, MAE, or Huber)

In addition, training uses a warmup + cosine decay learning-rate schedule, where the learning rate first increases linearly during the warmup phase and then decreases smoothly

following a cosine schedule. This strategy is particularly useful for Transformer models, which are often sensitive to learning-rate selection during the initial training stages.

9.3 Training Wrapper and Callbacks

To support systematic experimentation across multiple hyperparameter configurations, a dedicated training wrapper function was implemented for the Transformer model. This function is responsible for building the model from a sampled configuration, training it on the training set, validating it on the validation set, and returning both the trained model and its best validation metrics.

A key aspect of the training procedure is that the Transformer model uses a warmup + cosine decay learning-rate schedule built directly into the optimizer. For this reason, the number of training steps per epoch is computed dynamically from the training set size and batch size, so that the learning-rate schedule is correctly defined for each configuration.

During training, EarlyStopping is used to monitor the validation loss and stop training when no further improvement is observed for several epochs. The best model weights are restored automatically, ensuring that the final model corresponds to the best validation performance achieved during training.

Unlike the GRU training pipeline, ReduceLROnPlateau is not used here, since the Transformer already relies on an explicit warmup-cosine learning-rate schedule. Combining both approaches would introduce conflicting learning-rate updates and could destabilize optimization.

After training, the wrapper extracts the minimum validation loss and validation MAE reached during the training process. These metrics are then used to compare different Transformer configurations during hyperparameter optimization.

This modular training procedure ensures that all Transformer configurations are trained under a consistent protocol, making performance comparisons across trials fair and reproducible.

9.4 Hyperparameter Optimization with Optuna

Hyperparameter optimization for the Transformer model was performed using Optuna, following the same general optimization framework adopted for the GRU experiments. However, in this case the search space was made more focused, based on insights obtained from earlier Transformer experiments.

A custom pruning callback was implemented to report the validation loss to Optuna at the end of each training epoch. This allowed unpromising trials to be terminated early, reducing computational cost and accelerating the optimization process.

The optimization objective minimizes the validation loss, while the corresponding validation MAE is also recorded for later analysis. During the search, the best-performing model, configuration, and training history are stored whenever a new minimum validation loss is found.

Focused Search Strategy

Unlike the broader search space used in the GRU experiments, the Transformer search was restricted to the most relevant architectural and training parameters. Some design choices were fixed based on previous empirical findings:

- Optimizer: AdamW
- Loss function: Huber loss ($\delta = 1$)
- Gaussian input noise: 0.05

These parameters were kept fixed because they consistently produced strong results in previous experiments. The optimization was therefore concentrated on the remaining parameters that most directly affect Transformer behavior, such as:

- number of attention heads,
- embedding dimension,
- number of encoder blocks,
- feed-forward dimension,
- patch size,
- dropout rate,
- dense projection layer,
- learning rate,
- batch size,
- gradient clipping,
- and weight decay.

Architectural Constraints

A divisibility constraint was enforced to ensure compatibility between the embedding dimension and the number of attention heads. Specifically, the embedding size (`d_model`) was adjusted so that it is divisible by the number of heads, which is required for multi-head attention.

Training and Pruning

Each trial was trained for up to 60 epochs, using `EarlyStopping` to restore the best model weights when validation performance stopped improving. In addition, the `Optuna` pruning callback allowed poorly performing trials to be stopped early based on validation loss progression.

This optimization procedure allowed the search to focus on the most promising Transformer configurations while keeping the computational cost manageable.

9.5 Hyperparameter Optimization Run

The Transformer hyperparameter optimization was executed using Optuna with a TPE sampler and a Hyperband pruning strategy, following the same general experimental protocol adopted for the GRU model.

A total of 150 trials were launched during the optimization process. Among these, 63 trials were completed successfully, while 87 trials were pruned early. This means that approximately 58% of the trials were terminated before full training, showing that the pruning strategy substantially reduced computational cost by stopping configurations that were unlikely to outperform the current best model.

The best validation loss achieved during the search was 0.0354 (scaled MSE) (table 21). The optimization process converged relatively early, and after the best configuration was found, most subsequent trials produced validation losses in a narrow range close to the best value. This suggests that the search successfully identified a stable region of the Transformer hyperparameter space.

In addition, the average duration of completed trials was approximately 24 seconds, indicating that the Transformer architecture remained computationally efficient despite the use of self-attention and patch-based representations.

Metric	Value
Total trials	150
Completed trials	63
Pruned trials	87
Pruned proportion	58.0%
Best validation loss	0.0354
Optimization method	TPE (Bayesian)
Pruning strategy	Hyperband
Mean completed trial duration	24.0 s

Table 21 - Summary of the Transformer Hyperparameter Optimization Run

9.5.1 Best Transformer Configuration

The best-performing Transformer configuration was obtained in trial 41, achieving a validation loss of 0.0354 and a validation MAE of 0.1985 (table 22).

This configuration corresponds to a relatively compact PatchTST-style Transformer, with a small embedding dimension, a single encoder block, and a moderate patch size. The model also includes a dense projection layer before the output, which increases its capacity to model non-linear relationships before generating the final 24-hour forecast vector.

The full set of hyperparameters for the best Transformer configuration is shown in Table 22.

Hyperparameter	Best Value
Trial	41
best_val_loss	0.0354
best_val_mae	0.1985
train_time_sec	31.0
n_heads	2
d_model	64
n_blocks	1
ff_dim	128
patch_size	6
dropout	0.1
dense_units	128
dense_activation	Elu
learning_rate	0.002
batch_size	512
clipnorm	2.0
optimizer	AdamW (fixed)
weight_decay	1e-5
loss_name	Huber (fixed)
gaussian_noise_std	0.05 (fixed)

Table 22 - Best Transformer Configuration

9.5.2 Analysis of Hyperparameter Trials

Beyond identifying the best-performing configuration, it is important to analyze how different hyperparameter choices influenced Transformer performance.

A clear pattern emerges across the top-performing trials. Most of the best models share a compact architecture, typically with:

- 2 attention heads
- `d_model` = 64
- 1 encoder block
- `ff_dim` = 128 or 256
- patch sizes of 4 or 6
- `dense_units` = 128
- batch size = 512

- clipnorm = 2.0

This consistency suggests that the optimization process converged to a stable and well-defined region of the hyperparameter space.

Model Depth

The best-performing configurations consistently used only one encoder block. This indicates that, for this forecasting task, increasing Transformer depth was not necessary to achieve strong performance. Deeper models likely introduced additional complexity without corresponding gains in predictive accuracy.

Embedding Size and Attention Heads

Smaller embedding sizes, particularly $d_{\text{model}} = 64$, performed well across many trials. Similarly, 2 attention heads appeared frequently among the strongest configurations. This suggests that a lightweight attention mechanism was sufficient to capture the relevant temporal interactions in the patched input sequence.

Patch Size

The best results were obtained with patch sizes of 4 or 6, indicating that relatively short temporal patches provide a good balance between temporal resolution and noise reduction. Larger patch sizes tended to appear less often among the top-performing models.

Dropout and Regularization

Transformer configurations with low or moderate dropout generally performed best. The optimal trial used dropout = 0.1, suggesting that a small amount of regularization was beneficial, unlike the GRU experiments where dropout was not necessary.

Learning Rate and Batch Size

The best Transformer model used a comparatively high learning rate (0.002) combined with a large batch size (512). This is consistent with the use of the warmup-cosine learning-rate schedule, which stabilizes training even when the base learning rate is relatively large (table 23).

Taken together, the optimization results indicate that the Transformer performed best when configured as a small, efficient PatchTST-style model, rather than as a large and deep attention architecture.

Rank	Val Loss	Val MAE	d_{model}	Dropout	Dense	Act	LR	Batch	Weight Decay
1	0.0354	0.1985	64	0.1	128	Elu	0.002	512	1e-5
2	0.0364	0.2025	64	0.0	128	Relu	0.0005	512	1e-5
3	0.0365	0.2018	128	0.1	128	Elu	0.002	512	1e-5
4	0.0365	0.2017	64	0.0	128	Relu	0.001	512	0.0

5	0.0367	0.2017	256	0.0	128	Relu	0.001	512	0.0
6	0.0367	0.2033	64	0.0	128	Elu	0.001	512	1e-5
7	0.0368	0.2010	256	0.0	128	Relu	0.001	512	0.0
8	0.0368	0.2035	64	0.0	128	Relu	0.0005	512	1e-5
9	0.0369	0.2050	64	0.0	128	Relu	0.001	256	0.0
10	0.0369	0.2018	256	0.0	128	Relu	0.001	512	0.0

Table 23 - Top 10 trials Transformer

9.5.3 Interpretation of the Optimization Behavior

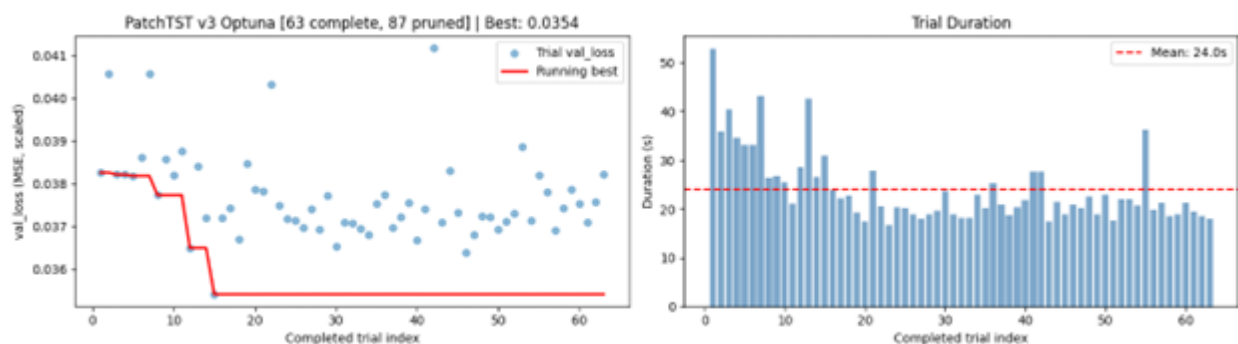
The optimization behavior of the Transformer differs slightly from that observed for the GRU model. While the GRU search converged toward moderately deep recurrent architectures, the Transformer search favored smaller and simpler attention-based models.

This suggests that, for the present dataset and forecasting setup, the Transformer does not require many encoder layers or large embedding dimensions to achieve strong performance. Instead, a compact architecture with a small number of heads and a single encoder block appears sufficient to capture the relevant temporal patterns once the input sequence is transformed into temporal patches.

The pruning rate was also higher than in the GRU search, which indicates that Transformer performance was more sensitive to poor hyperparameter combinations. At the same time, the relatively low duration of the completed trials shows that the patch-based design remained computationally efficient (figure 92).

[150/150] Trial #149 → PRUNED

Complete: 63 | Pruned: 87 | Best so far: 0.0354



Optimization complete!

Best trial #41 – val_loss: 0.0354

Best params: {'n_heads': 2, 'd_model': 64, 'n_blocks': 1, 'ff_dim': 128, 'patch_size': 6, 'dropout': 0.1, 'dense_units': 128, 'dense_activation': 'elu', 'lr': 0.002, 'batch_size': 512, 'clipnorm': 2.0, 'weight_decay': 1e-05}

Figure 92 - Transformer Hyperparameter Optimization: Validation Loss Across Trials

9.6 Learning Curves of the Best Transformer Configuration

To better understand the training behavior of the selected Transformer architecture, the evolution of the training and validation metrics was analyzed across training epochs. The learning curves of the best configuration (trial 41) are presented in Figures 93 and 94.

Training and Validation Loss

The loss curve shows a rapid decrease during the first few training epochs, indicating that the model quickly learns the main temporal structure of the data. After the initial learning phase, both training and validation loss gradually stabilize and converge toward similar values.

The relatively small gap between training and validation loss suggests that the model does not exhibit significant overfitting. Instead, the Transformer appears to generalize well with the validation data, maintaining stable performance throughout training.

Training and Validation MAE

A similar pattern can be observed in the MAE curves. The error decreases sharply during the first epochs and then gradually converges toward a stable level around 0.20 MAE (scaled). The training and validation curves remain closely aligned, further confirming that the model maintains balanced learning dynamics without severe overfitting.

Convergence Behavior

Overall, the learning curves indicate that the PatchTST-style Transformer converges efficiently during training. The combination of warmup-cosine learning rate scheduling, AdamW optimization, and Huber loss appears to provide stable optimization dynamics.

The smooth convergence of both loss and MAE metrics suggests that the selected hyperparameter configuration leads to reliable training behavior and effective learning of temporal patterns present in the meteorological time series.

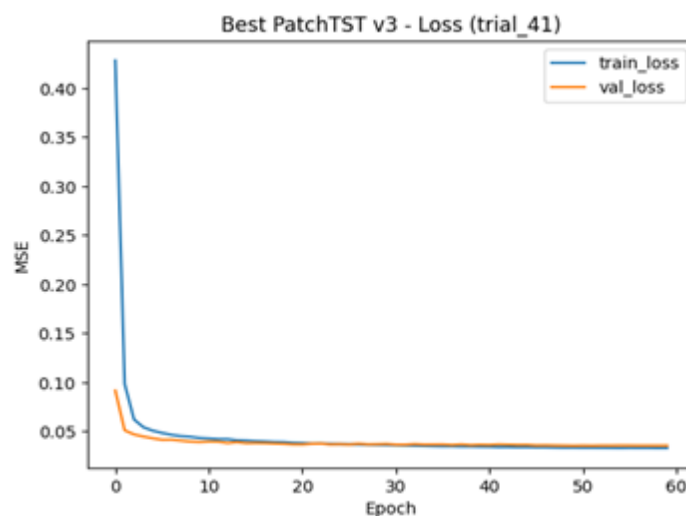


Figure 93 - Training and validation loss curves of the best PatchTST v3 model (trial 41)

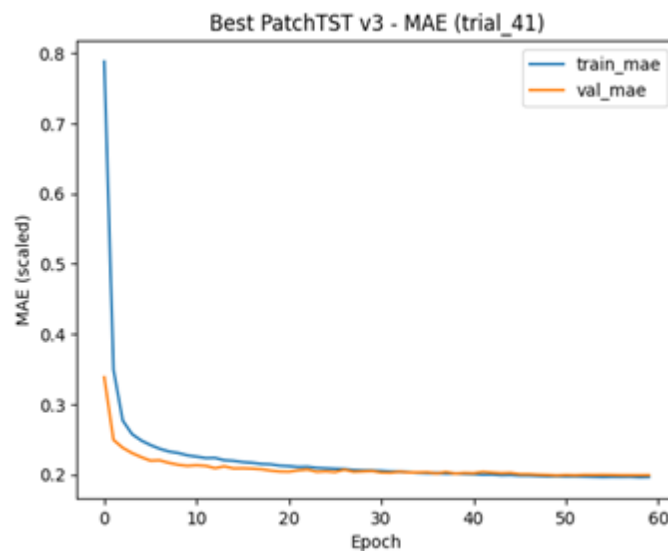


Figure 94 - Training and validation MAE curves of the best PatchTST v3 model (trial 41)

9.7 Test Set Evaluation (Scaled Space)

After identifying the best-performing Transformer configuration during hyperparameter optimization, the selected PatchTST model was evaluated on the independent test dataset. The evaluation was first performed in the scaled feature space, consistent with the representation used during model training.

Three standard regression metrics were computed: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). These metrics provide complementary perspectives on predictive performance, measuring absolute error magnitude, squared error sensitivity, and the proportion of variance explained by the model.

The PatchTST model achieved the following performance on the test set (table 24):

Metric	Value
MAE (scaled)	0.1968
RMSE (scaled)	0.2604
R^2	0.9170

Table 24 - PatchTST Test Set Performance in Scaled Feature Space

The relatively low error values and high R^2 indicate that the Transformer model can capture a substantial portion of the variability present in the temperature time series. An R^2 value above 0.91 suggests that the model explains more than 90% of the variance in the target variable, demonstrating strong predictive capability.

These results confirm that the PatchTST-style Transformer generalizes well to unseen data and can effectively model the temporal dependencies present in the meteorological dataset.

9.8 Convert Predictions to °C

Since the model was trained using standardized input features, the predicted temperature values are initially produced in the scaled feature space. To obtain interpretable evaluation metrics, both the predictions and the ground-truth target values are transformed back to the original temperature scale (degrees Celsius).

The inverse transformation is applied using the mean and standard deviation learned by the scaler during preprocessing. Specifically, the transformation reverses the standardization process applied to the target variable during training.

This step ensures that the resulting error metrics are expressed in physically meaningful units, allowing the forecasting accuracy of the model to be interpreted directly in terms of temperature differences.

9.8.1 Metrics in °C

After converting both predictions and ground-truth values back to degrees Celsius, the model performance is evaluated using the Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2).

Evaluating the model in the original temperature scale provides a more intuitive interpretation of the forecasting accuracy. In this context, the MAE indicates the average magnitude of prediction errors in degrees Celsius, while the RMSE gives additional weight to larger deviations. The R^2 metric measures how well the model explains the variance present in the observed temperature series.

On the test set, the PatchTST Transformer achieves the following performance (table 25):

Metric	Value
MAE (°C)	1.70
RMSE (°C)	2.25
R^2	0.917

Table 25 - PatchTST Test Set Performance in Degrees Celsius

These results indicate that the model can produce accurate multi-step temperature forecasts, with average errors below two degrees Celsius. The high R^2 value also suggests that the model successfully captures most of the variability present in the temperature time series.

9.9 MAE by Horizon

To further analyze the predictive performance of the PatchTST Transformer, the Mean Absolute Error (MAE) was computed separately for each forecasting horizon from 1 to 24 hours ahead (figure 95). This horizon-wise evaluation provides insight into how prediction accuracy evolves as the forecast extends further into the future.

As expected in multi-step forecasting tasks, the prediction error gradually increases with the forecast horizon. Short-term predictions exhibit the lowest error levels, while longer horizons show slightly larger deviations due to the accumulation of uncertainty over time.

The results show that the model achieves an MAE of approximately 0.6 °C for the one-hour-ahead forecast, indicating very accurate short-term predictions. As the forecasting horizon increases, the error rises smoothly, reaching values slightly above 2.2 °C for the 24-hour horizon.

Despite this gradual increase, the overall error remains within a relatively narrow and acceptable range across the entire forecast window. This behavior suggests that the Transformer model maintains stable predictive performance even for longer prediction horizons.

The smooth and monotonic increase of the MAE also indicates that the model does not exhibit sudden degradation in performance, which is often a sign of unstable multi-step forecasting. Instead, the PatchTST architecture appears to capture the temporal dynamics of the temperature series consistently across the full 24-hour prediction horizon.

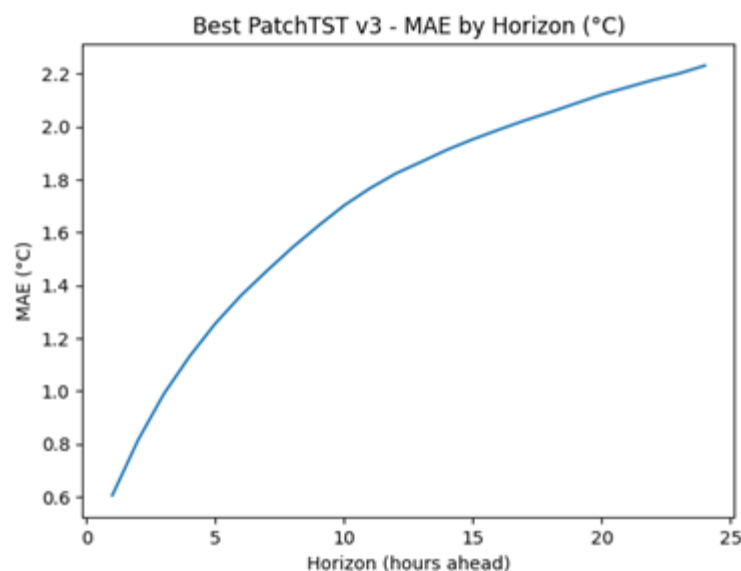


Figure 95 - PatchTST MAE by Forecasting Horizon (°C)

10. Overview of Transformer Hyperparameter Configurations

This section presents an overview of the hyperparameter configurations explored during the optimization of the PatchTST Transformer model. Each configuration corresponds to a specific trial evaluated during the Optuna-based search process.

For every configuration, several architectural and training parameters were varied, including the number of attention heads, embedding dimension (d_{model}), number of Transformer encoder blocks, feed-forward network size, patch size, dropout rate, learning rate, and batch size.

10.1 Histogram: Distribution of Validation Performance

To better understand the variability of model performance across the evaluated Transformer configurations, histograms were generated for both the best validation loss and the best validation MAE obtained during training.

The distribution of validation loss shows that most configurations achieve values within a relatively narrow interval, concentrated around approximately 0.036–0.038 (scaled MSE). This clustering indicates that many hyperparameter combinations produce similar levels of predictive performance, suggesting that the model is relatively robust within this region of the search space.

Only a small number of configurations produce substantially higher validation losses, appearing as outliers on the right side of the distribution. These cases correspond to less effective architectural or optimization settings identified during the hyperparameter search.

A similar pattern can be observed in the distribution of validation MAE. Most configurations fall within a narrow range around 0.20–0.21, while only a few trials exhibit noticeably higher error values.

These histograms indicate that the Transformer architecture demonstrates relatively stable behavior across a wide range of hyperparameter settings (figures 96 – 97). However, the presence of a few higher-error configurations highlights the importance of systematic hyperparameter optimization in identifying the most effective model configuration.

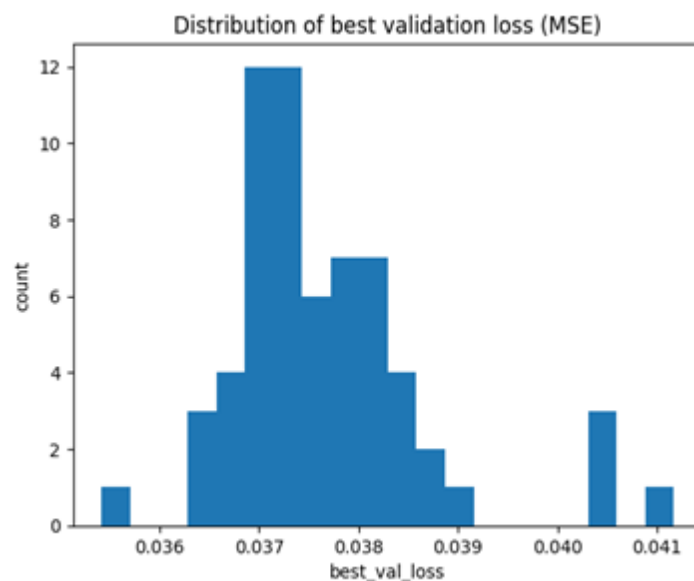


Figure 96 - Distribution of the best validation loss (MSE) across optimization trials

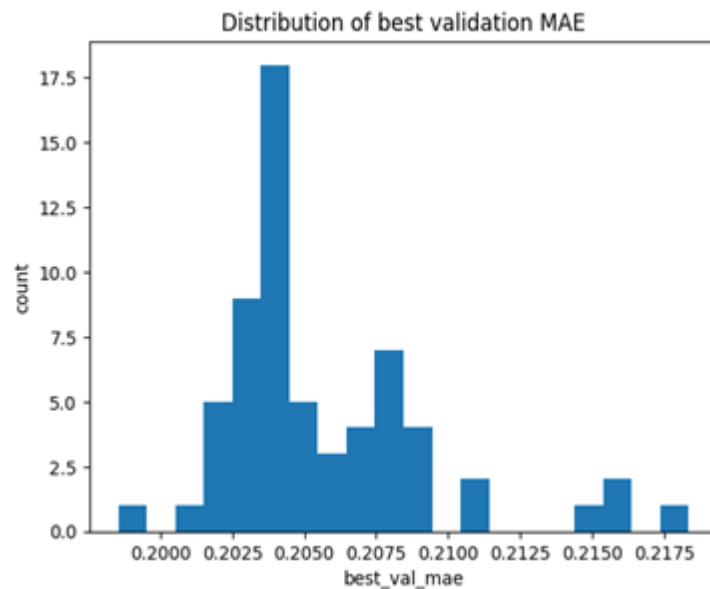


Figure 97 - Distribution of the best validation MAE across optimization trials

10.2 Hyperparameter Search Progress and Learning Rate Analysis

To better understand how the hyperparameter optimization process evolved, two complementary visualizations are presented: the search progress across trials and the relationship between validation performance and the learning rate.

Search Progress

The search progress plot illustrates how the validation loss evolves as successive configurations are evaluated during the Optuna optimization process. Each point corresponds to a completed trial, while the red line represents the best validation loss obtained up to that point in the search.

The results show that the optimization process quickly identifies several reasonably strong configurations during the early trials. A significant improvement is observed around trial 32, where the lowest validation loss is achieved. After this point, most subsequent trials produce performance values that fluctuate slightly around this optimum but do not significantly outperform the best configuration.

This behavior suggests that the optimization process successfully converged toward a stable region of the hyperparameter space, indicating that the selected search space contains multiple configurations with comparable predictive performance.

Effect of Learning Rate

The boxplot analysis (figure 98) illustrates how the validation loss varies across different learning rate values explored during the hyperparameter search.

The results indicate that learning rates around 0.0005 and 0.001 tend to produce the most consistent performance, showing relatively low median validation losses and smaller variability across trials. In contrast, larger learning rates sometimes lead to higher variability and slightly worse validation performance.

This behavior highlights the importance of selecting an appropriate learning rate when training Transformer-based models. Although the warmup-cosine learning rate schedule helps stabilize optimization, the base learning rate still plays a crucial role in determining the convergence behavior and final predictive accuracy of the model.

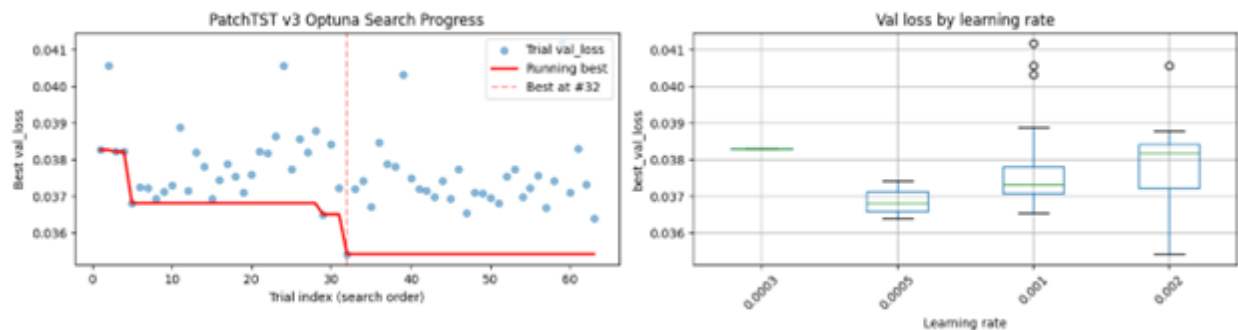


Figure 98 - PatchTST Optuna Search Progress (Validation Loss Across Trials)

10.3 Scatter: Performance vs Training Time

The scatter plot in figure 99 illustrates the relationship between training time and validation performance across the evaluated Transformer configurations.

Each point corresponds to a specific hyperparameter configuration tested during the optimization process. The horizontal axis represents the total training time in seconds, while the vertical axis indicates the best validation loss achieved by that configuration.

The results show that most configurations cluster within a relatively narrow training-time range of approximately 18 to 25 seconds, while achieving validation losses between 0.036 and 0.038. This concentration suggests that most Transformer configurations have similar computational cost and predictive performance.

Importantly, the plot indicates that longer training times do not necessarily lead to better predictive accuracy. Some configurations with longer training durations produce validation losses comparable to, or even worse than, models that train more quickly. This observation suggests that model complexity and training duration alone are not sufficient indicators of improved forecasting performance.

From a practical perspective, this analysis highlights that efficient Transformer configurations can achieve competitive predictive accuracy without requiring excessive computational cost. Such configurations represent favorable trade-offs between performance and training efficiency, which is particularly relevant in real-world forecasting applications.

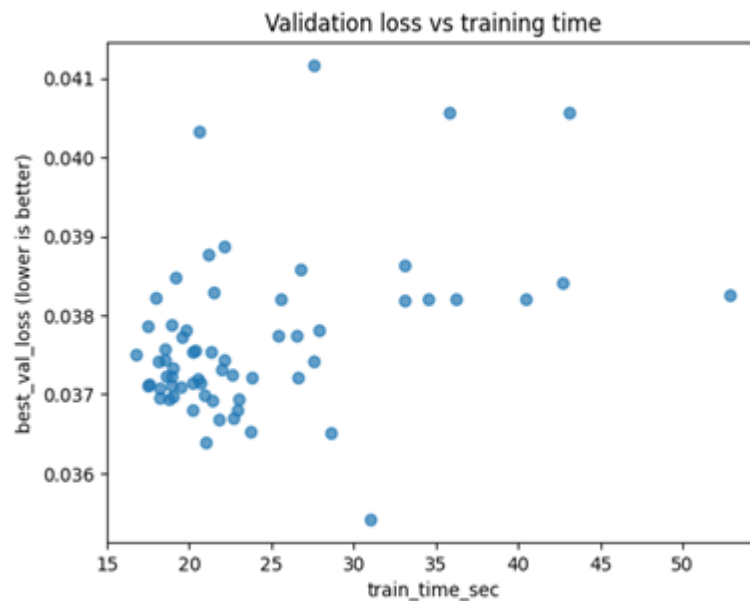


Figure 99 - Validation Loss vs Training Time for Transformer Configurations

10.4 Important Hyperparameters

To better understand how architectural choices influence the performance of the Transformer model, additional analysis was conducted focusing on key hyperparameters explored during the optimization process. In particular, the impact of the number of encoder blocks on validation performance was examined.

10.4.1 Effect of the Number of Encoder Blocks

The boxplot, in figure 100, shows how the validation loss varies across models with different numbers of Transformer encoder blocks.

The results indicate that models with a single encoder block tend to achieve the lowest validation losses and display relatively low variability across trials. This suggests that a shallow Transformer architecture is sufficient to capture the relevant temporal dependencies in the temperature time series.

Configurations with two encoder blocks generally produce slightly higher validation losses, while deeper architectures appear less frequently and do not provide clear performance improvements. Models with three or four blocks show no consistent advantage over simpler configurations.

These observations suggest that increasing the depth of the Transformer does not necessarily improve forecasting accuracy for this task. Instead, more compact architecture appears to provide a better balance between model capacity, training stability, and generalization performance.

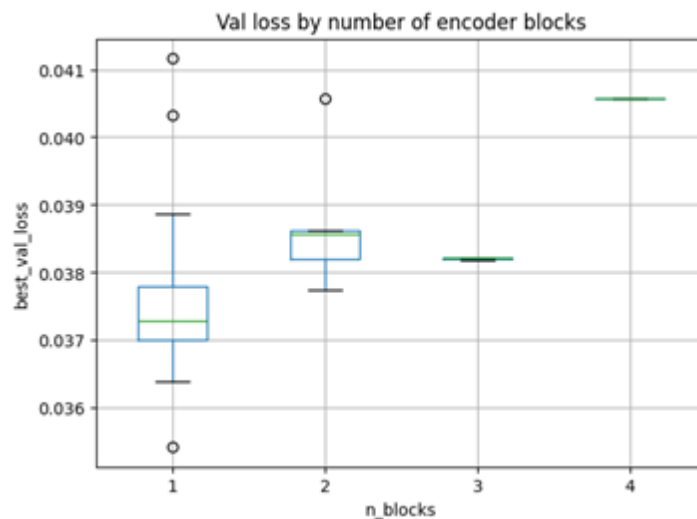


Figure 100 - Validation Loss by Number of Transformer Encoder Blocks

10.4.2 Effect of Patch Size

The boxplot in figure 101 illustrates how validation performance varies across different patch sizes used in the PatchTST Transformer architecture. The patch size determines how many consecutive time steps are grouped together to form a single token before being processed by the Transformer encoder.

Smaller patch sizes, particularly 4 and 6 hours per token, tend to produce the lowest validation losses and show relatively consistent performance across trials. These configurations preserve more fine-grained temporal information while still benefiting from the noise reduction provided by patch-based representations.

In contrast, larger patch sizes such as 8 or 12 hours per token generally lead to slightly higher validation losses. This behavior suggests that grouping too many time steps into a single patch may reduce the model's ability to capture short-term variations in the temperature signal.

The results indicate that moderate patch sizes provide the best balance between temporal resolution and representation efficiency. Using patches that are too large may oversimplify the temporal structure of the input sequence, whereas smaller patches allow the Transformer to better model short-term dynamics in the time series.

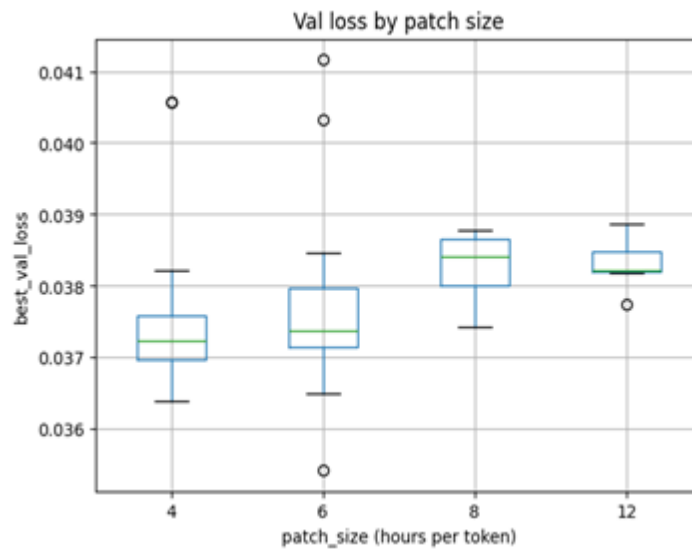


Figure 101 - Validation Loss by Patch Size in the PatchTST Transformer

10.4.3 Effect of Dropout

The boxplot in figure 102 shows how validation performance varies across different dropout rates used during training. Dropout is a regularization technique designed to reduce overfitting by randomly deactivating a fraction of neurons during each training iteration.

The results indicate that low or moderate dropout values tend to produce the best validation performance. Configurations with dropout values of 0.0 or 0.1 generally achieve the lowest validation losses and show relatively stable behavior across trials.

As the dropout rate increases, the validation loss tends to become slightly larger and more variable. Higher dropout values, such as 0.2 and 0.3, may remove too much information from intermediate representations, limiting the model's ability to learn meaningful temporal patterns from the data.

These findings suggest that only mild regularization is necessary for the Transformer architecture in this forecasting task. A small amount of dropout helps stabilize training and improve generalization, while excessive regularization can negatively affect the model's predictive capacity.

This behavior contrasts slightly with the GRU experiments presented earlier, where the best-performing configurations used no dropout at all, indicating that the recurrent architecture did not require explicit regularization. In comparison, the Transformer architecture benefits from a small amount of regularization, likely due to the higher flexibility and parameterization introduced by the self-attention mechanism.

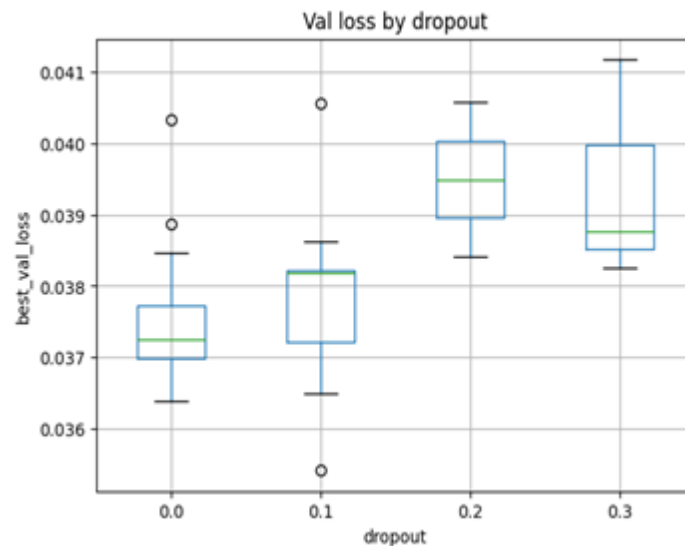


Figure 102 - Validation Loss by Dropout Rate in the PatchTST Transformer

10.4.4 Effect of the Number of Attention Heads

The boxplot in figure 103 illustrates how validation performance varies across different numbers of attention heads used in the Transformer architecture. The number of heads determines how many independent attention mechanisms operate in parallel when modelling relationships within the input sequence.

The results indicate that configurations using two attention heads tend to achieve the lowest validation losses and exhibit relatively stable performance across trials. This suggests that a small number of attention heads is sufficient to capture the relevant temporal dependencies present in the temperature time series.

Increasing the number of heads to four leads to slightly higher validation losses, while configurations with eight attention heads show greater variability and generally worse performance. Using too many heads may increase model complexity without providing additional useful information for the forecasting task.

The results suggest that a lightweight attention mechanism is sufficient for this dataset, and that increasing the number of attention heads does not necessarily improve predictive performance. Instead, simpler attention structures appear to provide a better balance between model capacity and generalization ability.

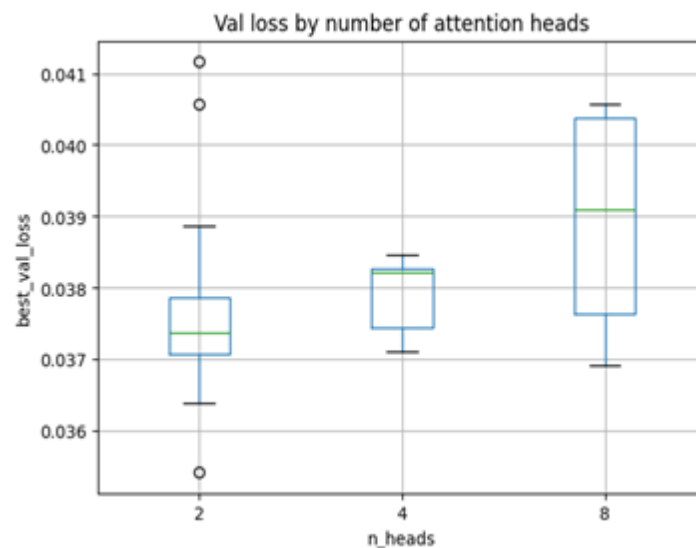


Figure 103 - Validation Loss by Number of Attention Heads in the PatchTST Transformer

10.4.5 Effect of the Embedding Dimension (d_{model})

The boxplot in figure 104 shows the relationship between the embedding dimension (d_{model}) and the validation loss obtained across different Transformer configurations. The parameter d_{model} determines the dimensionality of the vector representation used for each token processed by the Transformer encoder.

The results indicate that models with smaller embedding dimensions, particularly $d_{\text{model}} = 64$, tend to achieve the lowest validation losses and exhibit relatively stable performance across trials. This suggests that a compact representation is sufficient to capture the relevant temporal patterns present in the temperature time series.

Increasing the embedding dimension to 128 or 256 does not lead to consistent improvements in predictive performance. In fact, larger embedding sizes often introduce slightly higher validation losses and greater variability between configurations. This behavior may be explained by the increased number of parameters associated with larger embeddings, which can make optimization more difficult and potentially lead to overfitting.

These findings suggest that a relatively small embedding dimension provides an effective balance between model capacity and training stability for this forecasting task. The results reinforce the observation that compact Transformer architectures are sufficient to model the temporal dynamics of the dataset.

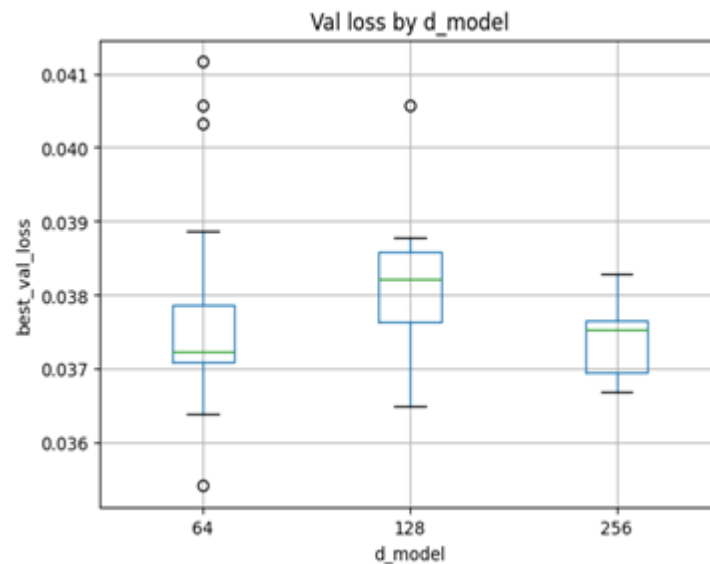


Figure 104 - Validation Loss by Embedding Dimension (d_{model}) in the PatchTST Transformer

11. Horizon-wise Comparison

To evaluate the effectiveness of the Transformer-based forecasting model, its performance is compared with a simple persistence baseline across the full 24-hour prediction horizon (figure 105). The persistence model assumes that the future temperature will remain equal to the most recently observed value, serving as a commonly used reference in time series forecasting.

The figure presents the Mean Absolute Error (MAE) in degrees Celsius for each forecast horizon from 1 to 24 hours ahead. As expected, the persistence baseline performs reasonably well for very short horizons but its error increases rapidly as the forecast horizon grows. This behavior reflects the inability of the baseline model to capture the evolving dynamics of the temperature signal over time.

In contrast, the PatchTST Transformer model maintains substantially lower errors across all horizons. While the error gradually increases with longer prediction horizons due to the accumulation of uncertainty, the increase is smooth and remains significantly below the baseline.

For example, while the persistence model reaches errors above 4 °C around the middle of the forecast horizon, the Transformer model maintains errors close to 2 °C even at longer horizons. This demonstrates that the Transformer architecture successfully learns temporal patterns and dependencies in the meteorological data, enabling it to produce more reliable multi-step forecasts.

Overall, the comparison confirms that the PatchTST model provides a clear improvement over the naive persistence baseline and can capture the underlying temporal dynamics of the temperature time series.

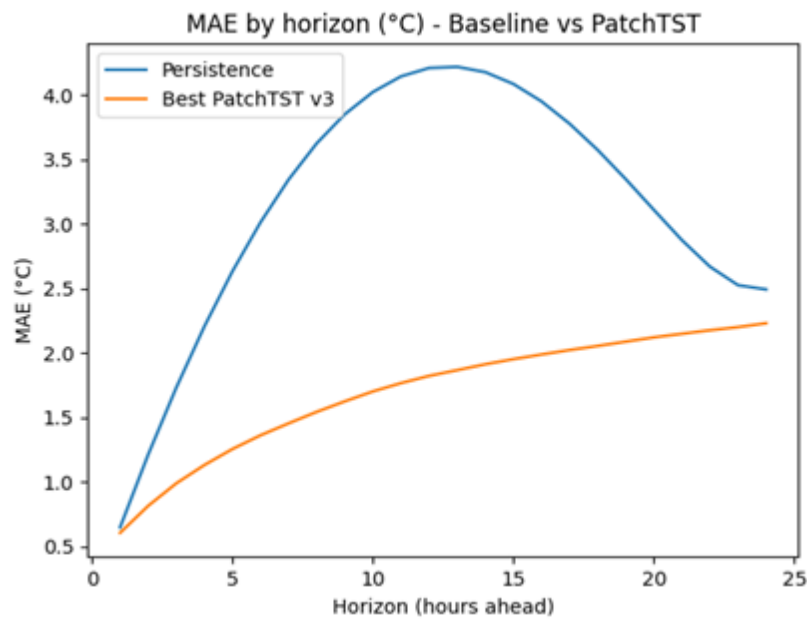


Figure 105 - Horizon-Wise MAE Comparison: Persistence Baseline vs PatchTST (°C)

11.1 Error Distribution (Residuals) on Test (°C)

To further assess the predictive behavior of the PatchTST Transformer model, the distribution of residuals on the test dataset was analyzed (figures 106 –107). Residuals are defined as the difference between the true temperature values and the predicted values generated by the model.

The residual histogram appears approximately centered around zero, indicating that the model does not exhibit a strong systematic bias. In other words, the Transformer does not consistently overestimate or underestimate the temperature values across the test set.

The distribution also shows a relatively symmetric bell-shaped pattern, suggesting that positive and negative prediction errors occur with similar frequency. Most residuals are concentrated within a relatively narrow interval around zero, which indicates that most predictions are close to the true observed temperatures.

The second figure presents the distribution of absolute prediction errors. As expected, the largest concentration of errors occurs at small values, with most predictions showing deviations of less than 2 °C. Larger errors appear less frequently and form a long but gradually decreasing tail.

These results confirm that the PatchTST model produces stable and unbiased forecasts, with most prediction errors remaining relatively small in magnitude. This behavior is consistent with the quantitative evaluation metrics obtained earlier and further supports the reliability of the Transformer model for multi-step temperature forecasting.

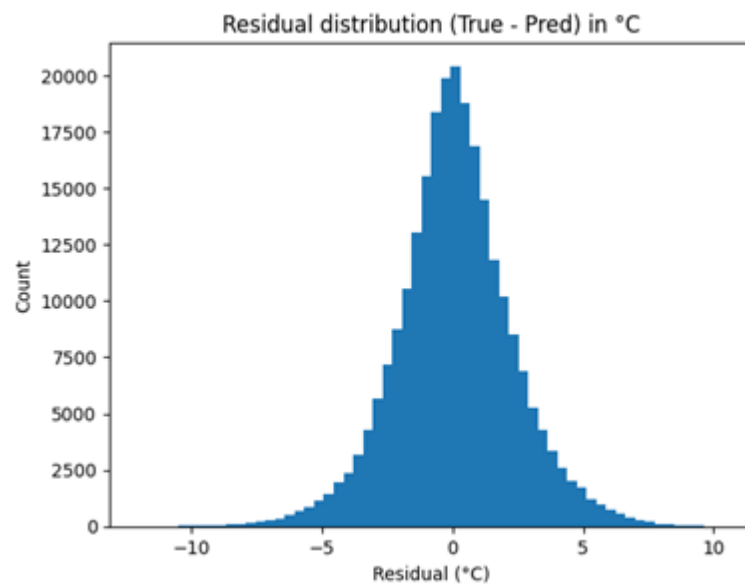


Figure 106 - Residual distribution (True – Predicted) in °C

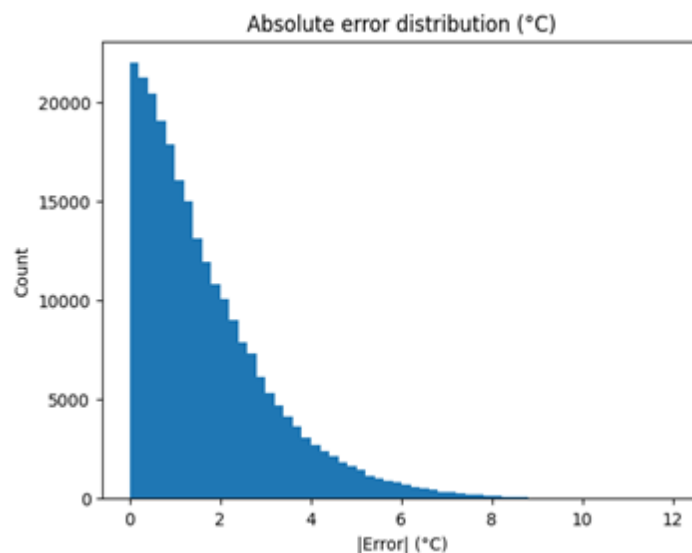


Figure 107 - Absolute error distribution in °C

11.2 Plot Actual vs Predicted for a Continuous Slice

To visually assess the short-term forecasting capability of the PatchTST Transformer, a continuous segment of the test dataset is plotted comparing the true temperature values with the model's one-hour-ahead predictions.

The figure shows a one-week slice of the test period, allowing the temporal evolution of both the observed and predicted temperature signals to be inspected. The predicted curve closely follows the overall dynamics of the true temperature series, capturing both the upward and downward trends associated with the daily temperature cycle.

In particular, the Transformer model successfully reproduces the main peaks and troughs of the temperature signal, indicating that it has learned the dominant diurnal patterns present in the meteorological data. Although small deviations between the predicted and true values can be observed, the overall trajectories remain highly consistent.

Some slight smoothing of the predicted signal is visible around sharp transitions, which is common in neural forecasting models. This behavior suggests that the model prioritizes capturing the general structure of the temperature evolution rather than reproducing every short-term fluctuation exactly.

Overall, this visual inspection confirms that the PatchTST Transformer is capable of accurately tracking short-term temperature variations and producing reliable one-step-ahead forecasts (figure 108).

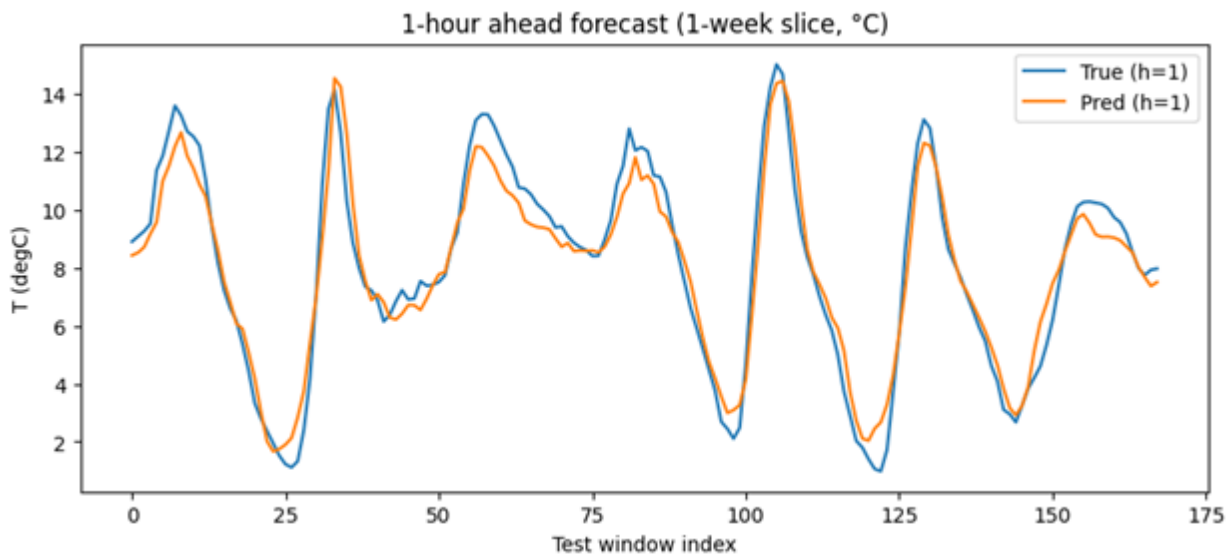


Figure 108 - Actual vs Predicted Temperature for a Continuous Test Slice (1-Hour Ahead Forecast)

11.3 Plot 24-hour Trajectories for a Few Examples

To further analyze the behavior of the Transformer model across the full forecasting horizon, several examples of 24-hour prediction trajectories from the test dataset are visualized (figures 109 – 111). Each plot compares the sequence of predicted temperatures with the corresponding true observations over the entire forecast window.

These examples illustrate how the PatchTST Transformer performs when generating multi-step forecasts, where all 24 future values are predicted simultaneously from the input window.

Across the different cases shown, the predicted trajectories generally follow the overall shape of the true temperature evolution. The model successfully captures the main upward and downward trends associated with daily temperature cycles, demonstrating that it has learned meaningful temporal dependencies in the data.

As expected, deviations between predicted and true values become more noticeable toward the later forecast horizons. This behavior is typical in multi-step forecasting, where prediction uncertainty accumulates as the model extrapolates further into the future.

In some cases, the predicted trajectories appear slightly smoother than the observed signal. This smoothing effect is common in neural forecasting models and reflects the model's

tendency to prioritize capturing the general structure of the temperature dynamics rather than reproducing every short-term fluctuation.

These examples confirm that the Transformer model can generate coherent multi-step temperature forecasts while maintaining a realistic representation of the underlying temporal patterns.

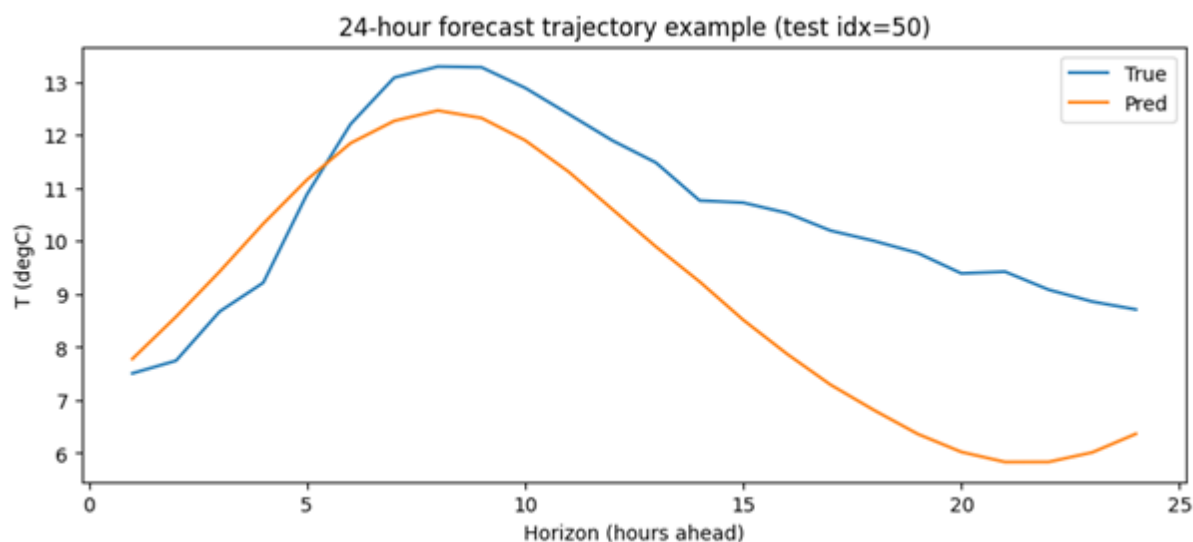


Figure 109 - Example of a 24-hour forecast trajectory (test index = 50)

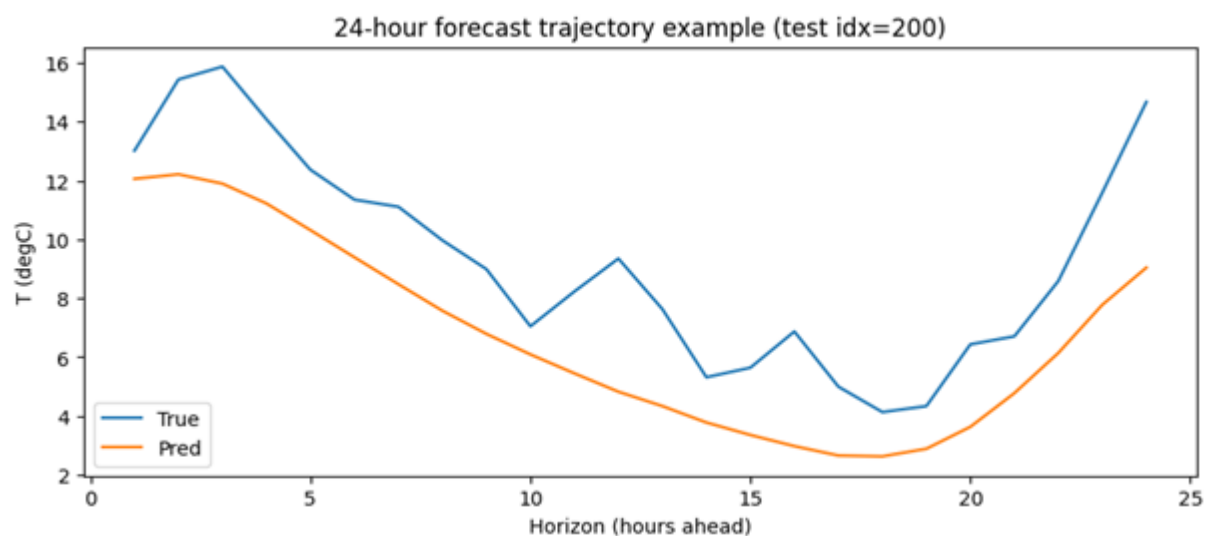


Figure 110 - Example of a 24-hour forecast trajectory (test index = 200)

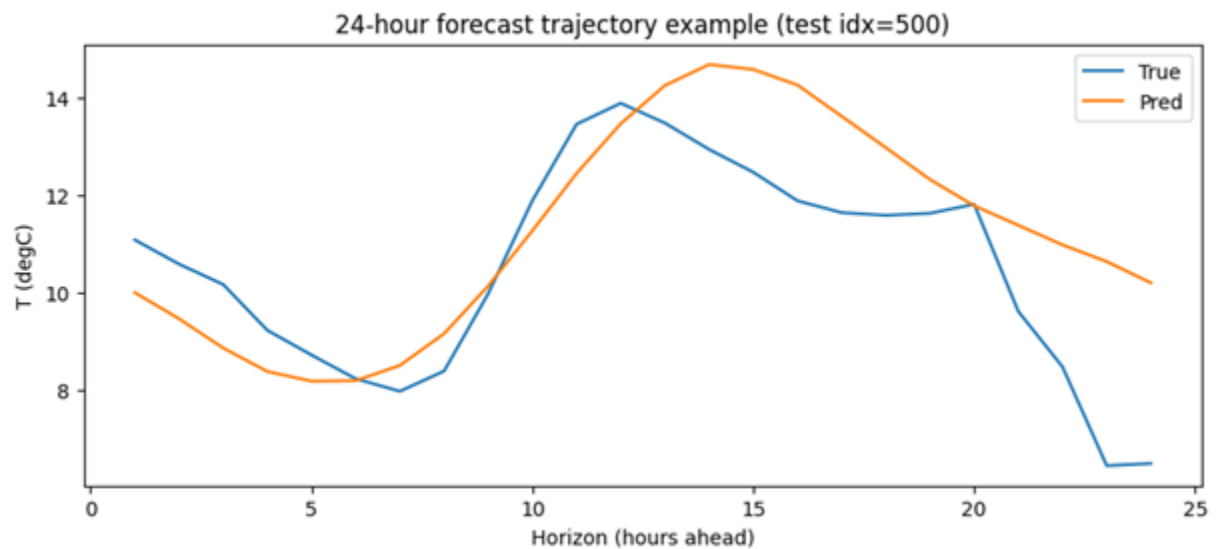


Figure 111 - Example of a 24-hour forecast trajectory (test index = 500)

12. Forecasting

Beyond evaluating predictive performance on historical data, an important objective of time series models is their ability to generate forecasts for future time periods. In practical applications, forecasting models are used to estimate upcoming values of the target variable based on the most recent observations available.

In this section, the trained PatchTST Transformer model is applied to generate temperature forecasts using the latest observations in the dataset. The objective is to simulate a realistic forecasting scenario in which the model produces predictions for the next 24 hours based on the most recent historical window.

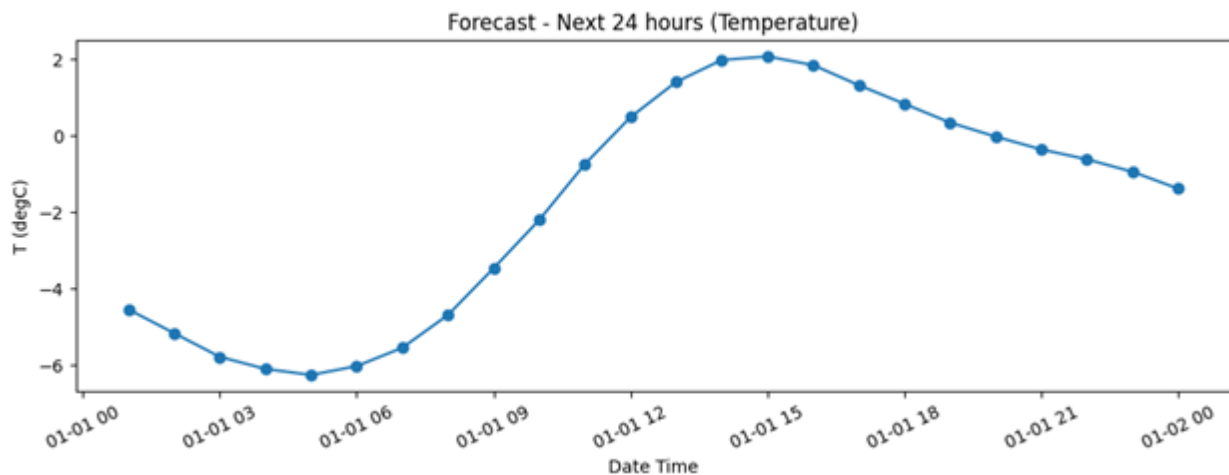
12.1 Forecasting for 1 Hour

To ensure consistency with the training procedure, the same preprocessing steps are applied to the input data. The most recent input window is first constructed from the dataset, including all features used during training. These values are then transformed using the previously fitted scaler so that the input representation matches the scaled format used by the model.

The scaled input window is then provided to the trained Transformer model, which generates a sequence of predictions corresponding to the next 24 hours. Since the model outputs predictions in the scaled feature space, the results are converted back to the original temperature scale using the inverse transformation of the scaler.

Finally, the predicted temperatures are associated with their corresponding future timestamps and visualized both as a time series plot and as a tabular representation (table 26). This forecast provides an estimate of how the temperature is expected to evolve over the next day according to the trained PatchTST model (figure 112).

Date Time	T_forecast_degC
2017-01-01 01:00	-4.541
2017-01-01 02:00	-5.162
2017-01-01 03:00	-5.780
2017-01-01 04:00	-6.097
2017-01-01 05:00	-6.254
2017-01-01 06:00	-6.023
2017-01-01 07:00	-5.543
2017-01-01 08:00	-4.687
2017-01-01 09:00	-3.466
2017-01-01 10:00	-2.203
2017-01-01 11:00	-0.828
2017-01-01 12:00	0.519
2017-01-01 13:00	1.394
2017-01-01 14:00	1.977
2017-01-01 15:00	2.069
2017-01-01 16:00	1.844
2017-01-01 17:00	1.318
2017-01-01 18:00	0.831
2017-01-01 19:00	0.342
2017-01-01 20:00	-0.027
2017-01-01 21:00	-0.361
2017-01-01 22:00	-0.620
2017-01-01 23:00	-0.946
2017-01-02 00:00	-1.386

Table 26 - Forecasted Temperature Values for the Next 24 Hours (°C)**Figure 112** - 24-Hour Temperature Forecast Generated by the PatchTST Transformer Model (°C)

12.2 Five-Day Forecasting

To further demonstrate the practical forecasting capability of the trained PatchTST Transformer, a longer prediction horizon is generated by extending the forecast to five days (120 hours) (figure 113). Since the model is trained to predict a fixed horizon of 24 hours, a rolling forecasting strategy is used to iteratively produce predictions beyond the initial forecast window.

At each step, the model receives the most recent input window and predicts the next 24 hours. The predicted temperature values are then appended to the simulated time series and used as part of the input for the next prediction step. This procedure is repeated sequentially until the full five-day horizon is obtained.

For exogenous meteorological variables included in the input features, such as pressure, humidity, or wind-related measurements, future values are approximated using observations from a previous temporal lag. This approach leverages the strong periodic patterns commonly observed in meteorological time series and provides a reasonable approximation for unknown future covariates.

The resulting forecast forms a continuous temperature trajectory over the five-day horizon. As shown in the figure, the predicted series exhibits smooth daily cycles, capturing the characteristic diurnal temperature pattern with alternating warming during the day and cooling during the night. Although uncertainty naturally increases as the forecast horizon extends further into the future, the overall structure of the predicted trajectory remains coherent and physically plausible.

This experiment illustrates how the trained Transformer model can be used to generate extended forecasts beyond the original training horizon, enabling multi-day temperature prediction in realistic forecasting scenarios.

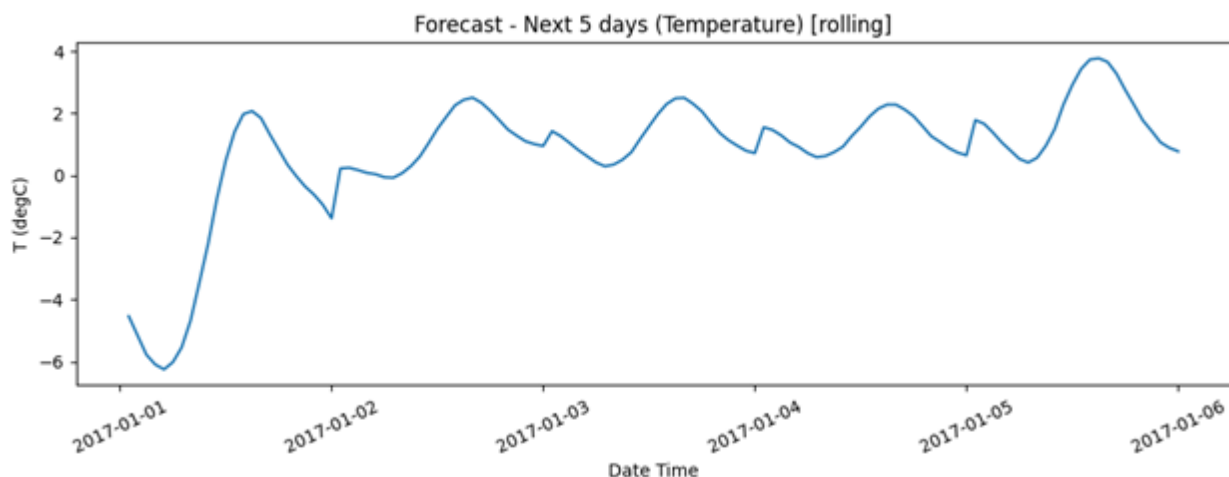


Figure 113 - Five-Day Temperature Forecast Generated Using Rolling Predictions with the PatchTST Transformer Model (°C)

13. Limitations and Future Work

Although the results obtained in this study demonstrate that deep learning models can effectively capture temperature dynamics and produce accurate short-term forecasts, several limitations should be acknowledged.

First, the experiments were conducted using data from a single meteorological site. While this allows for a controlled evaluation of the forecasting models, it may limit the generalizability of the findings to other geographical locations or climatic conditions. Weather patterns can vary significantly across regions, and model performance may differ when applied to datasets with different seasonal structures or variability.

Second, the forecasting framework relies on approximations for future values of exogenous meteorological variables when generating extended forecasts. In the multi-day forecasting scenario, these variables were estimated using lagged observations, assuming a degree of temporal periodicity in the data. Although this approach provides a reasonable proxy, it introduces additional uncertainty that may affect long-horizon predictions.

Another limitation concerns the fixed prediction horizon used during training. Both the GRU and Transformer models were trained to produce forecasts for the next 24 hours. While rolling forecasting enables longer horizons to be generated, prediction errors can accumulate as forecasts extend further into the future, which may reduce reliability in longer-term predictions.

Additionally, although extensive hyperparameter optimization was performed using Optuna, the search space was still constrained by computational resources. More exhaustive exploration of architectural variations, learning rate schedules, and regularization strategies could potentially lead to further improvements in performance.

Future research could address these limitations in several ways. One possible direction is to evaluate the models across multiple weather stations or larger spatial datasets, allowing the analysis of spatial generalization and regional weather dynamics. Another promising extension would be the implementation of probabilistic forecasting techniques, such as quantile regression or Bayesian neural networks, which could provide uncertainty estimates alongside point forecasts.

Further work could also investigate more advanced time-series Transformer architectures that have recently been proposed in the literature, such as Informer, Autoformer, or FEDformer, which are specifically designed for long-horizon forecasting. These models may offer improved scalability and performance when predicting longer time horizons.

Finally, incorporating additional sources of information, such as numerical weather prediction outputs or satellite-based measurements, could further enhance forecasting accuracy by providing richer contextual information for the models.

While the present study demonstrates the effectiveness of deep learning approaches for temperature forecasting, addressing these limitations would provide valuable insights into the robustness, scalability, and practical applicability of such models in real-world meteorological forecasting systems.

14. Conclusion

This study investigated the application of deep learning architectures for multi-step temperature forecasting using meteorological time series data. Two different neural network approaches were explored: a recurrent model based on Gated Recurrent Units (GRU) and a Transformer-based architecture inspired by the PatchTST framework. Both models were trained to predict the next 24 hours of temperature based on historical observations and additional meteorological variables.

The GRU model demonstrated strong predictive performance and proved highly effective at capturing short-term temporal dependencies in the data. On the test set, the model achieved a Mean Absolute Error (MAE) of approximately 1.63°C , a Root Mean Squared Error (RMSE) of about 2.15°C , and an R^2 score of 0.924, indicating that the model explains a large proportion of the variance in the observed temperature series. Residual analysis showed that prediction errors were approximately centered around zero and mostly concentrated near small magnitudes, indicating stable and unbiased forecasts. The GRU model also produced smooth and realistic forecast trajectories, successfully tracking short-term temperature fluctuations. One notable observation was that the GRU architecture required relatively little regularization, suggesting that its sequential inductive bias naturally helps control overfitting in this forecasting setting.

The Transformer-based PatchTST model introduced a different modeling paradigm by leveraging self-attention mechanisms and patch-based representations of the time series. Through extensive hyperparameter optimization using Optuna, the Transformer model achieved strong predictive performance and demonstrated a high ability to capture global temporal patterns.

On the scaled test set, the best configuration achieved a MAE of 0.197, RMSE of 0.260, and an R^2 score of 0.917, indicating that the model successfully captures a large proportion of the variance in the target series. After converting predictions back to the original temperature scale, the model achieved a test MAE of approximately 1.70°C and an R^2 score above 0.91, confirming strong forecasting accuracy in real-world units.

A detailed hyperparameter analysis revealed several insights into the behavior of Transformer architecture. In contrast to the GRU model, the Transformer benefited from moderate regularization, particularly using dropout and Gaussian noise. Additionally, relatively simple architectures with a small number of attention heads and encoder blocks often performed better than deeper or more complex configurations, suggesting that excessive model capacity may not be necessary for this type of meteorological forecasting task.

When compared against a persistence baseline, the Transformer model demonstrated clear improvements across all prediction horizons. Error growth over time remained controlled, indicating that the model successfully learns meaningful temporal dynamics rather than simply extrapolating recent observations.

Overall, both architectures proved capable of modeling temperature dynamics effectively, but they exhibit different strengths. In this study, the GRU model achieved slightly better

overall predictive performance, suggesting that recurrent architecture remain highly competitive for meteorological time series forecasting tasks characterized by strong local temporal dependencies and periodic patterns. The Transformer model, on the other hand, provides greater modeling flexibility and the ability to capture broader temporal relationships through attention mechanisms.

These findings highlight that modern Transformer-based approaches can be successfully applied to time series forecasting problems traditionally dominated by recurrent models. However, the results also suggest that architectural complexity should be carefully balanced with the characteristics of the dataset. For moderately sized meteorological datasets with strong periodic patterns, relatively compact models, such as GRU architectures, may already provide highly competitive or even superior forecasting performance.

15. References

Mercioni, M. A., & Holban, S. (2021). Weather Forecasting Modeling Using Soft-Clipping Swish Activation Function. IEEE International Conference on Intelligent Computer Communication and Processing

Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. Proceedings of the NIPS Deep Learning Workshop.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS).

Nie, Y., Nguyen, N., Sinthong, P., & Kalagnanam, J. (2023). A Time Series is Worth 64 Words: Long-term Forecasting with Transformers (PatchTST). International Conference on Learning Representations (ICLR).